

AHDS - SystemVerilog pour l'ingénierie Hardware & FPGA

SystemVerilog for Hardware & FPGA Engineering - RTL Design, Data Types, Interfaces, Assertions, Verification, Synthesis & Timing - FR / EN
500 questions/réponses orientées entretien et pratique. Le document est vendor-independent: il couvre SystemVerilog pour le RTL synthétisable et la verification, sans dépendre d'un outil FPGA ou simulateur particulier.

Approche: relier chaque construction SystemVerilog à son rôle réel - hardware synthétisé, comportement de simulation ou infrastructure de verification. Les fragments de code sont inclus lorsqu'ils clarifient le RTL, le scheduling, les assertions ou un piège d'entretien.

Scope: SystemVerilog moderne. Le document distingue explicitement le sous-ensemble RTL synthétisable des mécanismes de verification tels que classes, randomisation, SVA, coverage et UVM.

Structure du document / Document Structure

1. Verilog vs SystemVerilog et choix du langage / Verilog vs SystemVerilog and Language Choice
2. Structure source, modules et ports / Source Structure, Modules, and Ports
3. Types `logic`, nets, variables, arrays et littéraux / `logic` Types, Nets, Variables, Arrays, and Literals
4. Opérateurs, expressions, casts et signedness / Operators, Expressions, Casts, and Signedness
5. Logique combinatoire et tables de vérité / Combinational Logic and Truth Tables
6. Affectations continues, tri-state et dataflow / Continuous Assignments, Tri-State, and Dataflow
7. `always\_comb`, `always\_ff`, `always\_latch` et sémantique procédurale / `always\_comb`, `always\_ff`, `always\_latch`, and Procedural Semantics
8. Blocking =, non-blocking <= et scheduling / Blocking =, Non-Blocking <=, and Scheduling
9. Registres, horloge, reset et enable / Registers, Clock, Reset, and Enable
10. Compteurs, timers, diviseurs et PWM / Counters, Timers, Dividers, and PWM
11. FSM avec `enum`, états et propriétés / FSMs with `enum`, States, and Properties
12. Hiérarchie, paramètres, packages, interfaces et generate / Hierarchy, Parameters, Packages, Interfaces, and Generate
13. Fonctions, tasks, typedefs, macros et organisation du code / Functions, Tasks, Typedefs, Macros, and Code Organization
14. Arrays, mémoire, ROM, RAM et FIFO / Arrays, Memory, ROM, RAM, and FIFO
15. Arithmétique, datapaths, largeur et overflow / Arithmetic, Datapaths, Width, and Overflow
16. CDC, métastabilité et synchronisation / CDC, Metastability, and Synchronization
17. Entrées externes, debounce, pulses et événements / External Inputs, Debounce, Pulses, and Events
18. UART RTL / UART RTL
19. SPI RTL / SPI RTL
20. I2C RTL et open-drain / I2C RTL and Open-Drain
21. Testbench SystemVerilog: stimulus, interfaces, monitor et scoreboard / SystemVerilog Testbench: Stimulus, Interfaces, Monitor, and Scoreboard
22. Classes, randomisation contrainte et collections dynamiques / Classes, Constrained Randomization, and Dynamic Collections
23. SVA, assertions, couverture fonctionnelle et formal concepts / SVA, Assertions, Functional Coverage, and Formal Concepts
24. Scheduling, synthèse, lint, timing et simulation-synthesis mismatch / Scheduling, Synthesis, Lint, Timing, and Simulation-Synthesis Mismatch
25. UVM, troubleshooting et scénarios d'entretien / UVM, Troubleshooting, and Interview Scenarios

1. Verilog vs SystemVerilog et choix du langage / Verilog vs SystemVerilog and Language Choice

1. FR - Expliquez la différence entre Verilog et SystemVerilog. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : SystemVerilog étend Verilog avec un typage plus riche, `logic`, `always_comb`, `always_ff`, `enum`, `struct`, packages, interfaces, assertions, classes, randomisation et coverage. Il peut servir au RTL et à la verification. Verilog: reg [7:0] q; always @(posedge clk) q <= d; SystemVerilog: logic [7:0] q; always_ff @(posedge clk) q <= d;	EN - Explain the difference between Verilog and SystemVerilog. What important difference, structure, or idea should be understood? Answer: SystemVerilog extends Verilog with richer typing, `logic`, `always_comb`, `always_ff`, `enum`, `struct`, packages, interfaces, assertions, classes, randomization, and coverage. It can be used for both RTL and verification. Verilog: reg [7:0] q; always @(posedge clk) q <= d; SystemVerilog: logic [7:0] q; always_ff @(posedge clk) q <= d;
2. FR - Comment utiliseriez-vous la différence entre Verilog et SystemVerilog dans un projet SystemVerilog réel ? Réponse : Pour un nouveau projet, je préfère généralement SystemVerilog si le toolchain le supporte, tout en conservant la compatibilité avec les IP Verilog existantes. Verilog: reg [7:0] q; always @(posedge clk) q <= d; SystemVerilog: logic [7:0] q; always_ff @(posedge clk) q <= d;	EN - How would you use the difference between Verilog and SystemVerilog in a real SystemVerilog project? Answer: For a new project, I generally prefer SystemVerilog when the toolchain supports it while preserving compatibility with existing Verilog IP. Verilog: reg [7:0] q; always @(posedge clk) q <= d; SystemVerilog: logic [7:0] q; always_ff @(posedge clk) q <= d;
3. FR - Comment diagnostiqueriez-vous un problème lié à la différence entre Verilog et SystemVerilog ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour la différence entre Verilog et SystemVerilog, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to the difference between Verilog and SystemVerilog? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For the difference between Verilog and SystemVerilog, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

4. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant la différence entre Verilog et SystemVerilog ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour la différence entre Verilog et SystemVerilog, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding the difference between Verilog and SystemVerilog? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For the difference between Verilog and SystemVerilog, I also explain the expected hardware or simulation-only role, together with important corner cases.
5. FR - Expliquez quand choisir Verilog ou SystemVerilog. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Verilog reste courant dans les IP legacy et certains flows conservateurs. SystemVerilog est souvent plus pratique pour les nouveaux designs grâce à un RTL plus explicite et à ses outils de verification.	EN - Explain when to choose Verilog or SystemVerilog. What important difference, structure, or idea should be understood? Answer: Verilog remains common in legacy IP and conservative flows. SystemVerilog is often more practical for new designs because of more explicit RTL and stronger verification features.
6. FR - Comment utiliseriez-vous quand choisir Verilog ou SystemVerilog dans un projet SystemVerilog réel ? Réponse : Je base le choix sur le support des outils, les IP existantes, les conventions d'équipe, la maintenabilité et le niveau de verification requis.	EN - How would you use when to choose Verilog or SystemVerilog in a real SystemVerilog project? Answer: I base the choice on tool support, existing IP, team conventions, maintainability, and required verification depth.
7. FR - Comment diagnostiqueriez-vous un problème lié à quand choisir Verilog ou SystemVerilog ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour quand choisir Verilog ou SystemVerilog, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to when to choose Verilog or SystemVerilog? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For when to choose Verilog or SystemVerilog, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
8. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant quand choisir Verilog ou SystemVerilog ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour quand choisir Verilog ou SystemVerilog, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding when to choose Verilog or SystemVerilog? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For when to choose Verilog or SystemVerilog, I also explain the expected hardware or simulation-only role, together with important corner cases.
9. FR - Expliquez SystemVerilog RTL versus SystemVerilog verification. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Le sous-ensemble RTL décrit portes, registres, mémoires, FSM et interconnexions. Les classes, mailboxes, covergroups et constrained random sont principalement simulation-only.	EN - Explain SystemVerilog RTL versus SystemVerilog verification. What important difference, structure, or idea should be understood? Answer: The RTL subset describes gates, registers, memories, FSMs, and interconnects. Classes, mailboxes, covergroups, and constrained random are mainly simulation-only.
10. FR - Comment utiliseriez-vous SystemVerilog RTL versus SystemVerilog verification dans un projet SystemVerilog réel ? Réponse : Je sépare clairement `rtl` et `tb` et je précise toujours le rôle d'une construction lorsque je la présente en entretien.	EN - How would you use SystemVerilog RTL versus SystemVerilog verification in a real SystemVerilog project? Answer: I clearly separate `rtl` and `tb` and always state the role of a construct when discussing it in an interview.
11. FR - Comment diagnostiqueriez-vous un problème lié à SystemVerilog RTL versus SystemVerilog verification ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour SystemVerilog RTL versus SystemVerilog verification, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to SystemVerilog RTL versus SystemVerilog verification? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For SystemVerilog RTL versus SystemVerilog verification, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
12. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant SystemVerilog RTL versus SystemVerilog verification ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour SystemVerilog RTL versus SystemVerilog verification, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding SystemVerilog RTL versus SystemVerilog verification? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For SystemVerilog RTL versus SystemVerilog verification, I also explain the expected hardware or simulation-only role, together with important corner cases.

13. FR - Expliquez la rétrocompatibilité avec Verilog. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : La majorité du Verilog-2001 valide peut être compilée comme SystemVerilog, ce qui permet une migration progressive. De nouvelles règles et mots-clés peuvent cependant révéler des ambiguïtés de vieux code.	EN - Explain backward compatibility with Verilog. What important difference, structure, or idea should be understood? Answer: Most valid Verilog-2001 can be compiled as SystemVerilog, enabling gradual migration. New rules and keywords can still expose ambiguities in older code.
14. FR - Comment utiliseriez-vous la rétrocompatibilité avec Verilog dans un projet SystemVerilog réel ? Réponse : Je migre par étapes: compilation <code>.sv`</code>, régression identique, puis modernisation contrôlée avec <code>`logic`</code>, <code>`always_ff`</code> et types nommés.	EN - How would you use backward compatibility with Verilog in a real SystemVerilog project? Answer: I migrate incrementally: compile as <code>.sv`</code>, preserve regression behavior, then modernize carefully with <code>`logic`</code>, <code>`always_ff`</code>, and named types.
15. FR - Comment diagnostiqueriez-vous un problème lié à la rétrocompatibilité avec Verilog ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour la rétrocompatibilité avec Verilog, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to backward compatibility with Verilog? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For backward compatibility with Verilog, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
16. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant la rétrocompatibilité avec Verilog ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour la rétrocompatibilité avec Verilog, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding backward compatibility with Verilog? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For backward compatibility with Verilog, I also explain the expected hardware or simulation-only role, together with important corner cases.
17. FR - Expliquez les avantages de SystemVerilog pour un hardware engineer. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : SystemVerilog exprime mieux l'intention RTL et permet assertions et testbench avancés dans le même écosystème de langage. Il faut toutefois connaître le sous-ensemble réellement supporté par le synthétiseur.	EN - Explain SystemVerilog advantages for a hardware engineer. What important difference, structure, or idea should be understood? Answer: SystemVerilog expresses RTL intent more clearly and enables assertions and advanced testbenches in the same language ecosystem. You still need to know the subset actually supported by synthesis.
18. FR - Comment utiliseriez-vous les avantages de SystemVerilog pour un hardware engineer dans un projet SystemVerilog réel ? Réponse : Je l'utilise pour rendre les interfaces, FSM, types et règles de clocking plus explicites, puis j'emploie SVA/coverage lorsque le projet demande une vérification plus forte.	EN - How would you use SystemVerilog advantages for a hardware engineer in a real SystemVerilog project? Answer: I use it to make interfaces, FSMs, types, and clocking rules more explicit, then use SVA/coverage when stronger verification is required.
19. FR - Comment diagnostiqueriez-vous un problème lié à les avantages de SystemVerilog pour un hardware engineer ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour les avantages de SystemVerilog pour un hardware engineer, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to SystemVerilog advantages for a hardware engineer? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For SystemVerilog advantages for a hardware engineer, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
20. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant les avantages de SystemVerilog pour un hardware engineer ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour les avantages de SystemVerilog pour un hardware engineer, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding SystemVerilog advantages for a hardware engineer? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For SystemVerilog advantages for a hardware engineer, I also explain the expected hardware or simulation-only role, together with important corner cases.

2. Structure source, modules et ports / Source Structure, Modules, and Ports

21. FR - Expliquez structure minimale d'un module. Quelle structure ou idée hardware se cache derrière ? Réponse : Un module SystemVerilog commence par <code>`module`</code>, déclare son interface et se termine par <code>`endmodule`</code>. Le module est l'unité de base de hiérarchie et de réutilisation.	EN - Explain minimal module structure. What hardware structure or idea is behind it? Answer: A SystemVerilog module begins with <code>`module`</code>, declares its interface, and ends with <code>`endmodule`</code>. A module is the basic unit of hierarchy and reuse.
---	---

22. FR - Comment utiliseriez-vous structure minimale d'un module dans un design SystemVerilog réel ? Réponse : Je crée un petit bloc combinatoire avec des ports ANSI et une affectation continue. <pre>module and2(input wire a, input wire b, output wire y); assign y = a & b; endmodule</pre>	EN - How would you use minimal module structure in a real SystemVerilog design? Answer: I create a small combinational block with ANSI ports and a continuous assignment. <pre>module and2(input wire a, input wire b, output wire y); assign y = a & b; endmodule</pre>
23. FR - Comment diagnostiqueriez-vous un problème lié à structure minimale d'un module ? Réponse : Je vérifie le nom du module, sa liste de ports, directions, largeurs, déclarations et correspondance avec les instances. Une erreur de largeur ou de connexion peut rester syntaxiquement valide tout en étant fonctionnellement fautive. Pour structure minimale d'un module, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving minimal module structure? Answer: I check the module name, port list, directions, widths, declarations, and instance mapping. A width or connectivity error can remain syntactically valid while being functionally wrong. For minimal module structure, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
24. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant structure minimale d'un module ? Réponse : Je garde des interfaces explicites, des noms cohérents et des largeurs visibles. Je préfère le style ANSI-style SystemVerilog pour réduire les divergences entre la liste de ports et leurs déclarations. Pour structure minimale d'un module, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding minimal module structure? Answer: I keep interfaces explicit, names consistent, and widths visible. I prefer ANSI-style SystemVerilog to reduce mismatches between the port list and declarations. For minimal module structure, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
25. FR - Expliquez ports input, output et inout. Quelle structure ou idée hardware se cache derrière ? Réponse : 'input' reçoit un signal dans le module, 'output' l'envoie vers l'extérieur, et 'inout' modélise un signal bidirectionnel. 'inout' est surtout pertinent aux frontières I/O et exige une gestion tri-state correcte.	EN - Explain input, output, and inout ports. What hardware structure or idea is behind it? Answer: 'input' receives a signal into the module, 'output' drives a signal outward, and 'inout' models a bidirectional signal. 'inout' is mainly relevant at I/O boundaries and requires correct tri-state control.
26. FR - Comment utiliseriez-vous ports input, output et inout dans un design SystemVerilog réel ? Réponse : Pour un bus I2C SDA, un 'inout' peut représenter la ligne physique open-drain; dans le cœur du design je préfère séparer 'sda_in' et 'sda_drive_low'.	EN - How would you use input, output, and inout ports in a real SystemVerilog design? Answer: For an I2C SDA bus, an 'inout' can represent the physical open-drain line; inside the design I prefer separate 'sda_in' and 'sda_drive_low' signals.
27. FR - Comment diagnostiqueriez-vous un problème lié à ports input, output et inout ? Réponse : Je vérifie le nom du module, sa liste de ports, directions, largeurs, déclarations et correspondance avec les instances. Une erreur de largeur ou de connexion peut rester syntaxiquement valide tout en étant fonctionnellement fautive. Pour ports input, output et inout, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving input, output, and inout ports? Answer: I check the module name, port list, directions, widths, declarations, and instance mapping. A width or connectivity error can remain syntactically valid while being functionally wrong. For input, output, and inout ports, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
28. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant ports input, output et inout ? Réponse : Je garde des interfaces explicites, des noms cohérents et des largeurs visibles. Je préfère le style ANSI-style SystemVerilog pour réduire les divergences entre la liste de ports et leurs déclarations. Pour ports input, output et inout, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding input, output, and inout ports? Answer: I keep interfaces explicit, names consistent, and widths visible. I prefer ANSI-style SystemVerilog to reduce mismatches between the port list and declarations. For input, output, and inout ports, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
29. FR - Expliquez largeur des ports. Quelle structure ou idée hardware se cache derrière ? Réponse : Une plage comme '[7:0]' définit un vecteur de huit bits. Les mismatches de largeur peuvent provoquer troncature, extension ou warnings, donc la taille fait partie de l'interface.	EN - Explain port widths. What hardware structure or idea is behind it? Answer: A range such as '[7:0]' defines an eight-bit vector. Width mismatches can cause truncation, extension, or warnings, so width is part of the interface.

30. FR - Comment utiliseriez-vous largeur des ports dans un design SystemVerilog réel ? Réponse : Je déclare explicitement <code>input wire [7:0] data_in`</code> et vérifie la largeur à chaque instance. <pre>module block(input wire [7:0] data_in, output wire [7:0] data_out); assign data_out = data_in; endmodule</pre>	EN - How would you use port widths in a real SystemVerilog design? Answer: I explicitly declare <code>input wire [7:0] data_in`</code> and verify the width at each instance. <pre>module block(input wire [7:0] data_in, output wire [7:0] data_out); assign data_out = data_in; endmodule</pre>
31. FR - Comment diagnostiqueriez-vous un problème lié à largeur des ports ? Réponse : Je vérifie le nom du module, sa liste de ports, directions, largeurs, déclarations et correspondance avec les instances. Une erreur de largeur ou de connexion peut rester syntaxiquement valide tout en étant fonctionnellement fausse. Pour largeur des ports, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving port widths? Answer: I check the module name, port list, directions, widths, declarations, and instance mapping. A width or connectivity error can remain syntactically valid while being functionally wrong. For port widths, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
32. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant largeur des ports ? Réponse : Je garde des interfaces explicites, des noms cohérents et des largeurs visibles. Je préfère le style ANSI-style SystemVerilog pour réduire les divergences entre la liste de ports et leurs déclarations. Pour largeur des ports, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding port widths? Answer: I keep interfaces explicit, names consistent, and widths visible. I prefer ANSI-style SystemVerilog to reduce mismatches between the port list and declarations. For port widths, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
33. FR - Expliquez port logique versus pin physique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un port SystemVerilog est un signal logique de l'interface du module. L'association avec une broche physique et son standard électrique se fait dans les contraintes du projet, pas dans le langage SystemVerilog lui-même.	EN - Explain logical port versus physical pin. What hardware structure or idea is behind it? Answer: A SystemVerilog port is a logical signal at the module interface. Mapping to a physical package pin and electrical standard is done in project constraints, not in SystemVerilog itself.
34. FR - Comment utiliseriez-vous port logique versus pin physique dans un design SystemVerilog réel ? Réponse : Je nomme par exemple <code>`uart_rx`</code> dans le top-level, puis le flow FPGA associe ce port à une broche via les contraintes vendor-specific.	EN - How would you use logical port versus physical pin in a real SystemVerilog design? Answer: I name a top-level port such as <code>`uart_rx`</code>, then the FPGA flow maps that port to a package pin through vendor-specific constraints.
35. FR - Comment diagnostiqueriez-vous un problème lié à port logique versus pin physique ? Réponse : Je vérifie le nom du module, sa liste de ports, directions, largeurs, déclarations et correspondance avec les instances. Une erreur de largeur ou de connexion peut rester syntaxiquement valide tout en étant fonctionnellement fausse. Pour port logique versus pin physique, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving logical port versus physical pin? Answer: I check the module name, port list, directions, widths, declarations, and instance mapping. A width or connectivity error can remain syntactically valid while being functionally wrong. For logical port versus physical pin, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
36. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant port logique versus pin physique ? Réponse : Je garde des interfaces explicites, des noms cohérents et des largeurs visibles. Je préfère le style ANSI-style SystemVerilog pour réduire les divergences entre la liste de ports et leurs déclarations. Pour port logique versus pin physique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding logical port versus physical pin? Answer: I keep interfaces explicit, names consistent, and widths visible. I prefer ANSI-style SystemVerilog to reduce mismatches between the port list and declarations. For logical port versus physical pin, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
37. FR - Expliquez interrupt versus signal RTL. Quelle structure ou idée hardware se cache derrière ? Réponse : SystemVerilog n'a pas un mot-clé "interrupt" pour un pin. Un événement externe est un signal logique qu'on synchronise et détecte; il ne devient une IRQ que si on le raccorde à une architecture processeur qui interprète cette ligne comme interruption.	EN - Explain interrupt versus RTL signal. What hardware structure or idea is behind it? Answer: SystemVerilog has no "interrupt" keyword for a pin. An external event is a logic signal that is synchronized and detected; it becomes an IRQ only when connected to a processor architecture that treats it as an interrupt line.

38. FR - Comment utiliseriez-vous interrupt versus signal RTL dans un design SystemVerilog réel ? Réponse : Je synchronise un bouton ou GPIO externe, détecte son front et génère éventuellement une impulsion d'IRQ d'un cycle vers un cœur processeur.	EN - How would you use interrupt versus RTL signal in a real SystemVerilog design? Answer: I synchronize an external button or GPIO, detect its edge, and optionally generate a one-cycle IRQ pulse toward a processor core.
39. FR - Comment diagnostiqueriez-vous un problème lié à interrupt versus signal RTL ? Réponse : Je vérifie le nom du module, sa liste de ports, directions, largeurs, déclarations et correspondance avec les instances. Une erreur de largeur ou de connexion peut rester syntaxiquement valide tout en étant fonctionnellement fautive. Pour interrupt versus signal RTL, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving interrupt versus RTL signal? Answer: I check the module name, port list, directions, widths, declarations, and instance mapping. A width or connectivity error can remain syntactically valid while being functionally wrong. For interrupt versus RTL signal, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
40. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant interrupt versus signal RTL ? Réponse : Je garde des interfaces explicites, des noms cohérents et des largeurs visibles. Je préfère le style ANSI-style SystemVerilog pour réduire les divergences entre la liste de ports et leurs déclarations. Pour interrupt versus signal RTL, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding interrupt versus RTL signal? Answer: I keep interfaces explicit, names consistent, and widths visible. I prefer ANSI-style SystemVerilog to reduce mismatches between the port list and declarations. For interrupt versus RTL signal, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

3. Types `logic`, nets, variables, arrays et littéraux / `logic` Types, Nets, Variables, Arrays, and Literals

41. FR - Expliquez le type `logic`. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `logic` est un type variable 4 états qui remplace la plupart des usages de `reg`. Il peut représenter 0, 1, X et Z et convient bien aux signaux RTL à propriétaire unique.	EN - Explain the `logic` type. What important difference, structure, or idea should be understood? Answer: `logic` is a 4-state variable type replacing most uses of `reg`. It can represent 0, 1, X, and Z and fits single-owner RTL signals well.
42. FR - Comment utiliseriez-vous le type `logic` dans un projet SystemVerilog réel ? Réponse : Je déclare la majorité des signaux RTL internes en `logic`, puis j'utilise un net explicite si une résolution multi-driver est réellement nécessaire.	EN - How would you use the `logic` type in a real SystemVerilog project? Answer: I declare most internal RTL signals as `logic`, then use an explicit net when true multi-driver resolution is required.
43. FR - Comment diagnostiqueriez-vous un problème lié à le type `logic` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour le type `logic`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to the `logic` type? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For the `logic` type, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
44. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant le type `logic` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour le type `logic`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding the `logic` type? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For the `logic` type, I also explain the expected hardware or simulation-only role, together with important corner cases.
45. FR - Expliquez net versus variable. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un net modélise une connexion résolue entre drivers; une variable conserve la valeur donnée par une procédure. `wire` est un net classique, alors que `logic` est normalement une variable.	EN - Explain net versus variable. What important difference, structure, or idea should be understood? Answer: A net models a resolved connection among drivers; a variable holds the value assigned procedurally. `wire` is a classic net, while `logic` is normally a variable.
46. FR - Comment utiliseriez-vous net versus variable dans un projet SystemVerilog réel ? Réponse : Je garde un propriétaire procédural unique pour les variables RTL et réserve les nets aux interconnexions qui en ont besoin.	EN - How would you use net versus variable in a real SystemVerilog project? Answer: I keep a single procedural owner for RTL variables and reserve nets for interconnects that need them.

47. FR - Comment diagnostiqueriez-vous un problème lié à net versus variable ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour net versus variable, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to net versus variable? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For net versus variable, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
48. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant net versus variable ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour net versus variable, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding net versus variable? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For net versus variable, I also explain the expected hardware or simulation-only role, together with important corner cases.
49. FR - Expliquez types 2 états `bit` versus 4 états `logic`. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `bit` ne contient que 0/1; `logic` conserve X/Z. Les types 2 états peuvent être utiles en testbench mais peuvent masquer une initialisation ou une contention incorrecte.	EN - Explain 2-state `bit` versus 4-state `logic`. What important difference, structure, or idea should be understood? Answer: `bit` contains only 0/1; `logic` preserves X/Z. Two-state types can be useful in testbenches but can hide bad initialization or contention.
50. FR - Comment utiliseriez-vous types 2 états `bit` versus 4 états `logic` dans un projet SystemVerilog réel ? Réponse : En RTL, je préfère souvent `logic` afin que les X révèlent les erreurs de reset ou de driver pendant la simulation.	EN - How would you use 2-state `bit` versus 4-state `logic` in a real SystemVerilog project? Answer: In RTL, I often prefer `logic` so X values expose reset or driver problems during simulation.
51. FR - Comment diagnostiqueriez-vous un problème lié à types 2 états `bit` versus 4 états `logic` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour types 2 états `bit` versus 4 états `logic`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to 2-state `bit` versus 4-state `logic`? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For 2-state `bit` versus 4-state `logic`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
52. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant types 2 états `bit` versus 4 états `logic` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour types 2 états `bit` versus 4 états `logic`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding 2-state `bit` versus 4-state `logic`? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For 2-state `bit` versus 4-state `logic`, I also explain the expected hardware or simulation-only role, together with important corner cases.
53. FR - Expliquez packed versus unpacked arrays. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une dimension packed forme un vecteur contigu de bits; une dimension unpacked représente une collection d'éléments. `logic [7:0] mem [0:15]` est donc 16 mots de 8 bits. `logic [7:0] mem [0:15];`	EN - Explain packed versus unpacked arrays. What important difference, structure, or idea should be understood? Answer: A packed dimension forms a contiguous bit vector; an unpacked dimension represents a collection of elements. `logic [7:0] mem [0:15]` is therefore 16 words of 8 bits. `logic [7:0] mem [0:15];`
54. FR - Comment utiliseriez-vous packed versus unpacked arrays dans un projet SystemVerilog réel ? Réponse : J'utilise packed pour les bus et unpacked pour les tableaux de mots, mémoires ou collections statiques. `logic [7:0] mem [0:15];`	EN - How would you use packed versus unpacked arrays in a real SystemVerilog project? Answer: I use packed dimensions for buses and unpacked dimensions for word arrays, memories, or static collections. `logic [7:0] mem [0:15];`
55. FR - Comment diagnostiqueriez-vous un problème lié à packed versus unpacked arrays ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour packed versus unpacked arrays, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to packed versus unpacked arrays? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For packed versus unpacked arrays, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

56. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant packed versus unpacked arrays ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour packed versus unpacked arrays, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding packed versus unpacked arrays? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For packed versus unpacked arrays, I also explain the expected hardware or simulation-only role, together with important corner cases.
57. FR - Expliquez `typedef`, `enum` et `struct`. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `typedef` crée des types nommés, `enum` rend les états/opcodes lisibles et `struct` regroupe des champs nommés. Ils réduisent fortement les slices et constantes numériques difficiles à maintenir. <pre>typedef enum logic [1:0] {IDLE, LOAD, RUN, DONE} state_t;</pre>	EN - Explain `typedef`, `enum`, and `struct`. What important difference, structure, or idea should be understood? Answer: `typedef` creates named types, `enum` makes states/opcodes readable, and `struct` groups named fields. They greatly reduce fragile slices and numeric constants. <pre>typedef enum logic [1:0] {IDLE, LOAD, RUN, DONE} state_t;</pre>
58. FR - Comment utiliseriez-vous `typedef`, `enum` et `struct` dans un projet SystemVerilog réel ? Réponse : Je mets les types partagés dans un package et j'utilise `enum` pour FSM et `struct packed` pour packets/buses structurés. <pre>typedef enum logic [1:0] {IDLE, LOAD, RUN, DONE} state_t;</pre>	EN - How would you use `typedef`, `enum`, and `struct` in a real SystemVerilog project? Answer: I put shared types in a package and use `enum` for FSMs and `struct packed` for structured packets/buses. <pre>typedef enum logic [1:0] {IDLE, LOAD, RUN, DONE} state_t;</pre>
59. FR - Comment diagnostiqueriez-vous un problème lié à `typedef`, `enum` et `struct` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `typedef`, `enum` et `struct`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `typedef`, `enum`, and `struct`? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `typedef`, `enum`, and `struct`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
60. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `typedef`, `enum` et `struct` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour `typedef`, `enum` et `struct`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `typedef`, `enum`, and `struct`? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `typedef`, `enum`, and `struct`, I also explain the expected hardware or simulation-only role, together with important corner cases.

4. Opérateurs, expressions, casts et signedness / Operators, Expressions, Casts, and Signedness

61. FR - Expliquez opérateurs bitwise. Quelle structure ou idée hardware se cache derrière ? Réponse : `&` et `^` opèrent bit par bit sur des vecteurs et construisent directement de la logique booléenne parallèle.	EN - Explain bitwise operators. What hardware structure or idea is behind it? Answer: `&` and `^` operate bit by bit on vectors and directly build parallel Boolean logic.
62. FR - Comment utiliseriez-vous opérateurs bitwise dans un design SystemVerilog réel ? Réponse : Je combine des masques et signaux de contrôle avec des opérateurs bitwise quand chaque bit est indépendant. <pre>assign y = (a & mask) (b & ~mask);</pre>	EN - How would you use bitwise operators in a real SystemVerilog design? Answer: I combine masks and control signals with bitwise operators when each bit is independent. <pre>assign y = (a & mask) (b & ~mask);</pre>
63. FR - Comment diagnostiqueriez-vous un problème lié à opérateurs bitwise ? Réponse : Je réduis l'expression en étapes, je vérifie largeur et signedness de chaque opérande, puis j'examine la priorité des opérateurs. Les erreurs silencieuses de taille sont particulièrement dangereuses en RTL. Pour opérateurs bitwise, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving bitwise operators? Answer: I reduce the expression step by step, verify width and signedness of every operand, then examine operator precedence. Silent sizing errors are especially dangerous in RTL. For bitwise operators, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
64. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant opérateurs bitwise ? Réponse : Je parenthèse les expressions non triviales et je dimensionne les constantes. Je ne mélange pas implicitement signed et unsigned dans un datapath critique. Pour opérateurs bitwise, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding bitwise operators? Answer: I parenthesize nontrivial expressions and size constants. I do not implicitly mix signed and unsigned values in a critical datapath. For bitwise operators, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

65. FR - Expliquez opérateurs logiques. Quelle structure ou idée hardware se cache derrière ? Réponse : `&&`, `  ` et `!` évaluent une condition logique globale, généralement pour du contrôle; ils ne sont pas équivalents aux opérateurs bitwise sur des bus.	EN - Explain logical operators. What hardware structure or idea is behind it? Answer: `&&`, `  `, and `!` evaluate a global logical condition, usually for control; they are not equivalent to bitwise operators on buses.
66. FR - Comment utiliseriez-vous opérateurs logiques dans un design SystemVerilog réel ? Réponse : J'utilise `if (valid && ready)` pour un handshake booléen et `data_a & data_b` pour un AND bit à bit.	EN - How would you use logical operators in a real SystemVerilog design? Answer: I use `if (valid && ready)` for a Boolean handshake and `data_a & data_b` for a bitwise AND.
67. FR - Comment diagnostiqueriez-vous un problème lié à opérateurs logiques ? Réponse : Je réduis l'expression en étapes, je vérifie largeur et signedness de chaque opérande, puis j'examine la priorité des opérateurs. Les erreurs silencieuses de taille sont particulièrement dangereuses en RTL. Pour opérateurs logiques, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving logical operators? Answer: I reduce the expression step by step, verify width and signedness of every operand, then examine operator precedence. Silent sizing errors are especially dangerous in RTL. For logical operators, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
68. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant opérateurs logiques ? Réponse : Je parenthèse les expressions non triviales et je dimensionne les constantes. Je ne mélange pas implicitement signed et unsigned dans un datapath critique. Pour opérateurs logiques, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding logical operators? Answer: I parenthesize nontrivial expressions and size constants. I do not implicitly mix signed and unsigned values in a critical datapath. For logical operators, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
69. FR - Expliquez réduction. Quelle structure ou idée hardware se cache derrière ? Réponse : Une réduction comme `&bus` ou ` bus` produit un seul bit à partir de tous les bits du vecteur; elle est utile pour all-ones, any-one et parity.	EN - Explain reduction operators. What hardware structure or idea is behind it? Answer: A reduction such as `&bus` or ` bus` produces one bit from all bits of a vector; it is useful for all-ones, any-one, and parity checks.
70. FR - Comment utiliseriez-vous réduction dans un design SystemVerilog réel ? Réponse : Je peux écrire `assign any_error =  error_flags;` pour savoir si au moins un bit d'erreur est actif. assign any_error = error_flags; assign parity = ^data;	EN - How would you use reduction operators in a real SystemVerilog design? Answer: I can write `assign any_error =  error_flags;` to detect whether any error bit is active. assign any_error = error_flags; assign parity = ^data;
71. FR - Comment diagnostiqueriez-vous un problème lié à réduction ? Réponse : Je réduis l'expression en étapes, je vérifie largeur et signedness de chaque opérande, puis j'examine la priorité des opérateurs. Les erreurs silencieuses de taille sont particulièrement dangereuses en RTL. Pour réduction, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving reduction operators? Answer: I reduce the expression step by step, verify width and signedness of every operand, then examine operator precedence. Silent sizing errors are especially dangerous in RTL. For reduction operators, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
72. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant réduction ? Réponse : Je parenthèse les expressions non triviales et je dimensionne les constantes. Je ne mélange pas implicitement signed et unsigned dans un datapath critique. Pour réduction, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding reduction operators? Answer: I parenthesize nontrivial expressions and size constants. I do not implicitly mix signed and unsigned values in a critical datapath. For reduction operators, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
73. FR - Expliquez concaténation et réplication. Quelle structure ou idée hardware se cache derrière ? Réponse : `{a,b}` concatène des vecteurs; `{N{value}}` réplique un motif. Ces opérations décrivent uniquement du câblage/assemblage de bits.	EN - Explain concatenation and replication. What hardware structure or idea is behind it? Answer: `{a,b}` concatenates vectors; `{N{value}}` replicates a pattern. These operations describe bit wiring/assembly.

74. FR - Comment utiliseriez-vous concaténation et réplication dans un design SystemVerilog réel ? Réponse : Je forme un mot 16 bits avec <code>{high_byte, low_byte}</code> ou étends un bit de signe par réplication. <pre>assign word = {high_byte, low_byte}; assign ext = {{8{data[7]}}, data};</pre>	EN - How would you use concatenation and replication in a real SystemVerilog design? Answer: I form a 16-bit word with <code>{high_byte, low_byte}</code> or extend a sign bit by replication. <pre>assign word = {high_byte, low_byte}; assign ext = {{8{data[7]}}, data};</pre>
75. FR - Comment diagnostiqueriez-vous un problème lié à concaténation et réplication ? Réponse : Je réduis l'expression en étapes, je vérifie largeur et signedness de chaque opérande, puis j'examine la priorité des opérateurs. Les erreurs silencieuses de taille sont particulièrement dangereuses en RTL. Pour concaténation et réplication, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving concatenation and replication? Answer: I reduce the expression step by step, verify width and signedness of every operand, then examine operator precedence. Silent sizing errors are especially dangerous in RTL. For concatenation and replication, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
76. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant concaténation et réplication ? Réponse : Je parenthèse les expressions non triviales et je dimensionne les constantes. Je ne mélange pas implicitement signed et unsigned dans un datapath critique. Pour concaténation et réplication, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding concatenation and replication? Answer: I parenthesize nontrivial expressions and size constants. I do not implicitly mix signed and unsigned values in a critical datapath. For concatenation and replication, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
77. FR - Expliquez signedness et extension. Quelle structure ou idée hardware se cache derrière ? Réponse : Les opérations arithmétiques dépendent de la largeur et du statut signed des opérandes. Une extension zéro ou signe incorrecte peut changer complètement un résultat.	EN - Explain signedness and extension. What hardware structure or idea is behind it? Answer: Arithmetic operations depend on operand width and signedness. Incorrect zero or sign extension can completely change a result.
78. FR - Comment utiliseriez-vous signedness et extension dans un design SystemVerilog réel ? Réponse : Je déclare explicitement les bus signed lorsque le datapath manipule des valeurs négatives et je vérifie les conversions aux frontières. <pre>wire signed [7:0] a; wire signed [8:0] sum = a + b;</pre>	EN - How would you use signedness and extension in a real SystemVerilog design? Answer: I explicitly declare buses as signed when the datapath handles negative values and verify conversions at boundaries. <pre>wire signed [7:0] a; wire signed [8:0] sum = a + b;</pre>
79. FR - Comment diagnostiqueriez-vous un problème lié à signedness et extension ? Réponse : Je réduis l'expression en étapes, je vérifie largeur et signedness de chaque opérande, puis j'examine la priorité des opérateurs. Les erreurs silencieuses de taille sont particulièrement dangereuses en RTL. Pour signedness et extension, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving signedness and extension? Answer: I reduce the expression step by step, verify width and signedness of every operand, then examine operator precedence. Silent sizing errors are especially dangerous in RTL. For signedness and extension, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
80. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant signedness et extension ? Réponse : Je parenthèse les expressions non triviales et je dimensionne les constantes. Je ne mélange pas implicitement signed et unsigned dans un datapath critique. Pour signedness et extension, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding signedness and extension? Answer: I parenthesize nontrivial expressions and size constants. I do not implicitly mix signed and unsigned values in a critical datapath. For signedness and extension, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

5. Logique combinatoire et tables de vérité / Combinational Logic and Truth Tables

81. FR - Expliquez porte AND et table de vérité. Quelle structure ou idée hardware se cache derrière ? Réponse : Un AND à deux entrées donne 1 seulement lorsque les deux entrées valent 1. En SystemVerilog, <code>assign y = a & b;</code> synthétise cette fonction.	EN - Explain AND gate and truth table. What hardware structure or idea is behind it? Answer: A two-input AND outputs 1 only when both inputs are 1. In SystemVerilog, <code>assign y = a & b;</code> synthesizes that function.
---	---

82. FR - Comment utiliseriez-vous porte AND et table de vérité dans un design SystemVerilog réel ? Réponse : Je peux vérifier exhaustivement les quatre combinaisons dans un petit testbench. <pre>assign y = a & b;</pre>	EN - How would you use AND gate and truth table in a real SystemVerilog design? Answer: I can exhaustively verify all four combinations in a small testbench. <pre>assign y = a & b;</pre>
83. FR - Comment diagnostiqueriez-vous un problème lié à porte AND et table de vérité ? Réponse : Je vérifie la table de vérité, la couverture complète des entrées et l'absence de mémoire implicite. Une fonction combinatoire doit dépendre uniquement des entrées actuelles. Pour porte AND et table de vérité, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving AND gate and truth table? Answer: I verify the truth table, complete input coverage, and absence of implicit storage. A combinational function must depend only on current inputs. For AND gate and truth table, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
84. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant porte AND et table de vérité ? Réponse : Pour les petits blocs, je garde la relation entrée-sortie évidente. Pour les décodeurs plus grands, je fournis des valeurs par défaut et une structure `case` lisible. Pour porte AND et table de vérité, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding AND gate and truth table? Answer: For small blocks, I keep the input-output relationship obvious. For larger decoders, I provide defaults and a readable `case` structure. For AND gate and truth table, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
85. FR - Expliquez multiplexeur 2:1. Quelle structure ou idée hardware se cache derrière ? Réponse : Un mux choisit une entrée selon `sel`. L'opérateur ternaire `?:` exprime naturellement un mux combinatoire.	EN - Explain 2:1 multiplexer. What hardware structure or idea is behind it? Answer: A mux selects one input according to `sel`. The ternary `?:` operator naturally expresses a combinational mux.
86. FR - Comment utiliseriez-vous multiplexeur 2:1 dans un design SystemVerilog réel ? Réponse : Je décris `assign y = sel ? b : a;` et vérifie que `sel=0` sélectionne bien `a`. <pre>assign y = sel ? b : a;</pre>	EN - How would you use 2:1 multiplexer in a real SystemVerilog design? Answer: I write `assign y = sel ? b : a;` and verify that `sel=0` correctly selects `a`. <pre>assign y = sel ? b : a;</pre>
87. FR - Comment diagnostiqueriez-vous un problème lié à multiplexeur 2:1 ? Réponse : Je vérifie la table de vérité, la couverture complète des entrées et l'absence de mémoire implicite. Une fonction combinatoire doit dépendre uniquement des entrées actuelles. Pour multiplexeur 2:1, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving 2:1 multiplexer? Answer: I verify the truth table, complete input coverage, and absence of implicit storage. A combinational function must depend only on current inputs. For 2:1 multiplexer, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
88. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant multiplexeur 2:1 ? Réponse : Pour les petits blocs, je garde la relation entrée-sortie évidente. Pour les décodeurs plus grands, je fournis des valeurs par défaut et une structure `case` lisible. Pour multiplexeur 2:1, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding 2:1 multiplexer? Answer: For small blocks, I keep the input-output relationship obvious. For larger decoders, I provide defaults and a readable `case` structure. For 2:1 multiplexer, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
89. FR - Expliquez décodeur. Quelle structure ou idée hardware se cache derrière ? Réponse : Un décodeur transforme une valeur codée en sorties sélectionnées, par exemple 2 bits vers 4 lignes one-hot. Il peut être décrit par `case` ou shifts.	EN - Explain decoder. What hardware structure or idea is behind it? Answer: A decoder transforms a coded value into selected outputs, for example 2 bits into four one-hot lines. It can be described with `case` or shifts.

90. FR - Comment utiliseriez-vous décodeur dans un design SystemVerilog réel ? Réponse : Pour rendre la couverture explicite, j'initialise la sortie puis active une seule branche. <pre>always @(*) begin y = 4'b0000; case (sel) 2'b00: y[0] = 1'b1; 2'b01: y[1] = 1'b1; 2'b10: y[2] = 1'b1; 2'b11: y[3] = 1'b1; endcase end</pre>	EN - How would you use decoder in a real SystemVerilog design? Answer: To make coverage explicit, I initialize the output then activate one branch. <pre>always @(*) begin y = 4'b0000; case (sel) 2'b00: y[0] = 1'b1; 2'b01: y[1] = 1'b1; 2'b10: y[2] = 1'b1; 2'b11: y[3] = 1'b1; endcase end</pre>
91. FR - Comment diagnostiqueriez-vous un problème lié à décodeur ? Réponse : Je vérifie la table de vérité, la couverture complète des entrées et l'absence de mémoire implicite. Une fonction combinatoire doit dépendre uniquement des entrées actuelles. Pour décodeur, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving decoder? Answer: I verify the truth table, complete input coverage, and absence of implicit storage. A combinational function must depend only on current inputs. For decoder, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
92. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant décodeur ? Réponse : Pour les petits blocs, je garde la relation entrée-sortie évidente. Pour les décodeurs plus grands, je fournis des valeurs par défaut et une structure `case` lisible. Pour décodeur, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding decoder? Answer: For small blocks, I keep the input-output relationship obvious. For larger decoders, I provide defaults and a readable `case` structure. For decoder, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
93. FR - Expliquez comparateur. Quelle structure ou idée hardware se cache derrière ? Réponse : Les opérateurs `==`, `<`, `>` etc. décrivent des comparateurs. Le coût et le chemin critique augmentent généralement avec la largeur et le type d'opération.	EN - Explain comparator. What hardware structure or idea is behind it? Answer: Operators such as `==`, `<`, and `>` describe comparators. Cost and critical path generally grow with width and operation type.
94. FR - Comment utiliseriez-vous comparateur dans un design SystemVerilog réel ? Réponse : Je garde la signedness cohérente avant de comparer deux valeurs numériques.	EN - How would you use comparator in a real SystemVerilog design? Answer: I keep signedness consistent before comparing two numeric values.
95. FR - Comment diagnostiqueriez-vous un problème lié à comparateur ? Réponse : Je vérifie la table de vérité, la couverture complète des entrées et l'absence de mémoire implicite. Une fonction combinatoire doit dépendre uniquement des entrées actuelles. Pour comparateur, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving comparator? Answer: I verify the truth table, complete input coverage, and absence of implicit storage. A combinational function must depend only on current inputs. For comparator, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
96. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant comparateur ? Réponse : Pour les petits blocs, je garde la relation entrée-sortie évidente. Pour les décodeurs plus grands, je fournis des valeurs par défaut et une structure `case` lisible. Pour comparateur, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding comparator? Answer: For small blocks, I keep the input-output relationship obvious. For larger decoders, I provide defaults and a readable `case` structure. For comparator, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
97. FR - Expliquez additionneur combinatoire. Quelle structure ou idée hardware se cache derrière ? Réponse : Une expression `a + b` synthétise un additionneur. La largeur de sortie doit être suffisante si le carry doit être conservé.	EN - Explain combinational adder. What hardware structure or idea is behind it? Answer: An `a + b` expression synthesizes an adder. The output width must be sufficient if the carry must be preserved.
98. FR - Comment utiliseriez-vous additionneur combinatoire dans un design SystemVerilog réel ? Réponse : Pour deux entrées 8 bits non signées, je choisis une sortie 9 bits lorsque le carry final compte. <pre>wire [8:0] sum = {1'b0,a} + {1'b0,b};</pre>	EN - How would you use combinational adder in a real SystemVerilog design? Answer: For two unsigned 8-bit inputs, I choose a 9-bit output when the final carry matters. <pre>wire [8:0] sum = {1'b0,a} + {1'b0,b};</pre>

99. FR - Comment diagnostiqueriez-vous un problème lié à additionneur combinatoire ? Réponse : Je vérifie la table de vérité, la couverture complète des entrées et l'absence de mémoire implicite. Une fonction combinatoire doit dépendre uniquement des entrées actuelles. Pour additionneur combinatoire, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving combinational adder? Answer: I verify the truth table, complete input coverage, and absence of implicit storage. A combinational function must depend only on current inputs. For combinational adder, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
100. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant additionneur combinatoire ? Réponse : Pour les petits blocs, je garde la relation entrée-sortie évidente. Pour les décodeurs plus grands, je fournis des valeurs par défaut et une structure `case` lisible. Pour additionneur combinatoire, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding combinational adder? Answer: For small blocks, I keep the input-output relationship obvious. For larger decoders, I provide defaults and a readable `case` structure. For combinational adder, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

6. Affectations continues, tri-state et dataflow / Continuous Assignments, Tri-State, and Dataflow

101. FR - Expliquez assign continu. Quelle structure ou idée hardware se cache derrière ? Réponse : `assign` pilote continuellement un net à partir d'une expression. Il correspond à de la logique combinatoire active en permanence.	EN - Explain continuous assign. What hardware structure or idea is behind it? Answer: `assign` continuously drives a net from an expression. It corresponds to combinational logic that is continuously active.
102. FR - Comment utiliseriez-vous assign continu dans un design SystemVerilog réel ? Réponse : Je l'utilise pour un flag, un mux simple ou un décodage compact. <pre>assign alarm = over_temp over_current;</pre>	EN - How would you use continuous assign in a real SystemVerilog design? Answer: I use it for a flag, simple mux, or compact decode. <pre>assign alarm = over_temp over_current;</pre>
103. FR - Comment diagnostiqueriez-vous un problème lié à assign continu ? Réponse : Je recherche qui pilote chaque net et si plusieurs drivers sont intentionnels. Pour les tri-states, je vérifie qu'ils ne sont utilisés qu'aux endroits supportés par le matériel cible. Pour assign continu, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving continuous assign? Answer: I identify every net driver and whether multiple drivers are intentional. For tri-states, I verify they are used only where the target hardware supports them. For continuous assign, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
104. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant assign continu ? Réponse : J'utilise `assign` pour les relations combinatoires simples et garde un propriétaire clair par signal. Je réserve `Z` aux vrais bus partagés ou I/O bidirectionnels. Pour assign continu, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding continuous assign? Answer: I use `assign` for simple combinational relationships and keep clear ownership per signal. I reserve `Z` for real shared buses or bidirectional I/O. For continuous assign, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
105. FR - Expliquez opérateur ternaire. Quelle structure ou idée hardware se cache derrière ? Réponse : `condition ? a : b` décrit une sélection combinatoire et se synthétise typiquement en multiplexeur.	EN - Explain ternary operator. What hardware structure or idea is behind it? Answer: `condition ? a : b` describes a combinational selection and typically synthesizes into a multiplexer.
106. FR - Comment utiliseriez-vous opérateur ternaire dans un design SystemVerilog réel ? Réponse : Pour une priorité à plusieurs niveaux, je fais attention aux ternaires imbriqués car ils peuvent créer une chaîne de mux.	EN - How would you use ternary operator in a real SystemVerilog design? Answer: For multi-level priority, I am careful with nested ternaries because they can create a mux chain.
107. FR - Comment diagnostiqueriez-vous un problème lié à opérateur ternaire ? Réponse : Je recherche qui pilote chaque net et si plusieurs drivers sont intentionnels. Pour les tri-states, je vérifie qu'ils ne sont utilisés qu'aux endroits supportés par le matériel cible. Pour opérateur ternaire, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving ternary operator? Answer: I identify every net driver and whether multiple drivers are intentional. For tri-states, I verify they are used only where the target hardware supports them. For ternary operator, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

108. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant opérateur ternaire ? Réponse : J'utilise `assign` pour les relations combinatoires simples et garde un propriétaire clair par signal. Je réserve `Z` aux vrais bus partagés ou I/O bidirectionnels. Pour opérateur ternaire, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding ternary operator? Answer: I use `assign` for simple combinational relationships and keep clear ownership per signal. I reserve `Z` for real shared buses or bidirectional I/O. For ternary operator, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
109. FR - Expliquez haute impédance Z. Quelle structure ou idée hardware se cache derrière ? Réponse : Piloter `Z` signifie que la source se déconnecte électriquement du net simulé. C'est utile pour représenter des bus tri-state aux I/O.	EN - Explain high impedance Z. What hardware structure or idea is behind it? Answer: Driving `Z` means the source electrically disconnects from the simulated net. It is useful for representing tri-state buses at I/O boundaries.
110. FR - Comment utiliseriez-vous haute impédance Z dans un design SystemVerilog réel ? Réponse : Un GPIO bidirectionnel peut être `assign pin = oe ? data_out : 1'bz;` puis lu via `pin`. <code>assign gpio = oe ? dout : 1'bz;</code> <code>assign din = gpio;</code>	EN - How would you use high impedance Z in a real SystemVerilog design? Answer: A bidirectional GPIO can use `assign pin = oe ? data_out : 1'bz;` and then read the pin value. <code>assign gpio = oe ? dout : 1'bz;</code> <code>assign din = gpio;</code>
111. FR - Comment diagnostiqueriez-vous un problème lié à haute impédance Z ? Réponse : Je recherche qui pilote chaque net et si plusieurs drivers sont intentionnels. Pour les tri-states, je vérifie qu'ils ne sont utilisés qu'aux endroits supportés par le matériel cible. Pour haute impédance Z, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving high impedance Z? Answer: I identify every net driver and whether multiple drivers are intentional. For tri-states, I verify they are used only where the target hardware supports them. For high impedance Z, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
112. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant haute impédance Z ? Réponse : J'utilise `assign` pour les relations combinatoires simples et garde un propriétaire clair par signal. Je réserve `Z` aux vrais bus partagés ou I/O bidirectionnels. Pour haute impédance Z, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding high impedance Z? Answer: I use `assign` for simple combinational relationships and keep clear ownership per signal. I reserve `Z` for real shared buses or bidirectional I/O. For high impedance Z, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
113. FR - Expliquez multiple drivers. Quelle structure ou idée hardware se cache derrière ? Réponse : Plusieurs sources peuvent piloter un net résolu, mais des valeurs contradictoires produisent souvent `X` en simulation et une architecture invalide ou non portable en synthèse interne.	EN - Explain multiple drivers. What hardware structure or idea is behind it? Answer: Multiple sources may drive a resolved net, but conflicting values often produce `X` in simulation and an invalid or nonportable internal synthesis structure.
114. FR - Comment utiliseriez-vous multiple drivers dans un design SystemVerilog réel ? Réponse : Je cherche un seul propriétaire par signal interne et combine l'arbitrage avant le driver final.	EN - How would you use multiple drivers in a real SystemVerilog design? Answer: I aim for one owner per internal signal and combine arbitration before the final driver.
115. FR - Comment diagnostiqueriez-vous un problème lié à multiple drivers ? Réponse : Je recherche qui pilote chaque net et si plusieurs drivers sont intentionnels. Pour les tri-states, je vérifie qu'ils ne sont utilisés qu'aux endroits supportés par le matériel cible. Pour multiple drivers, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving multiple drivers? Answer: I identify every net driver and whether multiple drivers are intentional. For tri-states, I verify they are used only where the target hardware supports them. For multiple drivers, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
116. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant multiple drivers ? Réponse : J'utilise `assign` pour les relations combinatoires simples et garde un propriétaire clair par signal. Je réserve `Z` aux vrais bus partagés ou I/O bidirectionnels. Pour multiple drivers, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding multiple drivers? Answer: I use `assign` for simple combinational relationships and keep clear ownership per signal. I reserve `Z` for real shared buses or bidirectional I/O. For multiple drivers, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

117. FR - Expliquez dataflow versus procedural. Quelle structure ou idée hardware se cache derrière ? Réponse : Le style dataflow avec <code>`assign`</code> exprime directement les équations; le style procedural avec <code>`always @(*)`</code> est souvent plus lisible pour conditions et <code>`case`</code>. Les deux peuvent décrire le même hardware combinatoire.	EN - Explain dataflow versus procedural. What hardware structure or idea is behind it? Answer: Dataflow style with <code>`assign`</code> directly expresses equations; procedural style with <code>`always @(*)`</code> is often clearer for conditions and <code>`case`</code>. Both can describe the same combinational hardware.
118. FR - Comment utiliseriez-vous dataflow versus procedural dans un design SystemVerilog réel ? Réponse : Je choisis la forme qui rend l'intention RTL la plus évidente sans dupliquer de logique.	EN - How would you use dataflow versus procedural in a real SystemVerilog design? Answer: I choose the form that makes RTL intent clearest without duplicating logic.
119. FR - Comment diagnostiqueriez-vous un problème lié à dataflow versus procedural ? Réponse : Je recherche qui pilote chaque net et si plusieurs drivers sont intentionnels. Pour les tri-states, je vérifie qu'ils ne sont utilisés qu'aux endroits supportés par le matériel cible. Pour dataflow versus procedural, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving dataflow versus procedural? Answer: I identify every net driver and whether multiple drivers are intentional. For tri-states, I verify they are used only where the target hardware supports them. For dataflow versus procedural, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
120. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant dataflow versus procedural ? Réponse : J'utilise <code>`assign`</code> pour les relations combinatoires simples et garde un propriétaire clair par signal. Je réserve <code>`Z`</code> aux vrais bus partagés ou I/O bidirectionnels. Pour dataflow versus procedural, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding dataflow versus procedural? Answer: I use <code>`assign`</code> for simple combinational relationships and keep clear ownership per signal. I reserve <code>`Z`</code> for real shared buses or bidirectional I/O. For dataflow versus procedural, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

7. ``always_comb``, ``always_ff``, ``always_latch`` et sémantique procédurale / ``always_comb``, ``always_ff``, ``always_latch``, and Procedural Semantics

121. FR - Expliquez <code>`always_comb`</code>. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : <code>`always_comb`</code> déclare explicitement une logique combinatoire, gère automatiquement la sensibilité et permet aux outils de vérifier certains usages incohérents.	EN - Explain <code>`always_comb`</code>. What important difference, structure, or idea should be understood? Answer: <code>`always_comb`</code> explicitly declares combinational logic, automatically handles sensitivity, and lets tools check some inconsistent uses.
122. FR - Comment utiliseriez-vous <code>`always_comb`</code> dans un projet SystemVerilog réel ? Réponse : Je l'utilise pour mux, décodeurs, next-state logic et calculs combinatoires avec affectations complètes.	EN - How would you use <code>`always_comb`</code> in a real SystemVerilog project? Answer: I use it for muxes, decoders, next-state logic, and combinational calculations with complete assignments.
123. FR - Comment diagnostiqueriez-vous un problème lié à <code>`always_comb`</code> ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour <code>`always_comb`</code>, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to <code>`always_comb`</code>? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For <code>`always_comb`</code>, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
124. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant <code>`always_comb`</code> ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour <code>`always_comb`</code>, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding <code>`always_comb`</code>? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For <code>`always_comb`</code>, I also explain the expected hardware or simulation-only role, together with important corner cases.
125. FR - Expliquez <code>`always_ff`</code>. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : <code>`always_ff`</code> exprime un registre/flip-flop piloté par une horloge et aide à imposer un propriétaire unique de la variable écrite. <pre>always_ff @(posedge clk) begin if (!rst_n) q <= '0; else q <= d; end</pre>	EN - Explain <code>`always_ff`</code>. What important difference, structure, or idea should be understood? Answer: <code>`always_ff`</code> expresses clocked register/flip-flop logic and helps enforce a single owner for the written variable. <pre>always_ff @(posedge clk) begin if (!rst_n) q <= '0; else q <= d; end</pre>

126. FR - Comment utiliseriez-vous `always_ff` dans un projet SystemVerilog réel ? Réponse : Je l'utilise pour state registers, pipelines, counters et registres de contrôle avec non-blocking assignments. <pre>always_ff @(posedge clk) begin if (!rst_n) q <= '0; else q <= d; end</pre>	EN - How would you use `always_ff` in a real SystemVerilog project? Answer: I use it for state registers, pipelines, counters, and control registers with non-blocking assignments. <pre>always_ff @(posedge clk) begin if (!rst_n) q <= '0; else q <= d; end</pre>
127. FR - Comment diagnostiqueriez-vous un problème lié à `always_ff` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `always_ff`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `always_ff`? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `always_ff`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
128. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `always_ff` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour `always_ff`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `always_ff`? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `always_ff`, I also explain the expected hardware or simulation-only role, together with important corner cases.
129. FR - Expliquez `always_latch`. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `always_latch` rend explicite l'intention d'inférer un latch niveau-sensible. En FPGA RTL synchrone, il est beaucoup moins fréquent que `always_ff`.	EN - Explain `always_latch`. What important difference, structure, or idea should be understood? Answer: `always_latch` makes the intent to infer a level-sensitive latch explicit. In synchronous FPGA RTL it is much less common than `always_ff`.
130. FR - Comment utiliseriez-vous `always_latch` dans un projet SystemVerilog réel ? Réponse : Je l'emploie seulement si l'architecture exige réellement un latch; sinon je corrige les affectations incomplètes d'un `always_comb`.	EN - How would you use `always_latch` in a real SystemVerilog project? Answer: I use it only if the architecture truly requires a latch; otherwise I fix incomplete assignments in `always_comb`.
131. FR - Comment diagnostiqueriez-vous un problème lié à `always_latch` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `always_latch`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `always_latch`? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `always_latch`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
132. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `always_latch` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour `always_latch`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `always_latch`? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `always_latch`, I also explain the expected hardware or simulation-only role, together with important corner cases.
133. FR - Expliquez la sensibilité automatique et les fonctions appelées. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `always_comb` tient compte des signaux lus dans le bloc et les fonctions appelées, ce qui évite les sensitivity lists incomplètes typiques du SystemVerilog ancien.	EN - Explain automatic sensitivity and called functions. What important difference, structure, or idea should be understood? Answer: `always_comb` accounts for signals read in the block and called functions, avoiding incomplete sensitivity lists common in older SystemVerilog.
134. FR - Comment utiliseriez-vous la sensibilité automatique et les fonctions appelées dans un projet SystemVerilog réel ? Réponse : Je préfère `always_comb` à `always @(*)` dans le nouveau SystemVerilog pour exprimer clairement l'intention combinatoire.	EN - How would you use automatic sensitivity and called functions in a real SystemVerilog project? Answer: I prefer `always_comb` to `always @(*)` in new SystemVerilog to clearly express combinational intent.

135. FR - Comment diagnostiqueriez-vous un problème lié à la sensibilité automatique et les fonctions appelées ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour la sensibilité automatique et les fonctions appelées, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to automatic sensitivity and called functions? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For automatic sensitivity and called functions, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
136. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant la sensibilité automatique et les fonctions appelées ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour la sensibilité automatique et les fonctions appelées, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding automatic sensitivity and called functions? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For automatic sensitivity and called functions, I also explain the expected hardware or simulation-only role, together with important corner cases.
137. FR - Expliquez l'ownership procédural d'un signal. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un signal écrit dans un bloc procédural devrait généralement avoir un seul propriétaire. SystemVerilog et le lint peuvent détecter plus facilement les multiple drivers.	EN - Explain procedural ownership of a signal. What important difference, structure, or idea should be understood? Answer: A signal written procedurally should generally have a single owner. SystemVerilog and lint can detect multiple drivers more effectively.
138. FR - Comment utiliseriez-vous l'ownership procédural d'un signal dans un projet SystemVerilog réel ? Réponse : Je rassemble les conditions d'écriture d'un registre dans un seul `always_ff` et celles d'une variable combinatoire dans un seul `always_comb`.	EN - How would you use procedural ownership of a signal in a real SystemVerilog project? Answer: I group all write conditions of a register in one `always_ff` and of a combinational variable in one `always_comb`.
139. FR - Comment diagnostiqueriez-vous un problème lié à l'ownership procédural d'un signal ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour l'ownership procédural d'un signal, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to procedural ownership of a signal? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For procedural ownership of a signal, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
140. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant l'ownership procédural d'un signal ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour l'ownership procédural d'un signal, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding procedural ownership of a signal? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For procedural ownership of a signal, I also explain the expected hardware or simulation-only role, together with important corner cases.

8. Blocking =, non-blocking <= et scheduling / Blocking =, Non-Blocking <=, and Scheduling

141. FR - Expliquez blocking assignment =. Quelle structure ou idée hardware se cache derrière ? Réponse : Une affectation blocking met à jour la variable immédiatement dans le flot procédural du bloc. L'instruction suivante voit la nouvelle valeur.	EN - Explain blocking assignment =. What hardware structure or idea is behind it? Answer: A blocking assignment updates the variable immediately in the procedural flow of the block. The next statement sees the new value.
142. FR - Comment utiliseriez-vous blocking assignment = dans un design SystemVerilog réel ? Réponse : Je l'utilise dans `always @(*)` pour construire un calcul combinatoire étape par étape. <pre>always @(*) begin tmp = a + b; y = tmp ^ mask; end</pre>	EN - How would you use blocking assignment = in a real SystemVerilog design? Answer: I use it in `always @(*)` to build a combinational calculation step by step. <pre>always @(*) begin tmp = a + b; y = tmp ^ mask; end</pre>

143. FR - Comment diagnostiqueriez-vous un problème lié à blocking assignment = ? Réponse : Je recherche d'abord le mélange de `=` et `<=`, puis je vérifie si les dépendances sont voulues dans le même événement. Les bugs de scheduling sont parmi les pièges classiques de SystemVerilog. Pour blocking assignment =, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving blocking assignment =? Answer: I first look for mixing `=` and `<=`, then verify whether same-event dependencies are intentional. Scheduling bugs are among classic SystemVerilog pitfalls. For blocking assignment =, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
144. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant blocking assignment = ? Réponse : Règle de base: blocking `=` pour la combinatoire procédurale, non-blocking `<=` pour les registres clockés. Je ne déroge à cette règle qu'avec une raison claire et revue. Pour blocking assignment =, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding blocking assignment =? Answer: Base rule: blocking `=` for procedural combinational logic, non-blocking `<=` for clocked registers. I deviate only with a clear, reviewed reason. For blocking assignment =, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
145. FR - Expliquez non-blocking assignment <=. Quelle structure ou idée hardware se cache derrière ? Réponse : Une affectation non-blocking planifie la mise à jour pour la phase NBA de l'instant simulé. Plusieurs registres clockés utilisent ainsi leurs anciennes valeurs puis sont mis à jour conceptuellement ensemble.	EN - Explain non-blocking assignment <=. What hardware structure or idea is behind it? Answer: A non-blocking assignment schedules the update for the NBA phase of the simulation time step. Multiple clocked registers therefore use old values and conceptually update together.
146. FR - Comment utiliseriez-vous non-blocking assignment <= dans un design SystemVerilog réel ? Réponse : Je l'utilise dans `always @(posedge clk)` pour pipeline, compteur et state register. <pre>always @(posedge clk) begin q1 <= d; q2 <= q1; end</pre>	EN - How would you use non-blocking assignment <= in a real SystemVerilog design? Answer: I use it in `always @(posedge clk)` for pipelines, counters, and state registers. <pre>always @(posedge clk) begin q1 <= d; q2 <= q1; end</pre>
147. FR - Comment diagnostiqueriez-vous un problème lié à non-blocking assignment <= ? Réponse : Je recherche d'abord le mélange de `=` et `<=`, puis je vérifie si les dépendances sont voulues dans le même événement. Les bugs de scheduling sont parmi les pièges classiques de SystemVerilog. Pour non-blocking assignment <=, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving non-blocking assignment <=? Answer: I first look for mixing `=` and `<=`, then verify whether same-event dependencies are intentional. Scheduling bugs are among classic SystemVerilog pitfalls. For non-blocking assignment <=, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
148. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant non-blocking assignment <= ? Réponse : Règle de base: blocking `=` pour la combinatoire procédurale, non-blocking `<=` pour les registres clockés. Je ne déroge à cette règle qu'avec une raison claire et revue. Pour non-blocking assignment <=, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding non-blocking assignment <=? Answer: Base rule: blocking `=` for procedural combinational logic, non-blocking `<=` for clocked registers. I deviate only with a clear, reviewed reason. For non-blocking assignment <=, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
149. FR - Expliquez pipeline avec non-blocking. Quelle structure ou idée hardware se cache derrière ? Réponse : Avec `q1 <= d; q2 <= q1;`, `q2` reçoit l'ancienne valeur de `q1`, ce qui modélise deux registres en série.	EN - Explain pipeline with non-blocking assignments. What hardware structure or idea is behind it? Answer: With `q1 <= d; q2 <= q1;`, `q2` receives the old value of `q1`, modeling two registers in series.
150. FR - Comment utiliseriez-vous pipeline avec non-blocking dans un design SystemVerilog réel ? Réponse : Je peux expliquer sur waveform que l'information avance d'un étage par front d'horloge.	EN - How would you use pipeline with non-blocking assignments in a real SystemVerilog design? Answer: I can explain on a waveform that information advances one stage per clock edge.

151. FR - Comment diagnostiqueriez-vous un problème lié à pipeline avec non-blocking ? Réponse : Je recherche d'abord le mélange de `=` et `<=`, puis je vérifie si les dépendances sont voulues dans le même événement. Les bugs de scheduling sont parmi les pièges classiques de SystemVerilog. Pour pipeline avec non-blocking, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving pipeline with non-blocking assignments? Answer: I first look for mixing `=` and `<=`, then verify whether same-event dependencies are intentional. Scheduling bugs are among classic SystemVerilog pitfalls. For pipeline with non-blocking assignments, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
152. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant pipeline avec non-blocking ? Réponse : Règle de base: blocking `=` pour la combinatoire procédurale, non-blocking `<=` pour les registres clockés. Je ne déroge à cette règle qu'avec une raison claire et revue. Pour pipeline avec non-blocking, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding pipeline with non-blocking assignments? Answer: Base rule: blocking `=` for procedural combinational logic, non-blocking `<=` for clocked registers. I deviate only with a clear, reviewed reason. For pipeline with non-blocking assignments, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
153. FR - Expliquez race condition de simulation. Quelle structure ou idée hardware se cache derrière ? Réponse : Des blocs procéduraux utilisant des blocking assignments sur des signaux partagés au même timestamp peuvent produire un résultat dépendant de l'ordre d'exécution du simulateur.	EN - Explain simulation race condition. What hardware structure or idea is behind it? Answer: Procedural blocks using blocking assignments on shared signals at the same timestamp can produce results dependent on simulator execution ordering.
154. FR - Comment utiliseriez-vous race condition de simulation dans un design SystemVerilog réel ? Réponse : Je supprime les dépendances de race avec un ownership clair et des non-blocking assignments pour l'état séquentiel.	EN - How would you use simulation race condition in a real SystemVerilog design? Answer: I remove race dependencies through clear ownership and non-blocking assignments for sequential state.
155. FR - Comment diagnostiqueriez-vous un problème lié à race condition de simulation ? Réponse : Je recherche d'abord le mélange de `=` et `<=`, puis je vérifie si les dépendances sont voulues dans le même événement. Les bugs de scheduling sont parmi les pièges classiques de SystemVerilog. Pour race condition de simulation, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving simulation race condition? Answer: I first look for mixing `=` and `<=`, then verify whether same-event dependencies are intentional. Scheduling bugs are among classic SystemVerilog pitfalls. For simulation race condition, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
156. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant race condition de simulation ? Réponse : Règle de base: blocking `=` pour la combinatoire procédurale, non-blocking `<=` pour les registres clockés. Je ne déroge à cette règle qu'avec une raison claire et revue. Pour race condition de simulation, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding simulation race condition? Answer: Base rule: blocking `=` for procedural combinational logic, non-blocking `<=` for clocked registers. I deviate only with a clear, reviewed reason. For simulation race condition, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
157. FR - Expliquez mélange = et <= dans un bloc. Quelle structure ou idée hardware se cache derrière ? Réponse : Le mélange peut être légal mais rend la sémantique plus difficile à lire et peut créer des différences subtiles de simulation. Il faut distinguer variables temporaires combinatoires et vrais registres.	EN - Explain mixing = and <= in one block. What hardware structure or idea is behind it? Answer: Mixing can be legal but makes semantics harder to read and can create subtle simulation differences. Temporary combinational variables and true registers must be distinguished.
158. FR - Comment utiliseriez-vous mélange = et <= dans un bloc dans un design SystemVerilog réel ? Réponse : En entretien, je préfère montrer un style discipliné plutôt qu'un code "astucieux" dont le scheduling est ambigu.	EN - How would you use mixing = and <= in one block in a real SystemVerilog design? Answer: In an interview, I prefer disciplined style over "clever" code with ambiguous scheduling.
159. FR - Comment diagnostiqueriez-vous un problème lié à mélange = et <= dans un bloc ? Réponse : Je recherche d'abord le mélange de `=` et `<=`, puis je vérifie si les dépendances sont voulues dans le même événement. Les bugs de scheduling sont parmi les pièges classiques de SystemVerilog. Pour mélange = et <= dans un bloc, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving mixing = and <= in one block? Answer: I first look for mixing `=` and `<=`, then verify whether same-event dependencies are intentional. Scheduling bugs are among classic SystemVerilog pitfalls. For mixing = and <= in one block, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

160. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant mélange = et <= dans un bloc ? Réponse : Règle de base: blocking `=` pour la combinatoire procédurale, non-blocking `<=` pour les registres clockés. Je ne déroge à cette règle qu'avec une raison claire et revue. Pour mélange = et <= dans un bloc, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding mixing = and <= in one block? Answer: Base rule: blocking `=` for procedural combinational logic, non-blocking `<=` for clocked registers. I deviate only with a clear, reviewed reason. For mixing = and <= in one block, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
--	---

9. Registres, horloge, reset et enable / Registers, Clock, Reset, and Enable

161. FR - Expliquez D flip-flop. Quelle structure ou idée hardware se cache derrière ? Réponse : Un bloc `always @(posedge clk)` avec `q <= d;` infère un flip-flop D sensible au front montant.	EN - Explain D flip-flop. What hardware structure or idea is behind it? Answer: An `always @(posedge clk)` block with `q <= d;` infers a rising-edge-triggered D flip-flop.
---	--

162. FR - Comment utiliserez-vous D flip-flop dans un design SystemVerilog réel ? Réponse : Je l'utilise comme élément de base des pipelines et registres de contrôle. <pre>always @(posedge clk) begin q <= d; end</pre>	EN - How would you use D flip-flop in a real SystemVerilog design? Answer: I use it as the basic element of pipelines and control registers. <pre>always @(posedge clk) begin q <= d; end</pre>
--	--

163. FR - Comment diagnostiqueriez-vous un problème lié à D flip-flop ? Réponse : Je vérifie le front d'horloge, reset, enable, valeurs initiales attendues et tous les chemins d'affectation. Je regarde aussi si une entrée asynchrone a été synchronisée avant d'atteindre la logique clockée. Pour D flip-flop, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving D flip-flop? Answer: I check the clock edge, reset, enable, expected initialization, and every assignment path. I also check whether asynchronous inputs are synchronized before reaching clocked logic. For D flip-flop, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
--	---

164. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant D flip-flop ? Réponse : Je garde un domaine d'horloge clair par bloc et utilise des non-blocking assignments. Le choix reset sync/async est une décision d'architecture, pas une habitude syntaxique. Pour D flip-flop, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding D flip-flop? Answer: I keep one clear clock domain per block and use non-blocking assignments. Choosing synchronous or asynchronous reset is an architectural decision, not a syntax habit. For D flip-flop, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
--	--

165. FR - Expliquez reset synchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Un reset synchrone est testé à l'intérieur du bloc après l'événement d'horloge; il n'agit qu'au front actif du clock.	EN - Explain synchronous reset. What hardware structure or idea is behind it? Answer: A synchronous reset is tested inside the block after the clock event; it acts only on the active clock edge.
--	--

166. FR - Comment utiliserez-vous reset synchrone dans un design SystemVerilog réel ? Réponse : Je l'utilise lorsque l'architecture privilégie une logique de reset synchronisée et que le clock est garanti disponible. <pre>always @(posedge clk) begin if (rst) q <= 1'b0; else q <= d; end</pre>	EN - How would you use synchronous reset in a real SystemVerilog design? Answer: I use it when the architecture favors synchronized reset logic and the clock is guaranteed to be available. <pre>always @(posedge clk) begin if (rst) q <= 1'b0; else q <= d; end</pre>
--	--

167. FR - Comment diagnostiqueriez-vous un problème lié à reset synchrone ? Réponse : Je vérifie le front d'horloge, reset, enable, valeurs initiales attendues et tous les chemins d'affectation. Je regarde aussi si une entrée asynchrone a été synchronisée avant d'atteindre la logique clockée. Pour reset synchrone, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving synchronous reset? Answer: I check the clock edge, reset, enable, expected initialization, and every assignment path. I also check whether asynchronous inputs are synchronized before reaching clocked logic. For synchronous reset, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
--	---

168. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant reset synchrone ? Réponse : Je garde un domaine d'horloge clair par bloc et utilise des non-blocking assignments. Le choix reset sync/async est une décision d'architecture, pas une habitude syntaxique. Pour reset synchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding synchronous reset? Answer: I keep one clear clock domain per block and use non-blocking assignments. Choosing synchronous or asynchronous reset is an architectural decision, not a syntax habit. For synchronous reset, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
169. FR - Expliquez reset asynchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Un reset asynchrone apparaît dans la sensitivity list, par exemple `posedge rst`, et peut modifier le registre indépendamment du clock. Sa désassertion doit être traitée soigneusement vis-à-vis du domaine d'horloge.	EN - Explain asynchronous reset. What hardware structure or idea is behind it? Answer: An asynchronous reset appears in the sensitivity list, for example `posedge rst`, and can change the register independently of the clock. Its deassertion must be handled carefully relative to the clock domain.
170. FR - Comment utiliseriez-vous reset asynchrone dans un design SystemVerilog réel ? Réponse : Je peux utiliser assertion asynchrone et désassertion synchronisée selon la stratégie reset du système. <pre>always @(posedge clk or posedge rst) begin if (rst) q <= 1'b0; else q <= d; end</pre>	EN - How would you use asynchronous reset in a real SystemVerilog design? Answer: I can use asynchronous assertion and synchronized deassertion according to the system reset strategy. <pre>always @(posedge clk or posedge rst) begin if (rst) q <= 1'b0; else q <= d; end</pre>
171. FR - Comment diagnostiqueriez-vous un problème lié à reset asynchrone ? Réponse : Je vérifie le front d'horloge, reset, enable, valeurs initiales attendues et tous les chemins d'affectation. Je regarde aussi si une entrée asynchrone a été synchronisée avant d'atteindre la logique clockée. Pour reset asynchrone, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving asynchronous reset? Answer: I check the clock edge, reset, enable, expected initialization, and every assignment path. I also check whether asynchronous inputs are synchronized before reaching clocked logic. For asynchronous reset, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
172. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant reset asynchrone ? Réponse : Je garde un domaine d'horloge clair par bloc et utilise des non-blocking assignments. Le choix reset sync/async est une décision d'architecture, pas une habitude syntaxique. Pour reset asynchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding asynchronous reset? Answer: I keep one clear clock domain per block and use non-blocking assignments. Choosing synchronous or asynchronous reset is an architectural decision, not a syntax habit. For asynchronous reset, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
173. FR - Expliquez clock enable. Quelle structure ou idée hardware se cache derrière ? Réponse : Un `if (en)` dans un bloc clocké conserve naturellement l'ancienne valeur lorsque `en=0`, ce qui infère un enable de registre sans fabriquer une nouvelle horloge.	EN - Explain clock enable. What hardware structure or idea is behind it? Answer: An `if (en)` inside a clocked block naturally holds the old value when `en=0`, inferring a register enable without creating a new clock.
174. FR - Comment utiliseriez-vous clock enable dans un design SystemVerilog réel ? Réponse : Je préfère un enable synchrone à une horloge divisée par logique pour ralentir une opération. <pre>always @(posedge clk) begin if (en) count <= count + 1'b1; end</pre>	EN - How would you use clock enable in a real SystemVerilog design? Answer: I prefer a synchronous enable over a logic-divided clock to slow an operation. <pre>always @(posedge clk) begin if (en) count <= count + 1'b1; end</pre>
175. FR - Comment diagnostiqueriez-vous un problème lié à clock enable ? Réponse : Je vérifie le front d'horloge, reset, enable, valeurs initiales attendues et tous les chemins d'affectation. Je regarde aussi si une entrée asynchrone a été synchronisée avant d'atteindre la logique clockée. Pour clock enable, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock enable? Answer: I check the clock edge, reset, enable, expected initialization, and every assignment path. I also check whether asynchronous inputs are synchronized before reaching clocked logic. For clock enable, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

176. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock enable ? Réponse : Je garde un domaine d'horloge clair par bloc et utilise des non-blocking assignments. Le choix reset sync/async est une décision d'architecture, pas une habitude syntaxique. Pour clock enable, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock enable? Answer: I keep one clear clock domain per block and use non-blocking assignments. Choosing synchronous or asynchronous reset is an architectural decision, not a syntax habit. For clock enable, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
177. FR - Expliquez clock gating versus enable. Quelle structure ou idée hardware se cache derrière ? Réponse : Gater une horloge avec de la logique RTL ordinaire peut introduire glitches, skew et problèmes de timing. Les FPGA disposent généralement de ressources dédiées si le gating est réellement nécessaire.	EN - Explain clock gating versus enable. What hardware structure or idea is behind it? Answer: Gating a clock with ordinary RTL logic can introduce glitches, skew, and timing problems. FPGAs generally provide dedicated resources if gating is truly required.
178. FR - Comment utiliseriez-vous clock gating versus enable dans un design SystemVerilog réel ? Réponse : Pour une logique fonctionnelle simple, je garde le clock propre et utilise un `enable`.	EN - How would you use clock gating versus enable in a real SystemVerilog design? Answer: For simple functional logic, I keep the clock clean and use an `enable`.
179. FR - Comment diagnostiqueriez-vous un problème lié à clock gating versus enable ? Réponse : Je vérifie le front d'horloge, reset, enable, valeurs initiales attendues et tous les chemins d'affectation. Je regarde aussi si une entrée asynchrone a été synchronisée avant d'atteindre la logique clockée. Pour clock gating versus enable, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock gating versus enable? Answer: I check the clock edge, reset, enable, expected initialization, and every assignment path. I also check whether asynchronous inputs are synchronized before reaching clocked logic. For clock gating versus enable, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
180. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock gating versus enable ? Réponse : Je garde un domaine d'horloge clair par bloc et utilise des non-blocking assignments. Le choix reset sync/async est une décision d'architecture, pas une habitude syntaxique. Pour clock gating versus enable, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock gating versus enable? Answer: I keep one clear clock domain per block and use non-blocking assignments. Choosing synchronous or asynchronous reset is an architectural decision, not a syntax habit. For clock gating versus enable, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

10. Compteurs, timers, diviseurs et PWM / Counters, Timers, Dividers, and PWM

181. FR - Expliquez compteur binaire. Quelle structure ou idée hardware se cache derrière ? Réponse : Un registre auquel on affecte `count <= count + 1` à chaque front synthétise un compteur avec additionneur et banque de flip-flops.	EN - Explain binary counter. What hardware structure or idea is behind it? Answer: A register assigned `count <= count + 1` on each edge synthesizes a counter with an adder and flip-flop bank.
182. FR - Comment utiliseriez-vous compteur binaire dans un design SystemVerilog réel ? Réponse : Je choisis la largeur pour éviter un overflow non voulu et je définis explicitement reset/enable. <pre>always @(posedge clk) begin if (rst) count <= 8'd0; else if (en) count <= count + 8'd1; end</pre>	EN - How would you use binary counter in a real SystemVerilog design? Answer: I choose the width to avoid unintended overflow and explicitly define reset/enable. <pre>always @(posedge clk) begin if (rst) count <= 8'd0; else if (en) count <= count + 8'd1; end</pre>
183. FR - Comment diagnostiqueriez-vous un problème lié à compteur binaire ? Réponse : Je recalcule les rapports fréquence/compte, vérifie overflow et largeur, puis j'observe les pulses générés sur waveform. Une erreur off-by-one est fréquente dans les timers. Pour compteur binaire, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving binary counter? Answer: I recompute frequency/count ratios, check overflow and width, then inspect generated pulses on the waveform. Off-by-one errors are common in timers. For binary counter, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

184. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant compteur binaire ? Réponse : Je préfère générer des clock-enables plutôt que des pseudo-clocks avec des bascules RTL. Je dimensionne les compteurs à partir de la valeur maximale réellement nécessaire. Pour compteur binaire, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding binary counter? Answer: I prefer generating clock-enables rather than pseudo-clocks with RTL toggles. I size counters from the actual maximum value required. For binary counter, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
185. FR - Expliquez timer et pulse périodique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un compteur comparé à `N-1` peut produire une impulsion d'un cycle toutes les N périodes d'horloge.	EN - Explain timer and periodic pulse. What hardware structure or idea is behind it? Answer: A counter compared to `N-1` can produce a one-cycle pulse every N clock periods.
186. FR - Comment utiliseriez-vous timer et pulse périodique dans un design SystemVerilog réel ? Réponse : Je calcule `N = Fclk/Fevent`, puis vérifie si la division est entière et si le compteur couvre `N-1`.	EN - How would you use timer and periodic pulse in a real SystemVerilog design? Answer: I calculate `N = Fclk/Fevent`, then verify whether the division is exact and whether the counter covers `N-1`.
187. FR - Comment diagnostiqueriez-vous un problème lié à timer et pulse périodique ? Réponse : Je recalcule les rapports fréquence/compte, vérifie overflow et largeur, puis j'observe les pulses générés sur waveform. Une erreur off-by-one est fréquente dans les timers. Pour timer et pulse périodique, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving timer and periodic pulse? Answer: I recompute frequency/count ratios, check overflow and width, then inspect generated pulses on the waveform. Off-by-one errors are common in timers. For timer and periodic pulse, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
188. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant timer et pulse périodique ? Réponse : Je préfère générer des clock-enables plutôt que des pseudo-clocks avec des bascules RTL. Je dimensionne les compteurs à partir de la valeur maximale réellement nécessaire. Pour timer et pulse périodique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding timer and periodic pulse? Answer: I prefer generating clock-enables rather than pseudo-clocks with RTL toggles. I size counters from the actual maximum value required. For timer and periodic pulse, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
189. FR - Expliquez clock divider par toggle. Quelle structure ou idée hardware se cache derrière ? Réponse : Basculer un registre toutes les N périodes crée un signal plus lent, mais ce signal devient un nouveau clock domain s'il clocke d'autres registres.	EN - Explain toggle clock divider. What hardware structure or idea is behind it? Answer: Toggling a register every N periods creates a slower signal, but that signal becomes a new clock domain if it clocks other registers.
190. FR - Comment utiliseriez-vous clock divider par toggle dans un design SystemVerilog réel ? Réponse : Je préfère souvent un clock-enable périodique afin de rester dans un seul domaine d'horloge.	EN - How would you use toggle clock divider in a real SystemVerilog design? Answer: I often prefer a periodic clock-enable to stay within one clock domain.
191. FR - Comment diagnostiqueriez-vous un problème lié à clock divider par toggle ? Réponse : Je recalcule les rapports fréquence/compte, vérifie overflow et largeur, puis j'observe les pulses générés sur waveform. Une erreur off-by-one est fréquente dans les timers. Pour clock divider par toggle, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving toggle clock divider? Answer: I recompute frequency/count ratios, check overflow and width, then inspect generated pulses on the waveform. Off-by-one errors are common in timers. For toggle clock divider, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
192. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock divider par toggle ? Réponse : Je préfère générer des clock-enables plutôt que des pseudo-clocks avec des bascules RTL. Je dimensionne les compteurs à partir de la valeur maximale réellement nécessaire. Pour clock divider par toggle, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding toggle clock divider? Answer: I prefer generating clock-enables rather than pseudo-clocks with RTL toggles. I size counters from the actual maximum value required. For toggle clock divider, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

193. FR - Expliquez PWM. Quelle structure ou idée hardware se cache derrière ? Réponse : Un PWM compare un compteur de phase à une valeur duty. Tant que `counter < duty`, la sortie est active; fréquence et résolution dépendent de la période du compteur.	EN - Explain PWM. What hardware structure or idea is behind it? Answer: A PWM compares a phase counter with a duty value. While `counter < duty`, the output is active; frequency and resolution depend on the counter period.
194. FR - Comment utiliseriez-vous PWM dans un design SystemVerilog réel ? Réponse : Je garde `duty` et le compteur de même largeur et traite explicitement 0% et 100%. assign pwm = (count < duty);	EN - How would you use PWM in a real SystemVerilog design? Answer: I keep `duty` and the counter at the same width and explicitly handle 0% and 100%. assign pwm = (count < duty);
195. FR - Comment diagnostiqueriez-vous un problème lié à PWM ? Réponse : Je recalcule les rapports fréquence/compte, vérifie overflow et largeur, puis j'observe les pulses générés sur waveform. Une erreur off-by-one est fréquente dans les timers. Pour PWM, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving PWM? Answer: I recompute frequency/count ratios, check overflow and width, then inspect generated pulses on the waveform. Off-by-one errors are common in timers. For PWM, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
196. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant PWM ? Réponse : Je préfère générer des clock-enables plutôt que des pseudo-clocks avec des bascules RTL. Je dimensionne les compteurs à partir de la valeur maximale réellement nécessaire. Pour PWM, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding PWM? Answer: I prefer generating clock-enables rather than pseudo-clocks with RTL toggles. I size counters from the actual maximum value required. For PWM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
197. FR - Expliquez edge detector synchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Un détecteur de front mémorise la valeur précédente d'un signal synchronisé et compare courant/précédent. Un front montant peut être `sync & ~sync_d`.	EN - Explain synchronous edge detector. What hardware structure or idea is behind it? Answer: A synchronous edge detector stores the previous value of a synchronized signal and compares current/previous values. A rising edge can be `sync & ~sync_d`.
198. FR - Comment utiliseriez-vous edge detector synchrone dans un design SystemVerilog réel ? Réponse : Je ne fais l'edge detection qu'après synchronisation si la source est asynchrone au clock local. always @(posedge clk) sync_d <= sync; assign rise = sync & ~sync_d;	EN - How would you use synchronous edge detector in a real SystemVerilog design? Answer: I perform edge detection only after synchronization when the source is asynchronous to the local clock. always @(posedge clk) sync_d <= sync; assign rise = sync & ~sync_d;
199. FR - Comment diagnostiqueriez-vous un problème lié à edge detector synchrone ? Réponse : Je recalcule les rapports fréquence/compte, vérifie overflow et largeur, puis j'observe les pulses générés sur waveform. Une erreur off-by-one est fréquente dans les timers. Pour edge detector synchrone, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving synchronous edge detector? Answer: I recompute frequency/count ratios, check overflow and width, then inspect generated pulses on the waveform. Off-by-one errors are common in timers. For synchronous edge detector, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
200. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant edge detector synchrone ? Réponse : Je préfère générer des clock-enables plutôt que des pseudo-clocks avec des bascules RTL. Je dimensionne les compteurs à partir de la valeur maximale réellement nécessaire. Pour edge detector synchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding synchronous edge detector? Answer: I prefer generating clock-enables rather than pseudo-clocks with RTL toggles. I size counters from the actual maximum value required. For synchronous edge detector, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

11. FSM avec `enum`, états et propriétés / FSMs with `enum`, States, and Properties

201. FR - Expliquez FSM avec `typedef enum`. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une FSM SystemVerilog utilise souvent un type `enum` nommé pour les états, ce qui rend les waveforms et les `case` beaucoup plus lisibles qu'avec des `localparam` numériques. typedef enum logic [1:0] {IDLE, START, DATA, DONE} state_t;	EN - Explain FSM with `typedef enum`. What important difference, structure, or idea should be understood? Answer: A SystemVerilog FSM often uses a named `enum` type for states, making waveforms and `case` statements much more readable than numeric `localparam` values. typedef enum logic [1:0] {IDLE, START, DATA, DONE} state_t;
--	--

202. FR - Comment utiliseriez-vous FSM avec `typedef enum` dans un projet SystemVerilog réel ? Réponse : Je définis `state_t`, puis `state` et `next_state`, avec un `always_ff` pour le registre et un `always_comb` pour transitions/outputs. <pre>typedef enum logic [1:0] {IDLE, START, DATA, DONE} state_t;</pre>	EN - How would you use FSM with `typedef enum` in a real SystemVerilog project? Answer: I define `state_t`, then `state` and `next_state`, with an `always_ff` for the state register and an `always_comb` for transitions/outputs. <pre>typedef enum logic [1:0] {IDLE, START, DATA, DONE} state_t;</pre>
203. FR - Comment diagnostiqueriez-vous un problème lié à FSM avec `typedef enum` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour FSM avec `typedef enum`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to FSM with `typedef enum` ? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For FSM with `typedef enum`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
204. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant FSM avec `typedef enum` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour FSM avec `typedef enum`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding FSM with `typedef enum` ? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For FSM with `typedef enum`, I also explain the expected hardware or simulation-only role, together with important corner cases.
205. FR - Expliquez Moore versus Mealy en SystemVerilog. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une Moore fait dépendre les sorties principalement de l'état; une Mealy permet aussi une dépendance combinatoire aux entrées. Le choix influence latence, timing et risque de glitch.	EN - Explain Moore versus Mealy in SystemVerilog. What important difference, structure, or idea should be understood? Answer: A Moore FSM makes outputs depend mainly on state; a Mealy FSM also allows combinational dependence on inputs. The choice affects latency, timing, and glitch risk.
206. FR - Comment utiliseriez-vous Moore versus Mealy en SystemVerilog dans un projet SystemVerilog réel ? Réponse : Je choisis selon le protocole et j'enregistre les sorties critiques si une réaction purement combinatoire est risquée.	EN - How would you use Moore versus Mealy in SystemVerilog in a real SystemVerilog project? Answer: I choose based on the protocol and register critical outputs if purely combinational reaction is risky.
207. FR - Comment diagnostiqueriez-vous un problème lié à Moore versus Mealy en SystemVerilog ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour Moore versus Mealy en SystemVerilog, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to Moore versus Mealy in SystemVerilog? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For Moore versus Mealy in SystemVerilog, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
208. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant Moore versus Mealy en SystemVerilog ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour Moore versus Mealy en SystemVerilog, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding Moore versus Mealy in SystemVerilog? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For Moore versus Mealy in SystemVerilog, I also explain the expected hardware or simulation-only role, together with important corner cases.
209. FR - Expliquez `unique case` dans une FSM. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `unique case` peut signaler qu'aucune branche ou plusieurs branches ne correspondent alors qu'une seule était attendue. Il sert de check de simulation autant que d'indication d'intention.	EN - Explain `unique case` in an FSM. What important difference, structure, or idea should be understood? Answer: `unique case` can warn when no branch or multiple branches match while exactly one was expected. It acts as a simulation check as well as an intent marker.
210. FR - Comment utiliseriez-vous `unique case` dans une FSM dans un projet SystemVerilog réel ? Réponse : Je l'utilise seulement si l'exclusivité/couverture complète est réellement garantie par la conception.	EN - How would you use `unique case` in an FSM in a real SystemVerilog project? Answer: I use it only when exclusivity/full coverage is genuinely guaranteed by design.

211. FR - Comment diagnostiqueriez-vous un problème lié à `unique case` dans une FSM ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `unique case` dans une FSM, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `unique case` in an FSM? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `unique case` in an FSM, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
212. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `unique case` dans une FSM ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour `unique case` dans une FSM, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `unique case` in an FSM? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `unique case` in an FSM, I also explain the expected hardware or simulation-only role, together with important corner cases.
213. FR - Expliquez états illégaux et assertions de FSM. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une assertion peut vérifier que `state` appartient toujours à l'ensemble légal et qu'une transition critique respecte un délai. Cela détecte les violations plus tôt qu'une simple observation de sortie.	EN - Explain illegal states and FSM assertions. What important difference, structure, or idea should be understood? Answer: An assertion can verify that `state` always belongs to the legal set and that a critical transition meets a timing bound. This detects violations earlier than output-only checking.
214. FR - Comment utiliseriez-vous états illégaux et assertions de FSM dans un projet SystemVerilog réel ? Réponse : J'ajoute des assertions sur les invariants importants et garde une stratégie de récupération distincte de la détection de bug.	EN - How would you use illegal states and FSM assertions in a real SystemVerilog project? Answer: I add assertions for important invariants and keep recovery strategy distinct from bug detection.
215. FR - Comment diagnostiqueriez-vous un problème lié à états illégaux et assertions de FSM ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour états illégaux et assertions de FSM, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to illegal states and FSM assertions? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For illegal states and FSM assertions, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
216. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant états illégaux et assertions de FSM ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour états illégaux et assertions de FSM, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding illegal states and FSM assertions? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For illegal states and FSM assertions, I also explain the expected hardware or simulation-only role, together with important corner cases.
217. FR - Expliquez encodage d'état versus type symbolique. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : `enum` donne des noms symboliques, tandis que l'encodage physique peut être choisi par le designer ou optimisé par le synthétiseur. L'interface fonctionnelle ne devrait pas dépendre des bits internes d'état.	EN - Explain state encoding versus symbolic type. What important difference, structure, or idea should be understood? Answer: `enum` provides symbolic names, while physical encoding may be chosen by the designer or optimized by synthesis. Functional interfaces should not depend on internal state bits.
218. FR - Comment utiliseriez-vous encodage d'état versus type symbolique dans un projet SystemVerilog réel ? Réponse : Je laisse souvent l'outil choisir l'encodage sauf exigence mesurée de timing, safety ou debug.	EN - How would you use state encoding versus symbolic type in a real SystemVerilog project? Answer: I often let the tool choose encoding unless there is a measured timing, safety, or debug requirement.
219. FR - Comment diagnostiqueriez-vous un problème lié à encodage d'état versus type symbolique ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour encodage d'état versus type symbolique, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to state encoding versus symbolic type? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For state encoding versus symbolic type, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

220. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant encodage d'état versus type symbolique ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour encodage d'état versus type symbolique, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding state encoding versus symbolic type? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For state encoding versus symbolic type, I also explain the expected hardware or simulation-only role, together with important corner cases.
---	--

12. Hiérarchie, paramètres, packages, interfaces et generate / Hierarchy, Parameters, Packages, Interfaces, and Generate

221. FR - Expliquez <code>`parameter`</code> et <code>`localparam`</code> typés. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : SystemVerilog permet de typer les paramètres. <code>`parameter`</code> expose une configuration à l'instance, tandis que <code>`localparam`</code> protège une constante dérivée interne.	EN - Explain typed <code>`parameter`</code> and <code>`localparam`</code>. What important difference, structure, or idea should be understood? Answer: SystemVerilog allows typed parameters. <code>`parameter`</code> exposes configuration at instantiation, while <code>`localparam`</code> protects an internal derived constant.
---	---

222. FR - Comment utiliseriez-vous <code>`parameter`</code> et <code>`localparam`</code> typés dans un projet SystemVerilog réel ? Réponse : Je type largeur, profondeur et modes configurables, puis je calcule les constantes internes avec <code>`localparam`</code>.	EN - How would you use typed <code>`parameter`</code> and <code>`localparam`</code> in a real SystemVerilog project? Answer: I type configurable widths, depths, and modes, then compute internal constants with <code>`localparam`</code>.
--	---

223. FR - Comment diagnostiqueriez-vous un problème lié à <code>`parameter`</code> et <code>`localparam`</code> typés ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour <code>`parameter`</code> et <code>`localparam`</code> typés, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to typed <code>`parameter`</code> and <code>`localparam`</code>? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For typed <code>`parameter`</code> and <code>`localparam`</code>, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
--	---

224. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant <code>`parameter`</code> et <code>`localparam`</code> typés ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour <code>`parameter`</code> et <code>`localparam`</code> typés, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding typed <code>`parameter`</code> and <code>`localparam`</code>? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For typed <code>`parameter`</code> and <code>`localparam`</code>, I also explain the expected hardware or simulation-only role, together with important corner cases.
---	--

225. FR - Expliquez packages et <code>`import`</code>. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un package regroupe types, paramètres et fonctions partagés dans un namespace compilé. Il réduit les <code>`include`</code> textuels et les définitions dupliquées.	EN - Explain packages and <code>`import`</code>. What important difference, structure, or idea should be understood? Answer: A package groups shared types, parameters, and functions in a compiled namespace. It reduces textual <code>`include`</code> dependencies and duplicate definitions.
---	--

226. FR - Comment utiliseriez-vous packages et <code>`import`</code> dans un projet SystemVerilog réel ? Réponse : Je place les protocol types et constantes communes dans un package compilé avant les modules qui l'importent.	EN - How would you use packages and <code>`import`</code> in a real SystemVerilog project? Answer: I place common protocol types and constants in a package compiled before modules that import it.
--	---

227. FR - Comment diagnostiqueriez-vous un problème lié à packages et <code>`import`</code> ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour packages et <code>`import`</code>, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to packages and <code>`import`</code>? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For packages and <code>`import`</code>, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
--	---

228. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant packages et <code>`import`</code> ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour packages et <code>`import`</code>, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding packages and <code>`import`</code>? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For packages and <code>`import`</code>, I also explain the expected hardware or simulation-only role, together with important corner cases.
---	--

229. FR - Expliquez interfaces et modports. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une `interface` regroupe les signaux d'un protocole; un `modport` définit la vue et les directions pour un rôle source/sink ou master/slave.	EN - Explain interfaces and modports. What important difference, structure, or idea should be understood? Answer: An `interface` groups protocol signals; a `modport` defines the view and directions for a source/sink or master/slave role.
230. FR - Comment utiliseriez-vous interfaces et modports dans un projet SystemVerilog réel ? Réponse : Je les utilise pour réduire les longues listes de ports et rendre les rôles du protocole explicites lorsque le toolchain les supporte bien.	EN - How would you use interfaces and modports in a real SystemVerilog project? Answer: I use them to reduce long port lists and make protocol roles explicit when the toolchain supports them well.
231. FR - Comment diagnostiqueriez-vous un problème lié à interfaces et modports ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour interfaces et modports, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to interfaces and modports? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For interfaces and modports, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
232. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant interfaces et modports ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour interfaces et modports, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding interfaces and modports? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For interfaces and modports, I also explain the expected hardware or simulation-only role, together with important corner cases.
233. FR - Expliquez `generate` et paramétrisation structurelle. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Les generate blocks créent ou répètent de la structure à l'élaboration selon des paramètres statiques. Ils décrivent donc des copies de hardware, pas une boucle exécutée dans le temps.	EN - Explain `generate` and structural parameterization. What important difference, structure, or idea should be understood? Answer: Generate blocks create or repeat structure at elaboration according to static parameters. They therefore describe hardware copies, not a time-executed loop.
234. FR - Comment utiliseriez-vous `generate` et paramétrisation structurelle dans un projet SystemVerilog réel ? Réponse : Je les utilise pour N canaux, largeurs configurables ou blocs optionnels, avec des noms de generate stables.	EN - How would you use `generate` and structural parameterization in a real SystemVerilog project? Answer: I use them for N channels, configurable widths, or optional blocks, with stable generate names.
235. FR - Comment diagnostiqueriez-vous un problème lié à `generate` et paramétrisation structurelle ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `generate` et paramétrisation structurelle, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `generate` and structural parameterization? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `generate` and structural parameterization, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
236. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `generate` et paramétrisation structurelle ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour `generate` et paramétrisation structurelle, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `generate` and structural parameterization? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `generate` and structural parameterization, I also explain the expected hardware or simulation-only role, together with important corner cases.
237. FR - Expliquez virtual interface côté verification. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une virtual interface est un handle de classe vers une instance d'interface statique. Elle permet à un driver/monitor class-based d'accéder au DUT sans référence hiérarchique codée en dur.	EN - Explain virtual interface on the verification side. What important difference, structure, or idea should be understood? Answer: A virtual interface is a class handle to a static interface instance. It lets a class-based driver/monitor access the DUT without hard-coded hierarchical references.
238. FR - Comment utiliseriez-vous virtual interface côté verification dans un projet SystemVerilog réel ? Réponse : Je la configure avant de démarrer les threads de verification et je vérifie explicitement qu'elle n'est pas null.	EN - How would you use virtual interface on the verification side in a real SystemVerilog project? Answer: I configure it before starting verification threads and explicitly check that it is not null.

239. FR - Comment diagnostiqueriez-vous un problème lié à virtual interface côté verification ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour virtual interface côté verification, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to virtual interface on the verification side? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For virtual interface on the verification side, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
240. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant virtual interface côté verification ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour virtual interface côté verification, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding virtual interface on the verification side? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For virtual interface on the verification side, I also explain the expected hardware or simulation-only role, together with important corner cases.

13. Functions, tasks, typedefs, macros et organisation du code / Functions, Tasks, Typedefs, Macros, and Code Organization

241. FR - Expliquez function SystemVerilog. Quelle structure ou idée hardware se cache derrière ? Réponse : Une function retourne une valeur et s'exécute sans consommer de temps de simulation dans le style synthétisable classique. Elle aide à factoriser un calcul combinatoire.	EN - Explain SystemVerilog function. What hardware structure or idea is behind it? Answer: A function returns a value and executes without consuming simulation time in classic synthesizable style. It helps factor a combinational calculation.
242. FR - Comment utiliseriez-vous function SystemVerilog dans un design SystemVerilog réel ? Réponse : Je peux créer une fonction de parity ou de conversion utilisée dans plusieurs expressions.	EN - How would you use SystemVerilog function in a real SystemVerilog design? Answer: I can create a parity or conversion function used in several expressions.
243. FR - Comment diagnostiqueriez-vous un problème lié à function SystemVerilog ? Réponse : Je vérifie la portée des déclarations, les side effects et la synthétisabilité. Les macros textuelles sont particulièrement sensibles aux collisions et à la lisibilité. Pour function SystemVerilog, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving SystemVerilog function? Answer: I verify declaration scope, side effects, and synthesizability. Textual macros are especially sensitive to collisions and readability issues. For SystemVerilog function, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
244. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant function SystemVerilog ? Réponse : J'utilise functions pour les calculs combinatoires réutilisables, tasks avec prudence, et `define` seulement lorsque la substitution textuelle est réellement la bonne abstraction. Pour function SystemVerilog, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding SystemVerilog function? Answer: I use functions for reusable combinational calculations, tasks carefully, and `define` only when textual substitution is truly the right abstraction. For SystemVerilog function, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
245. FR - Expliquez task. Quelle structure ou idée hardware se cache derrière ? Réponse : Une task peut avoir plusieurs entrées/sorties et, en testbench, contenir des contrôles temporels. En RTL synthétisable, son usage doit rester compatible avec le synthétiseur et sans timing non synthétisable.	EN - Explain task. What hardware structure or idea is behind it? Answer: A task can have multiple inputs/outputs and, in a testbench, contain timing controls. In synthesizable RTL, its use must remain synthesizer-compatible and free of nonsynthesizable timing.
246. FR - Comment utiliseriez-vous task dans un design SystemVerilog réel ? Réponse : Je l'utilise surtout dans testbench pour encapsuler "send_byte" ou "check_response".	EN - How would you use task in a real SystemVerilog design? Answer: I mainly use it in a testbench to encapsulate operations such as "send_byte" or "check_response".
247. FR - Comment diagnostiqueriez-vous un problème lié à task ? Réponse : Je vérifie la portée des déclarations, les side effects et la synthétisabilité. Les macros textuelles sont particulièrement sensibles aux collisions et à la lisibilité. Pour task, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving task? Answer: I verify declaration scope, side effects, and synthesizability. Textual macros are especially sensitive to collisions and readability issues. For task, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

248. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant task ? Réponse : J'utilise functions pour les calculs combinatoires réutilisables, tasks avec prudence, et `define` seulement lorsque la substitution textuelle est réellement la bonne abstraction. Pour task, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding task? Answer: I use functions for reusable combinational calculations, tasks carefully, and `define` only when textual substitution is truly the right abstraction. For task, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
249. FR - Expliquez `define macro. Quelle structure ou idée hardware se cache derrière ?</p> <p>Réponse : `define` effectue une substitution textuelle par le préprocesseur. C'est pratique pour des constantes globales simples mais moins typé et moins local qu'un parameter/localparam.	EN - Explain `define macro. What hardware structure or idea is behind it?</p> <p>Answer: `define` performs textual substitution through the preprocessor. It is convenient for simple global constants but less typed and less local than parameter/localparam.
250. FR - Comment utiliseriez-vous `define macro dans un design SystemVerilog réel ?</p> <p>Réponse : Je préfère `parameter`/`localparam` pour les valeurs de design et réserve les macros aux besoins de compilation conditionnelle ou conventions du projet.	EN - How would you use `define macro in a real SystemVerilog design?</p> <p>Answer: I prefer `parameter`/`localparam` for design values and reserve macros for conditional compilation needs or project conventions.
251. FR - Comment diagnostiqueriez-vous un problème lié à `define macro ?</p> <p>Réponse : Je vérifie la portée des déclarations, les side effects et la synthétisabilité. Les macros textuelles sont particulièrement sensibles aux collisions et à la lisibilité. Pour `define macro, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving `define macro?</p> <p>Answer: I verify declaration scope, side effects, and synthesizability. Textual macros are especially sensitive to collisions and readability issues. For `define macro, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
252. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `define macro ?</p> <p>Réponse : J'utilise functions pour les calculs combinatoires réutilisables, tasks avec prudence, et `define` seulement lorsque la substitution textuelle est réellement la bonne abstraction. Pour `define macro, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding `define macro?</p> <p>Answer: I use functions for reusable combinational calculations, tasks carefully, and `define` only when textual substitution is truly the right abstraction. For `define macro, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
253. FR - Expliquez `include. Quelle structure ou idée hardware se cache derrière ?</p> <p>Réponse : `include` insère textuellement le contenu d'un autre fichier avant compilation. Il peut partager des définitions mais crée aussi des dépendances de chemin et d'ordre.	EN - Explain `include. What hardware structure or idea is behind it?</p> <p>Answer: `include` textually inserts another file before compilation. It can share definitions but also creates path and ordering dependencies.
254. FR - Comment utiliseriez-vous `include dans un design SystemVerilog réel ? Réponse : Je garde les headers petits, avec guards si nécessaire, et je vérifie les include paths du flow.	EN - How would you use `include in a real SystemVerilog design? Answer: I keep header files small, add guards when needed, and verify the flow's include paths.
255. FR - Comment diagnostiqueriez-vous un problème lié à `include ?</p> <p>Réponse : Je vérifie la portée des déclarations, les side effects et la synthétisabilité. Les macros textuelles sont particulièrement sensibles aux collisions et à la lisibilité. Pour `include, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving `include?</p> <p>Answer: I verify declaration scope, side effects, and synthesizability. Textual macros are especially sensitive to collisions and readability issues. For `include, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
256. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `include ?</p> <p>Réponse : J'utilise functions pour les calculs combinatoires réutilisables, tasks avec prudence, et `define` seulement lorsque la substitution textuelle est réellement la bonne abstraction. Pour `include, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding `include?</p> <p>Answer: I use functions for reusable combinational calculations, tasks carefully, and `define` only when textual substitution is truly the right abstraction. For `include, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

257. FR - Expliquez compilation conditionnelle. Quelle structure ou idée hardware se cache derrière ? Réponse : Les directives <code>`ifdef/ifndef/else/endif`</code> permettent de compiler des variantes de texte selon les macros définies. Elles sont puissantes mais peuvent multiplier les configurations non testées.	EN - Explain conditional compilation. What hardware structure or idea is behind it? Answer: <code>`ifdef/ifndef/else/endif`</code> directives compile textual variants based on defined macros. They are powerful but can multiply untested configurations.
--	---

258. FR - Comment utiliseriez-vous compilation conditionnelle dans un design SystemVerilog réel ? Réponse : Je peux activer des assertions/debug simulation-only, tout en gardant les variantes de production maîtrisées.	EN - How would you use conditional compilation in a real SystemVerilog design? Answer: I can enable simulation-only assertions/debug while keeping production variants controlled.
---	--

259. FR - Comment diagnostiqueriez-vous un problème lié à compilation conditionnelle ? Réponse : Je vérifie la portée des déclarations, les side effects et la synthétisabilité. Les macros textuelles sont particulièrement sensibles aux collisions et à la lisibilité. Pour compilation conditionnelle, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving conditional compilation? Answer: I verify declaration scope, side effects, and synthesizability. Textual macros are especially sensitive to collisions and readability issues. For conditional compilation, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
---	---

260. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant compilation conditionnelle ? Réponse : J'utilise fonctions pour les calculs combinatoires réutilisables, tasks avec prudence, et <code>`define`</code> seulement lorsque la substitution textuelle est réellement la bonne abstraction. Pour compilation conditionnelle, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding conditional compilation? Answer: I use functions for reusable combinational calculations, tasks carefully, and <code>`define`</code> only when textual substitution is truly the right abstraction. For conditional compilation, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	--

14. Arrays, mémoire, ROM, RAM et FIFO / Arrays, Memory, ROM, RAM, and FIFO

261. FR - Expliquez mémoire reg array. Quelle structure ou idée hardware se cache derrière ? Réponse : Une déclaration <code>`reg [7:0] mem [0:255];`</code> décrit 256 mots de 8 bits en simulation. Selon le style d'accès, la synthèse peut inférer RAM distribuée, block RAM ou registres.	EN - Explain reg array memory. What hardware structure or idea is behind it? Answer: A declaration <code>`reg [7:0] mem [0:255];`</code> describes 256 eight-bit words in simulation. Depending on access style, synthesis may infer distributed RAM, block RAM, or registers.
--	--

262. FR - Comment utiliseriez-vous mémoire reg array dans un design SystemVerilog réel ? Réponse : Je distingue clairement largeur de mot <code>`[7:0]`</code> et profondeur <code>`[0:255]`</code>. <code>reg [7:0] mem [0:255];</code>	EN - How would you use reg array memory in a real SystemVerilog design? Answer: I clearly distinguish word width <code>`[7:0]`</code> from depth <code>`[0:255]`</code>. <code>reg [7:0] mem [0:255];</code>
--	--

263. FR - Comment diagnostiqueriez-vous un problème lié à mémoire reg array ? Réponse : Je vérifie profondeur, largeur, adressage, latence de lecture et comportement read-during-write. Je compare aussi le style RTL aux templates d'inférence supportés par la technologie cible. Pour mémoire reg array, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving reg array memory? Answer: I verify depth, width, addressing, read latency, and read-during-write behavior. I also compare RTL style with memory inference templates supported by the target technology. For reg array memory, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
---	---

264. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant mémoire reg array ? Réponse : Je décris d'abord le comportement mémoire voulu, puis je vérifie après synthèse qu'une vraie ressource RAM/ROM a été inférée plutôt qu'un grand réseau de registres. Pour mémoire reg array, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding reg array memory? Answer: I first describe the intended memory behavior, then verify after synthesis that a real RAM/ROM resource was inferred rather than a large register network. For reg array memory, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	--

265. FR - Expliquez RAM synchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Une RAM synchrone enregistre typiquement lecture et/ou écriture au front d'horloge. La latence de lecture fait partie de l'interface.	EN - Explain synchronous RAM. What hardware structure or idea is behind it? Answer: A synchronous RAM typically registers reads and/or writes on the clock edge. Read latency is part of the interface.
--	---

266. FR - Comment utiliseriez-vous RAM synchrone dans un design SystemVerilog réel ? Réponse : Je code le pattern recommandé pour la mémoire cible et vérifie le rapport de synthèse. <pre>always @(posedge clk) begin if (we) mem[addr] <= wdata; rdata <= mem[addr]; end</pre>	EN - How would you use synchronous RAM in a real SystemVerilog design? Answer: I code the recommended pattern for the target memory and verify the synthesis report. <pre>always @(posedge clk) begin if (we) mem[addr] <= wdata; rdata <= mem[addr]; end</pre>
267. FR - Comment diagnostiqueriez-vous un problème lié à RAM synchrone ? Réponse : Je vérifie profondeur, largeur, adressage, latence de lecture et comportement read-during-write. Je compare aussi le style RTL aux templates d'inférence supportés par la technologie cible. Pour RAM synchrone, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving synchronous RAM? Answer: I verify depth, width, addressing, read latency, and read-during-write behavior. I also compare RTL style with memory inference templates supported by the target technology. For synchronous RAM, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
268. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant RAM synchrone ? Réponse : Je décris d'abord le comportement mémoire voulu, puis je vérifie après synthèse qu'une vraie ressource RAM/ROM a été inférée plutôt qu'un grand réseau de registres. Pour RAM synchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding synchronous RAM? Answer: I first describe the intended memory behavior, then verify after synthesis that a real RAM/ROM resource was inferred rather than a large register network. For synchronous RAM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
269. FR - Expliquez ROM. Quelle structure ou idée hardware se cache derrière ? Réponse : Une table constante indexée par adresse peut être inférée en ROM. Son initialisation peut venir d'un `case`, d'une initialisation supportée ou d'un fichier selon le flow.	EN - Explain ROM. What hardware structure or idea is behind it? Answer: A constant table indexed by address can be inferred as ROM. Initialization may come from a `case`, supported initialization, or a file depending on the flow.
270. FR - Comment utiliseriez-vous ROM dans un design SystemVerilog réel ? Réponse : Je traite la méthode d'initialisation comme une propriété du flow de synthèse, pas comme une garantie universelle du langage.	EN - How would you use ROM in a real SystemVerilog design? Answer: I treat the initialization method as a property of the synthesis flow, not as a universal language guarantee.
271. FR - Comment diagnostiqueriez-vous un problème lié à ROM ? Réponse : Je vérifie profondeur, largeur, adressage, latence de lecture et comportement read-during-write. Je compare aussi le style RTL aux templates d'inférence supportés par la technologie cible. Pour ROM, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving ROM? Answer: I verify depth, width, addressing, read latency, and read-during-write behavior. I also compare RTL style with memory inference templates supported by the target technology. For ROM, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
272. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant ROM ? Réponse : Je décris d'abord le comportement mémoire voulu, puis je vérifie après synthèse qu'une vraie ressource RAM/ROM a été inférée plutôt qu'un grand réseau de registres. Pour ROM, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding ROM? Answer: I first describe the intended memory behavior, then verify after synthesis that a real RAM/ROM resource was inferred rather than a large register network. For ROM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
273. FR - Expliquez FIFO synchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Une FIFO utilise mémoire, write/read pointers et flags full/empty. En domaine unique, les pointeurs peuvent être comparés directement avec une convention de wrap adaptée.	EN - Explain synchronous FIFO. What hardware structure or idea is behind it? Answer: A FIFO uses memory, write/read pointers, and full/empty flags. In one clock domain, pointers can be compared directly with an appropriate wrap convention.
274. FR - Comment utiliseriez-vous FIFO synchrone dans un design SystemVerilog réel ? Réponse : Je vérifie soigneusement les conditions limites: write while full, read while empty, et transitions de flags.	EN - How would you use synchronous FIFO in a real SystemVerilog design? Answer: I carefully verify boundary conditions: write while full, read while empty, and flag transitions.

275. FR - Comment diagnostiqueriez-vous un problème lié à FIFO synchrone ? Réponse : Je vérifie profondeur, largeur, adressage, latence de lecture et comportement read-during-write. Je compare aussi le style RTL aux templates d'inférence supportés par la technologie cible. Pour FIFO synchrone, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving synchronous FIFO? Answer: I verify depth, width, addressing, read latency, and read-during-write behavior. I also compare RTL style with memory inference templates supported by the target technology. For synchronous FIFO, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
276. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant FIFO synchrone ? Réponse : Je décris d'abord le comportement mémoire voulu, puis je vérifie après synthèse qu'une vraie ressource RAM/ROM a été inférée plutôt qu'un grand réseau de registres. Pour FIFO synchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding synchronous FIFO? Answer: I first describe the intended memory behavior, then verify after synthesis that a real RAM/ROM resource was inferred rather than a large register network. For synchronous FIFO, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
277. FR - Expliquez FIFO asynchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Une FIFO asynchrone traverse deux clock domains. Elle utilise typiquement des pointeurs Gray synchronisés entre domaines plutôt que le bus de données lui-même.	EN - Explain asynchronous FIFO. What hardware structure or idea is behind it? Answer: An asynchronous FIFO crosses two clock domains. It typically uses Gray-coded pointers synchronized between domains rather than synchronizing the data bus itself.
278. FR - Comment utiliseriez-vous FIFO asynchrone dans un design SystemVerilog réel ? Réponse : Je considère l'async FIFO comme un bloc CDC spécialisé et je vérifie Gray conversion, synchronizers et calcul full/empty.	EN - How would you use asynchronous FIFO in a real SystemVerilog design? Answer: I treat an asynchronous FIFO as a specialized CDC block and verify Gray conversion, synchronizers, and full/empty calculation.
279. FR - Comment diagnostiqueriez-vous un problème lié à FIFO asynchrone ? Réponse : Je vérifie profondeur, largeur, adressage, latence de lecture et comportement read-during-write. Je compare aussi le style RTL aux templates d'inférence supportés par la technologie cible. Pour FIFO asynchrone, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving asynchronous FIFO? Answer: I verify depth, width, addressing, read latency, and read-during-write behavior. I also compare RTL style with memory inference templates supported by the target technology. For asynchronous FIFO, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
280. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant FIFO asynchrone ? Réponse : Je décris d'abord le comportement mémoire voulu, puis je vérifie après synthèse qu'une vraie ressource RAM/ROM a été inférée plutôt qu'un grand réseau de registres. Pour FIFO asynchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding asynchronous FIFO? Answer: I first describe the intended memory behavior, then verify after synthesis that a real RAM/ROM resource was inferred rather than a large register network. For asynchronous FIFO, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

15. Arithmétique, datapaths, largeur et overflow / Arithmetic, Datapaths, Width, and Overflow

281. FR - Expliquez addition et carry. Quelle structure ou idée hardware se cache derrière ? Réponse : Additionner deux nombres N bits peut nécessiter N+1 bits pour conserver le carry non signé.	EN - Explain addition and carry. What hardware structure or idea is behind it? Answer: Adding two N-bit unsigned numbers may require N+1 bits to preserve carry.
282. FR - Comment utiliseriez-vous addition et carry dans un design SystemVerilog réel ? Réponse : Je zero-extend les opérandes avant l'addition si le carry est fonctionnellement important. wire [8:0] sum = {1'b0,a} + {1'b0,b};	EN - How would you use addition and carry in a real SystemVerilog design? Answer: I zero-extend operands before the addition when carry is functionally important. wire [8:0] sum = {1'b0,a} + {1'b0,b};
283. FR - Comment diagnostiqueriez-vous un problème lié à addition et carry ? Réponse : Je vérifie width propagation, signedness, troncature, overflow et latence. Je regarde aussi si l'opération combinatoire est trop profonde pour la fréquence visée. Pour addition et carry, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving addition and carry? Answer: I verify width propagation, signedness, truncation, overflow, and latency. I also check whether the combinational operation is too deep for the target frequency. For addition and carry, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

284. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant addition et carry ? Réponse : Je dimensionne chaque résultat consciemment et pipeline les datapaths longs. Je ne compte pas sur les règles implicites de taille pour protéger un résultat numérique. Pour addition et carry, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding addition and carry? Answer: I consciously size every result and pipeline long datapaths. I do not rely on implicit sizing rules to protect numerical results. For addition and carry, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
285. FR - Expliquez soustraction et underflow. Quelle structure ou idée hardware se cache derrière ? Réponse : En unsigned, `a-b` wrappe modulo la largeur si `b>a`. Si un borrow ou résultat signé est requis, il faut élargir/représenter explicitement.	EN - Explain subtraction and underflow. What hardware structure or idea is behind it? Answer: In unsigned arithmetic, `a-b` wraps modulo the width if `b>a`. If borrow or a signed result is required, representation must be widened/explicit.
286. FR - Comment utiliseriez-vous soustraction et underflow dans un design SystemVerilog réel ? Réponse : Je définis la sémantique attendue avant de coder: saturation, wrap ou signal d'erreur.	EN - How would you use subtraction and underflow in a real SystemVerilog design? Answer: I define the intended arithmetic semantics before coding: saturation, wraparound, or an error/borrow indication.
287. FR - Comment diagnostiqueriez-vous un problème lié à soustraction et underflow ? Réponse : Je vérifie width propagation, signedness, troncature, overflow et latence. Je regarde aussi si l'opération combinatoire est trop profonde pour la fréquence visée. Pour soustraction et underflow, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving subtraction and underflow? Answer: I verify width propagation, signedness, truncation, overflow, and latency. I also check whether the combinational operation is too deep for the target frequency. For subtraction and underflow, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
288. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant soustraction et underflow ? Réponse : Je dimensionne chaque résultat consciemment et pipeline les datapaths longs. Je ne compte pas sur les règles implicites de taille pour protéger un résultat numérique. Pour soustraction et underflow, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding subtraction and underflow? Answer: I consciously size every result and pipeline long datapaths. I do not rely on implicit sizing rules to protect numerical results. For subtraction and underflow, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
289. FR - Expliquez multiplication. Quelle structure ou idée hardware se cache derrière ? Réponse : Une multiplication $N \times M$ peut produire $N+M$ bits. Sur FPGA, elle peut utiliser DSP blocks ou logique selon largeur, options et synthèse.	EN - Explain multiplication. What hardware structure or idea is behind it? Answer: An $N \times M$ multiplication can produce $N+M$ bits. On an FPGA, it may use DSP blocks or logic depending on width, options, and synthesis.
290. FR - Comment utiliseriez-vous multiplication dans un design SystemVerilog réel ? Réponse : Je vérifie la largeur complète du produit et pipeline si le timing l'exige.	EN - How would you use multiplication in a real SystemVerilog design? Answer: I verify full product width and pipeline if timing requires it.
291. FR - Comment diagnostiqueriez-vous un problème lié à multiplication ? Réponse : Je vérifie width propagation, signedness, troncature, overflow et latence. Je regarde aussi si l'opération combinatoire est trop profonde pour la fréquence visée. Pour multiplication, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving multiplication? Answer: I verify width propagation, signedness, truncation, overflow, and latency. I also check whether the combinational operation is too deep for the target frequency. For multiplication, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
292. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant multiplication ? Réponse : Je dimensionne chaque résultat consciemment et pipeline les datapaths longs. Je ne compte pas sur les règles implicites de taille pour protéger un résultat numérique. Pour multiplication, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding multiplication? Answer: I consciously size every result and pipeline long datapaths. I do not rely on implicit sizing rules to protect numerical results. For multiplication, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

293. FR - Expliquez shifts. Quelle structure ou idée hardware se cache derrière ? Réponse : Les shifts constants se synthétisent souvent en câblage; un shift variable peut inférer un barrel shifter plus coûteux. Le comportement de right shift dépend aussi du signedness et de l'opérateur.	EN - Explain shifts. What hardware structure or idea is behind it? Answer: Constant shifts often synthesize as wiring; a variable shift can infer a more expensive barrel shifter. Right-shift behavior also depends on signedness and operator choice.
294. FR - Comment utiliseriez-vous shifts dans un design SystemVerilog réel ? Réponse : Je choisis `>>` ou `>>>>` consciemment selon shift logique ou arithmétique sur données signed.	EN - How would you use shifts in a real SystemVerilog design? Answer: I deliberately choose `>>` or `>>>>` according to whether I need logical or arithmetic right shift on signed data.
295. FR - Comment diagnostiqueriez-vous un problème lié à shifts ? Réponse : Je vérifie width propagation, signedness, troncature, overflow et latence. Je regarde aussi si l'opération combinatoire est trop profonde pour la fréquence visée. Pour shifts, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving shifts? Answer: I verify width propagation, signedness, truncation, overflow, and latency. I also check whether the combinational operation is too deep for the target frequency. For shifts, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
296. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant shifts ? Réponse : Je dimensionne chaque résultat consciemment et pipeline les datapaths longs. Je ne compte pas sur les règles implicites de taille pour protéger un résultat numérique. Pour shifts, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding shifts? Answer: I consciously size every result and pipeline long datapaths. I do not rely on implicit sizing rules to protect numerical results. For shifts, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
297. FR - Expliquez pipeline datapath. Quelle structure ou idée hardware se cache derrière ? Réponse : Ajouter des registres entre opérations réduit la profondeur combinatoire par cycle au prix de latence supplémentaire.	EN - Explain datapath pipelining. What hardware structure or idea is behind it? Answer: Adding registers between operations reduces combinational depth per cycle at the cost of extra latency.
298. FR - Comment utiliseriez-vous pipeline datapath dans un design SystemVerilog réel ? Réponse : Pour un calcul multiply-add à haute fréquence, je peux enregistrer le produit puis l'additionner au cycle suivant.	EN - How would you use datapath pipelining in a real SystemVerilog design? Answer: For a high-frequency multiply-add, I can register the product and add it on the following cycle.
299. FR - Comment diagnostiqueriez-vous un problème lié à pipeline datapath ? Réponse : Je vérifie width propagation, signedness, troncature, overflow et latence. Je regarde aussi si l'opération combinatoire est trop profonde pour la fréquence visée. Pour pipeline datapath, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving datapath pipelining? Answer: I verify width propagation, signedness, truncation, overflow, and latency. I also check whether the combinational operation is too deep for the target frequency. For datapath pipelining, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
300. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant pipeline datapath ? Réponse : Je dimensionne chaque résultat consciemment et pipeline les datapaths longs. Je ne compte pas sur les règles implicites de taille pour protéger un résultat numérique. Pour pipeline datapath, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding datapath pipelining? Answer: I consciously size every result and pipeline long datapaths. I do not rely on implicit sizing rules to protect numerical results. For datapath pipelining, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

16. CDC, métastabilité et synchronisation / CDC, Metastability, and Synchronization

301. FR - Expliquez métastabilité. Quelle structure ou idée hardware se cache derrière ? Réponse : Lorsqu'un signal asynchrone change près du front d'échantillonnage, un flip-flop peut entrer temporairement dans un état analogique métastable. On ne l'élimine pas; on réduit sa probabilité de propagation.	EN - Explain metastability. What hardware structure or idea is behind it? Answer: When an asynchronous signal changes near a sampling edge, a flip-flop can temporarily enter an analog metastable state. It is not eliminated; its probability of propagation is reduced.
302. FR - Comment utiliseriez-vous métastabilité dans un design SystemVerilog réel ? Réponse : Je traite toute entrée asynchrone comme un CDC et évite de la distribuer directement à la logique.	EN - How would you use metastability in a real SystemVerilog design? Answer: I treat every asynchronous input as a CDC and avoid distributing it directly into logic.

303. FR - Comment diagnostiqueriez-vous un problème lié à métastabilité ? Réponse : Je demande toujours: source clock domain, destination domain, largeur du signal et protocole. Ensuite je choisis synchronizer, handshake ou FIFO plutôt qu'une solution générique. Pour métastabilité, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving metastability? Answer: I always ask: source clock domain, destination domain, signal width, and protocol. Then I choose a synchronizer, handshake, or FIFO rather than a generic solution. For metastability, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
304. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant métastabilité ? Réponse : Un synchronizer deux flip-flops convient à un niveau mono-bit qui reste stable assez longtemps; il ne rend pas cohérent un bus multi-bit. Les CDC doivent être conçus et vérifiés explicitement. Pour métastabilité, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding metastability? Answer: A two-flop synchronizer suits a single-bit level that stays stable long enough; it does not make a multi-bit bus coherent. CDCs must be explicitly designed and verified. For metastability, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
305. FR - Expliquez synchronizer deux flip-flops. Quelle structure ou idée hardware se cache derrière ? Réponse : Deux flip-flops en série dans le domaine destination donnent au premier temps pour résoudre sa métastabilité avant utilisation du second.	EN - Explain two-flop synchronizer. What hardware structure or idea is behind it? Answer: Two flip-flops in series in the destination domain give the first time to resolve metastability before the second is used.
306. FR - Comment utiliseriez-vous synchronizer deux flip-flops dans un design SystemVerilog réel ? Réponse : Je l'emploie pour un niveau mono-bit, puis effectue edge detection après le deuxième étage. <pre>always @(posedge clk) begin sync1 <= async_in; sync2 <= sync1; end</pre>	EN - How would you use two-flop synchronizer in a real SystemVerilog design? Answer: I use it for a single-bit level, then perform edge detection after the second stage. <pre>always @(posedge clk) begin sync1 <= async_in; sync2 <= sync1; end</pre>
307. FR - Comment diagnostiqueriez-vous un problème lié à synchronizer deux flip-flops ? Réponse : Je demande toujours: source clock domain, destination domain, largeur du signal et protocole. Ensuite je choisis synchronizer, handshake ou FIFO plutôt qu'une solution générique. Pour synchronizer deux flip-flops, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving two-flop synchronizer? Answer: I always ask: source clock domain, destination domain, signal width, and protocol. Then I choose a synchronizer, handshake, or FIFO rather than a generic solution. For two-flop synchronizer, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
308. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant synchronizer deux flip-flops ? Réponse : Un synchronizer deux flip-flops convient à un niveau mono-bit qui reste stable assez longtemps; il ne rend pas cohérent un bus multi-bit. Les CDC doivent être conçus et vérifiés explicitement. Pour synchronizer deux flip-flops, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding two-flop synchronizer? Answer: A two-flop synchronizer suits a single-bit level that stays stable long enough; it does not make a multi-bit bus coherent. CDCs must be explicitly designed and verified. For two-flop synchronizer, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
309. FR - Expliquez pulse crossing. Quelle structure ou idée hardware se cache derrière ? Réponse : Une impulsion plus courte qu'une période du clock destination peut être manquée par un synchronizer de niveau. Il faut l'étirer, utiliser toggle synchronization ou handshake.	EN - Explain pulse crossing. What hardware structure or idea is behind it? Answer: A pulse shorter than a destination clock period can be missed by a level synchronizer. It must be stretched, converted to a toggle, or transferred with a handshake.
310. FR - Comment utiliseriez-vous pulse crossing dans un design SystemVerilog réel ? Réponse : Je choisis la technique selon fréquence d'événement et tolérance de latence.	EN - How would you use pulse crossing in a real SystemVerilog design? Answer: I choose the transfer technique according to event frequency and acceptable latency.
311. FR - Comment diagnostiqueriez-vous un problème lié à pulse crossing ? Réponse : Je demande toujours: source clock domain, destination domain, largeur du signal et protocole. Ensuite je choisis synchronizer, handshake ou FIFO plutôt qu'une solution générique. Pour pulse crossing, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving pulse crossing? Answer: I always ask: source clock domain, destination domain, signal width, and protocol. Then I choose a synchronizer, handshake, or FIFO rather than a generic solution. For pulse crossing, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

312. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant pulse crossing ? Réponse : Un synchronizer deux flip-flops convient à un niveau mono-bit qui reste stable assez longtemps; il ne rend pas cohérent un bus multi-bit. Les CDC doivent être conçus et vérifiés explicitement. Pour pulse crossing, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding pulse crossing? Answer: A two-flop synchronizer suits a single-bit level that stays stable long enough; it does not make a multi-bit bus coherent. CDCs must be explicitly designed and verified. For pulse crossing, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
313. FR - Expliquez bus multi-bit CDC. Quelle structure ou idée hardware se cache derrière ? Réponse : Synchroniser chaque bit d'un bus indépendamment peut produire un mot incohérent car les bits peuvent être capturés à des cycles différents.	EN - Explain multi-bit CDC bus. What hardware structure or idea is behind it? Answer: Synchronizing each bit of a bus independently can produce an incoherent word because bits may be captured on different cycles.
314. FR - Comment utiliseriez-vous bus multi-bit CDC dans un design SystemVerilog réel ? Réponse : Je transfère le bus avec handshake stable-data, async FIFO ou protocole adapté.	EN - How would you use multi-bit CDC bus in a real SystemVerilog design? Answer: I transfer the bus using stable-data handshake, an asynchronous FIFO, or another protocol designed for coherent multi-bit transfer.
315. FR - Comment diagnostiqueriez-vous un problème lié à bus multi-bit CDC ? Réponse : Je demande toujours: source clock domain, destination domain, largeur du signal et protocole. Ensuite je choisis synchronizer, handshake ou FIFO plutôt qu'une solution générique. Pour bus multi-bit CDC, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving multi-bit CDC bus? Answer: I always ask: source clock domain, destination domain, signal width, and protocol. Then I choose a synchronizer, handshake, or FIFO rather than a generic solution. For multi-bit CDC bus, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
316. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant bus multi-bit CDC ? Réponse : Un synchronizer deux flip-flops convient à un niveau mono-bit qui reste stable assez longtemps; il ne rend pas cohérent un bus multi-bit. Les CDC doivent être conçus et vérifiés explicitement. Pour bus multi-bit CDC, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding multi-bit CDC bus? Answer: A two-flop synchronizer suits a single-bit level that stays stable long enough; it does not make a multi-bit bus coherent. CDCs must be explicitly designed and verified. For multi-bit CDC bus, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
317. FR - Expliquez Gray code pour pointeurs CDC. Quelle structure ou idée hardware se cache derrière ? Réponse : Le Gray code ne change idéalement qu'un bit entre valeurs successives, ce qui réduit l'ambiguïté lors de la synchronisation de pointeurs de FIFO asynchrone.	EN - Explain Gray code for CDC pointers. What hardware structure or idea is behind it? Answer: Gray code ideally changes only one bit between successive values, reducing ambiguity when asynchronous FIFO pointers are synchronized.
318. FR - Comment utiliseriez-vous Gray code pour pointeurs CDC dans un design SystemVerilog réel ? Réponse : Je synchronise les pointeurs Gray, puis effectue les comparaisons full/empty selon l'algorithme de FIFO.	EN - How would you use Gray code for CDC pointers in a real SystemVerilog design? Answer: I synchronize Gray-coded pointers and then calculate full/empty according to the FIFO algorithm.
319. FR - Comment diagnostiqueriez-vous un problème lié à Gray code pour pointeurs CDC ? Réponse : Je demande toujours: source clock domain, destination domain, largeur du signal et protocole. Ensuite je choisis synchronizer, handshake ou FIFO plutôt qu'une solution générique. Pour Gray code pour pointeurs CDC, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving Gray code for CDC pointers? Answer: I always ask: source clock domain, destination domain, signal width, and protocol. Then I choose a synchronizer, handshake, or FIFO rather than a generic solution. For Gray code for CDC pointers, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
320. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant Gray code pour pointeurs CDC ? Réponse : Un synchronizer deux flip-flops convient à un niveau mono-bit qui reste stable assez longtemps; il ne rend pas cohérent un bus multi-bit. Les CDC doivent être conçus et vérifiés explicitement. Pour Gray code pour pointeurs CDC, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding Gray code for CDC pointers? Answer: A two-flop synchronizer suits a single-bit level that stays stable long enough; it does not make a multi-bit bus coherent. CDCs must be explicitly designed and verified. For Gray code for CDC pointers, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

17. Entrées externes, debounce, pulses et événements / External Inputs, Debounce, Pulses, and Events

321. FR - Expliquez entrée asynchrone externe. Quelle structure ou idée hardware se cache derrière ? Réponse : Une entrée venant d'un pin externe n'est généralement pas alignée sur le clock interne. Elle doit être synchronisée avant utilisation dans le domaine destination.	EN - Explain external asynchronous input. What hardware structure or idea is behind it? Answer: An input from an external pin is generally not aligned with the internal clock. It must be synchronized before use in the destination domain.
322. FR - Comment utiliseriez-vous entrée asynchrone externe dans un design SystemVerilog réel ? Réponse : Je mets un synchronizer puis traite le signal synchronisé, jamais directement le pin dans une FSM critique.	EN - How would you use external asynchronous input in a real SystemVerilog design? Answer: I place a synchronizer first and process only the synchronized signal, never the raw pin directly in a critical FSM.
323. FR - Comment diagnostiqueriez-vous un problème lié à entrée asynchrone externe ? Réponse : Je vérifie d'abord la nature électrique et temporelle de l'entrée: asynchrone, bruitée, mécanique ou déjà synchronisée. Ensuite seulement je décide synchronizer, debounce, filter ou edge detector. Pour entrée asynchrone externe, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving external asynchronous input? Answer: I first verify the electrical and timing nature of the input: asynchronous, noisy, mechanical, or already synchronized. Only then do I choose a synchronizer, debounce, filter, or edge detector. For external asynchronous input, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
324. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant entrée asynchrone externe ? Réponse : Je sépare conditionnement matériel, synchronisation CDC et logique fonctionnelle. Un bouton, un capteur et une IRQ externe n'ont pas forcément la même stratégie. Pour entrée asynchrone externe, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding external asynchronous input? Answer: I separate electrical conditioning, CDC synchronization, and functional logic. A button, sensor, and external IRQ do not necessarily use the same strategy. For external asynchronous input, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
325. FR - Expliquez debounce de bouton. Quelle structure ou idée hardware se cache derrière ? Réponse : Un contact mécanique peut rebondir plusieurs fois pendant quelques millisecondes. Un debounce numérique exige que le niveau reste stable assez longtemps avant validation.	EN - Explain button debounce. What hardware structure or idea is behind it? Answer: A mechanical contact can bounce repeatedly for several milliseconds. Digital debounce requires the level to remain stable long enough before acceptance.
326. FR - Comment utiliseriez-vous debounce de bouton dans un design SystemVerilog réel ? Réponse : Je synchronise d'abord le bouton, puis utilise un compteur de stabilité avant de changer l'état debounced.	EN - How would you use button debounce in a real SystemVerilog design? Answer: I synchronize the button first, then use a stability counter before changing the debounced state.
327. FR - Comment diagnostiqueriez-vous un problème lié à debounce de bouton ? Réponse : Je vérifie d'abord la nature électrique et temporelle de l'entrée: asynchrone, bruitée, mécanique ou déjà synchronisée. Ensuite seulement je décide synchronizer, debounce, filter ou edge detector. Pour debounce de bouton, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving button debounce? Answer: I first verify the electrical and timing nature of the input: asynchronous, noisy, mechanical, or already synchronized. Only then do I choose a synchronizer, debounce, filter, or edge detector. For button debounce, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
328. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant debounce de bouton ? Réponse : Je sépare conditionnement matériel, synchronisation CDC et logique fonctionnelle. Un bouton, un capteur et une IRQ externe n'ont pas forcément la même stratégie. Pour debounce de bouton, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding button debounce? Answer: I separate electrical conditioning, CDC synchronization, and functional logic. A button, sensor, and external IRQ do not necessarily use the same strategy. For button debounce, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
329. FR - Expliquez glitch filter. Quelle structure ou idée hardware se cache derrière ? Réponse : Un filtre temporel rejette les impulsions plus courtes qu'un seuil. Contrairement au simple synchronizer, il traite une propriété de durée, pas seulement la métastabilité.	EN - Explain glitch filter. What hardware structure or idea is behind it? Answer: A temporal filter rejects pulses shorter than a threshold. Unlike a simple synchronizer, it handles a duration property, not just metastability.

330. FR - Comment utiliseriez-vous glitch filter dans un design SystemVerilog réel ? Réponse : Je l'utilise lorsque la spécification autorise d'ignorer les glitches courts.	EN - How would you use glitch filter in a real SystemVerilog design? Answer: I use it when the specification allows short glitches to be ignored.
331. FR - Comment diagnostiqueriez-vous un problème lié à glitch filter ? Réponse : Je vérifie d'abord la nature électrique et temporelle de l'entrée: asynchrone, bruitée, mécanique ou déjà synchronisée. Ensuite seulement je décide synchronizer, debounce, filter ou edge detector. Pour glitch filter, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving glitch filter? Answer: I first verify the electrical and timing nature of the input: asynchronous, noisy, mechanical, or already synchronized. Only then do I choose a synchronizer, debounce, filter, or edge detector. For glitch filter, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
332. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant glitch filter ? Réponse : Je sépare conditionnement matériel, synchronisation CDC et logique fonctionnelle. Un bouton, un capteur et une IRQ externe n'ont pas forcément la même stratégie. Pour glitch filter, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding glitch filter? Answer: I separate electrical conditioning, CDC synchronization, and functional logic. A button, sensor, and external IRQ do not necessarily use the same strategy. For glitch filter, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
333. FR - Expliquez one-shot pulse. Quelle structure ou idée hardware se cache derrière ? Réponse : Un one-shot génère une impulsion d'un cycle lorsqu'un événement est détecté, utile pour convertir un niveau ou changement d'état en enable.	EN - Explain one-shot pulse. What hardware structure or idea is behind it? Answer: A one-shot generates a one-cycle pulse when an event is detected, useful for converting a level or state change into an enable.
334. FR - Comment utiliseriez-vous one-shot pulse dans un design SystemVerilog réel ? Réponse : Après edge detection d'un signal synchronisé, j'utilise le pulse pour incrémenter un compteur exactement une fois.	EN - How would you use one-shot pulse in a real SystemVerilog design? Answer: After edge detection of a synchronized signal, I use the one-cycle pulse to trigger an action exactly once.
335. FR - Comment diagnostiqueriez-vous un problème lié à one-shot pulse ? Réponse : Je vérifie d'abord la nature électrique et temporelle de l'entrée: asynchrone, bruitée, mécanique ou déjà synchronisée. Ensuite seulement je décide synchronizer, debounce, filter ou edge detector. Pour one-shot pulse, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving one-shot pulse? Answer: I first verify the electrical and timing nature of the input: asynchronous, noisy, mechanical, or already synchronized. Only then do I choose a synchronizer, debounce, filter, or edge detector. For one-shot pulse, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
336. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant one-shot pulse ? Réponse : Je sépare conditionnement matériel, synchronisation CDC et logique fonctionnelle. Un bouton, un capteur et une IRQ externe n'ont pas forcément la même stratégie. Pour one-shot pulse, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding one-shot pulse? Answer: I separate electrical conditioning, CDC synchronization, and functional logic. A button, sensor, and external IRQ do not necessarily use the same strategy. For one-shot pulse, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
337. FR - Expliquez signal vers IRQ processeur. Quelle structure ou idée hardware se cache derrière ? Réponse : Pour un système FPGA+CPU, une logique RTL peut convertir une condition matérielle en ligne IRQ, éventuellement avec status, mask et clear. L'interrupt est alors une convention d'interface avec le processeur.	EN - Explain signal to processor IRQ. What hardware structure or idea is behind it? Answer: In an FPGA+CPU system, RTL can convert a hardware condition into an IRQ line, optionally with status, mask, and clear. The interrupt is then an interface convention with the processor.
338. FR - Comment utiliseriez-vous signal vers IRQ processeur dans un design SystemVerilog réel ? Réponse : Je définis si l'IRQ est level-sensitive ou pulse, comment elle est acquittée et comment éviter de perdre un événement.	EN - How would you use signal to processor IRQ in a real SystemVerilog design? Answer: I define whether the IRQ is level-sensitive or pulsed, how it is acknowledged/cleared, and how events are prevented from being lost.

339. FR - Comment diagnostiqueriez-vous un problème lié à signal vers IRQ processeur ? Réponse : Je vérifie d'abord la nature électrique et temporelle de l'entrée: asynchrone, bruitée, mécanique ou déjà synchronisée. Ensuite seulement je décide synchronizer, debounce, filter ou edge detector. Pour signal vers IRQ processeur, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving signal to processor IRQ? Answer: I first verify the electrical and timing nature of the input: asynchronous, noisy, mechanical, or already synchronized. Only then do I choose a synchronizer, debounce, filter, or edge detector. For signal to processor IRQ, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
340. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant signal vers IRQ processeur ? Réponse : Je sépare conditionnement matériel, synchronisation CDC et logique fonctionnelle. Un bouton, un capteur et une IRQ externe n'ont pas forcément la même stratégie. Pour signal vers IRQ processeur, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding signal to processor IRQ? Answer: I separate electrical conditioning, CDC synchronization, and functional logic. A button, sensor, and external IRQ do not necessarily use the same strategy. For signal to processor IRQ, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

18. UART RTL / UART RTL

341. FR - Expliquez format de trame UART. Quelle structure ou idée hardware se cache derrière ? Réponse : Une UART classique transmet idle=1, start=0, bits de données LSB first, éventuellement parity, puis un ou plusieurs stop bits à 1.	EN - Explain UART frame format. What hardware structure or idea is behind it? Answer: A classic UART transmits idle=1, start=0, data bits LSB first, optional parity, then one or more stop bits at 1.
342. FR - Comment utiliseriez-vous format de trame UART dans un design SystemVerilog réel ? Réponse : Je définis explicitement 8-N-1 ou le format requis et dimensionne TX/RX en conséquence.	EN - How would you use UART frame format in a real SystemVerilog design? Answer: I explicitly define 8-N-1 or the required frame format and size/control the TX and RX logic accordingly.
343. FR - Comment diagnostiqueriez-vous un problème lié à format de trame UART ? Réponse : Je vérifie baud timing, format de trame, idle level, sampling point, start/stop bits et framing errors. Je compare ensuite la waveform RTL au protocole attendu bit par bit. Pour format de trame UART, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART frame format? Answer: I verify baud timing, frame format, idle level, sampling point, start/stop bits, and framing errors. I then compare the RTL waveform with the expected protocol bit by bit. For UART frame format, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
344. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant format de trame UART ? Réponse : Je sépare baud tick, TX FSM, RX synchronization/sampling et buffers. Le RX externe passe d'abord par une stratégie CDC adaptée. Pour format de trame UART, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART frame format? Answer: I separate baud tick, TX FSM, RX synchronization/sampling, and buffering. The external RX first passes through an appropriate CDC strategy. For UART frame format, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
345. FR - Expliquez baud tick. Quelle structure ou idée hardware se cache derrière ? Réponse : Le baud tick est un enable périodique dérivé du clock système. Pour RX oversampling, la fréquence peut être 8x ou 16x le baud.	EN - Explain baud tick. What hardware structure or idea is behind it? Answer: The baud tick is a periodic enable derived from the system clock. For RX oversampling, it may run at 8x or 16x the baud rate.
346. FR - Comment utiliseriez-vous baud tick dans un design SystemVerilog réel ? Réponse : Je calcule le diviseur depuis `CLK_HZ` et `BAUD`, puis quantifie l'erreur de fréquence.	EN - How would you use baud tick in a real SystemVerilog design? Answer: I calculate the divider from `CLK_HZ` and `BAUD`, then quantify the resulting frequency error.
347. FR - Comment diagnostiqueriez-vous un problème lié à baud tick ? Réponse : Je vérifie baud timing, format de trame, idle level, sampling point, start/stop bits et framing errors. Je compare ensuite la waveform RTL au protocole attendu bit par bit. Pour baud tick, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving baud tick? Answer: I verify baud timing, frame format, idle level, sampling point, start/stop bits, and framing errors. I then compare the RTL waveform with the expected protocol bit by bit. For baud tick, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

348. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant baud tick ? Réponse : Je sépare baud tick, TX FSM, RX synchronization/sampling et buffers. Le RX externe passe d'abord par une stratégie CDC adaptée. Pour baud tick, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding baud tick? Answer: I separate baud tick, TX FSM, RX synchronization/sampling, and buffering. The external RX first passes through an appropriate CDC strategy. For baud tick, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
349. FR - Expliquez UART TX FSM. Quelle structure ou idée hardware se cache derrière ? Réponse : Le TX est naturellement une FSM: IDLE, START, DATA, STOP. Un compteur de bits et un shift register fournissent les données série.	EN - Explain UART TX FSM. What hardware structure or idea is behind it? Answer: The TX naturally maps to an FSM: IDLE, START, DATA, STOP. A bit counter and shift register provide serial data.
350. FR - Comment utiliseriez-vous UART TX FSM dans un design SystemVerilog réel ? Réponse : Je charge l'octet lors d'un handshake `start`, puis garde `busy` jusqu'à la fin du stop bit.	EN - How would you use UART TX FSM in a real SystemVerilog design? Answer: I load the byte on a `start` handshake and keep `busy` asserted until the stop bit has completed.
351. FR - Comment diagnostiqueriez-vous un problème lié à UART TX FSM ? Réponse : Je vérifie baud timing, format de trame, idle level, sampling point, start/stop bits et framing errors. Je compare ensuite la waveform RTL au protocole attendu bit par bit. Pour UART TX FSM, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART TX FSM? Answer: I verify baud timing, frame format, idle level, sampling point, start/stop bits, and framing errors. I then compare the RTL waveform with the expected protocol bit by bit. For UART TX FSM, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
352. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant UART TX FSM ? Réponse : Je sépare baud tick, TX FSM, RX synchronization/sampling et buffers. Le RX externe passe d'abord par une stratégie CDC adaptée. Pour UART TX FSM, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART TX FSM? Answer: I separate baud tick, TX FSM, RX synchronization/sampling, and buffering. The external RX first passes through an appropriate CDC strategy. For UART TX FSM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
353. FR - Expliquez UART RX sampling. Quelle structure ou idée hardware se cache derrière ? Réponse : Le RX détecte le start bit puis échantillonne près du centre de chaque bit pour maximiser la marge aux erreurs de phase et de baud.	EN - Explain UART RX sampling. What hardware structure or idea is behind it? Answer: The RX detects the start bit then samples near the center of each bit to maximize margin to phase and baud errors.
354. FR - Comment utiliseriez-vous UART RX sampling dans un design SystemVerilog réel ? Réponse : Je synchronise RX, valide le start au milieu du bit et vérifie le stop bit avant d'annoncer `data_valid`.	EN - How would you use UART RX sampling in a real SystemVerilog design? Answer: I synchronize RX, validate the start bit near its center, and check the stop bit before asserting `data_valid`.
355. FR - Comment diagnostiqueriez-vous un problème lié à UART RX sampling ? Réponse : Je vérifie baud timing, format de trame, idle level, sampling point, start/stop bits et framing errors. Je compare ensuite la waveform RTL au protocole attendu bit par bit. Pour UART RX sampling, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART RX sampling? Answer: I verify baud timing, frame format, idle level, sampling point, start/stop bits, and framing errors. I then compare the RTL waveform with the expected protocol bit by bit. For UART RX sampling, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
356. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant UART RX sampling ? Réponse : Je sépare baud tick, TX FSM, RX synchronization/sampling et buffers. Le RX externe passe d'abord par une stratégie CDC adaptée. Pour UART RX sampling, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART RX sampling? Answer: I separate baud tick, TX FSM, RX synchronization/sampling, and buffering. The external RX first passes through an appropriate CDC strategy. For UART RX sampling, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
357. FR - Expliquez UART framing error. Quelle structure ou idée hardware se cache derrière ? Réponse : Un framing error survient typiquement si le stop bit attendu à 1 est reçu à 0, signe de mauvais baud, bruit, mauvais format ou désalignement.	EN - Explain UART framing error. What hardware structure or idea is behind it? Answer: A framing error typically occurs when the expected high stop bit is received low, indicating wrong baud, noise, wrong format, or misalignment.

358. FR - Comment utiliseriez-vous UART framing error dans un design SystemVerilog réel ? Réponse : Je loggue l'erreur et inspecte les timings RX plutôt que de simplement accepter l'octet.	EN - How would you use UART framing error in a real SystemVerilog design? Answer: I record the error and inspect RX timing rather than simply accepting the byte.
359. FR - Comment diagnostiqueriez-vous un problème lié à UART framing error ? Réponse : Je vérifie baud timing, format de trame, idle level, sampling point, start/stop bits et framing errors. Je compare ensuite la waveform RTL au protocole attendu bit par bit. Pour UART framing error, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART framing error? Answer: I verify baud timing, frame format, idle level, sampling point, start/stop bits, and framing errors. I then compare the RTL waveform with the expected protocol bit by bit. For UART framing error, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
360. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant UART framing error ? Réponse : Je sépare baud tick, TX FSM, RX synchronization/sampling et buffers. Le RX externe passe d'abord par une stratégie CDC adaptée. Pour UART framing error, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART framing error? Answer: I separate baud tick, TX FSM, RX synchronization/sampling, and buffering. The external RX first passes through an appropriate CDC strategy. For UART framing error, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

19. SPI RTL / SPI RTL

361. FR - Expliquez signaux SPI. Quelle structure ou idée hardware se cache derrière ? Réponse : SPI utilise typiquement SCLK, MOSI, MISO et CS/SS. Le master génère SCLK et sélectionne l'esclave via CS.	EN - Explain SPI signals. What hardware structure or idea is behind it? Answer: SPI typically uses SCLK, MOSI, MISO, and CS/SS. The master generates SCLK and selects the slave with CS.
362. FR - Comment utiliseriez-vous signaux SPI dans un design SystemVerilog réel ? Réponse : Je définis clairement direction et idle level de chaque signal selon master ou slave.	EN - How would you use SPI signals in a real SystemVerilog design? Answer: I clearly define the direction and idle behavior of each signal according to whether the block is master or slave.
363. FR - Comment diagnostiqueriez-vous un problème lié à signaux SPI ? Réponse : Je vérifie mode CPOL/CPHA, rôle master/slave, ordre des bits, fréquence SCLK et moment exact d'échantillonnage/changement. Beaucoup de pannes SPI sont un demi-cycle de déphasage. Pour signaux SPI, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving SPI signals? Answer: I verify CPOL/CPHA mode, master/slave role, bit order, SCLK frequency, and exact sampling/change edges. Many SPI failures are a half-cycle phase mismatch. For SPI signals, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
364. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant signaux SPI ? Réponse : Je définis le mode SPI comme paramètre d'interface et dessine un timing diagram avant de coder la FSM. Je traite CS comme faisant partie du protocole, pas comme un simple GPIO décoratif. Pour signaux SPI, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding SPI signals? Answer: I define SPI mode as part of the interface and draw a timing diagram before coding the FSM. I treat CS as part of the protocol, not as a decorative GPIO. For SPI signals, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
365. FR - Expliquez CPOL et CPHA. Quelle structure ou idée hardware se cache derrière ? Réponse : CPOL définit le niveau idle de SCLK; CPHA définit la relation entre premier/second edge et sampling/shifting. Les quatre combinaisons donnent les modes 0 à 3.	EN - Explain CPOL and CPHA. What hardware structure or idea is behind it? Answer: CPOL defines the idle level of SCLK; CPHA defines the relationship between first/second edge and sampling/shifting. The four combinations produce modes 0 through 3.
366. FR - Comment utiliseriez-vous CPOL et CPHA dans un design SystemVerilog réel ? Réponse : Je vérifie le datasheet du périphérique et reproduis exactement les edges de capture et de changement.	EN - How would you use CPOL and CPHA in a real SystemVerilog design? Answer: I check the peripheral datasheet and reproduce the required capture and change edges exactly.

367. FR - Comment diagnostiqueriez-vous un problème lié à CPOL et CPHA ? Réponse : Je vérifie mode CPOL/CPHA, rôle master/slave, ordre des bits, fréquence SCLK et moment exact d'échantillonnage/changement. Beaucoup de pannes SPI sont un demi-cycle de déphasage. Pour CPOL et CPHA, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving CPOL and CPHA? Answer: I verify CPOL/CPHA mode, master/slave role, bit order, SCLK frequency, and exact sampling/change edges. Many SPI failures are a half-cycle phase mismatch. For CPOL and CPHA, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
368. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant CPOL et CPHA ? Réponse : Je définis le mode SPI comme paramètre d'interface et dessine un timing diagram avant de coder la FSM. Je traite CS comme faisant partie du protocole, pas comme un simple GPIO décoratif. Pour CPOL et CPHA, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding CPOL and CPHA? Answer: I define SPI mode as part of the interface and draw a timing diagram before coding the FSM. I treat CS as part of the protocol, not as a decorative GPIO. For CPOL and CPHA, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
369. FR - Expliquez SPI master clock divider. Quelle structure ou idée hardware se cache derrière ? Réponse : Le master dérive SCLK du clock système avec un diviseur ou enable. Le duty cycle et les contraintes de fréquence du slave doivent être respectés.	EN - Explain SPI master clock divider. What hardware structure or idea is behind it? Answer: The master derives SCLK from the system clock using a divider or enable. Duty cycle and slave frequency limits must be respected.
370. FR - Comment utiliserez-vous SPI master clock divider dans un design SystemVerilog réel ? Réponse : Je calcule la fréquence réelle obtenue et vérifie setup/hold autour des edges SPI.	EN - How would you use SPI master clock divider in a real SystemVerilog design? Answer: I calculate the actual SCLK frequency and verify setup/hold around the SPI edges.
371. FR - Comment diagnostiqueriez-vous un problème lié à SPI master clock divider ? Réponse : Je vérifie mode CPOL/CPHA, rôle master/slave, ordre des bits, fréquence SCLK et moment exact d'échantillonnage/changement. Beaucoup de pannes SPI sont un demi-cycle de déphasage. Pour SPI master clock divider, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving SPI master clock divider? Answer: I verify CPOL/CPHA mode, master/slave role, bit order, SCLK frequency, and exact sampling/change edges. Many SPI failures are a half-cycle phase mismatch. For SPI master clock divider, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
372. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant SPI master clock divider ? Réponse : Je définis le mode SPI comme paramètre d'interface et dessine un timing diagram avant de coder la FSM. Je traite CS comme faisant partie du protocole, pas comme un simple GPIO décoratif. Pour SPI master clock divider, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding SPI master clock divider? Answer: I define SPI mode as part of the interface and draw a timing diagram before coding the FSM. I treat CS as part of the protocol, not as a decorative GPIO. For SPI master clock divider, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
373. FR - Expliquez shift register SPI. Quelle structure ou idée hardware se cache derrière ? Réponse : Un registre à décalage sérialise MOSI et reconstruit les bits MISO. Le moment du shift dépend du mode CPHA/CPOL.	EN - Explain SPI shift register. What hardware structure or idea is behind it? Answer: A shift register serializes MOSI and reconstructs MISO bits. The shift timing depends on CPHA/CPOL mode.
374. FR - Comment utiliserez-vous shift register SPI dans un design SystemVerilog réel ? Réponse : Je sépare "sample edge" et "shift edge" dans le control RTL pour éviter les erreurs de phase.	EN - How would you use SPI shift register in a real SystemVerilog design? Answer: I separate the sample edge from the shift edge in the RTL control to avoid half-cycle phase errors.
375. FR - Comment diagnostiqueriez-vous un problème lié à shift register SPI ? Réponse : Je vérifie mode CPOL/CPHA, rôle master/slave, ordre des bits, fréquence SCLK et moment exact d'échantillonnage/changement. Beaucoup de pannes SPI sont un demi-cycle de déphasage. Pour shift register SPI, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving SPI shift register? Answer: I verify CPOL/CPHA mode, master/slave role, bit order, SCLK frequency, and exact sampling/change edges. Many SPI failures are a half-cycle phase mismatch. For SPI shift register, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.

376. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant shift register SPI ? Réponse : Je définis le mode SPI comme paramètre d'interface et dessine un timing diagram avant de coder la FSM. Je traite CS comme faisant partie du protocole, pas comme un simple GPIO décoratif. Pour shift register SPI, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding SPI shift register? Answer: I define SPI mode as part of the interface and draw a timing diagram before coding the FSM. I treat CS as part of the protocol, not as a decorative GPIO. For SPI shift register, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
377. FR - Expliquez chip select et transaction. Quelle structure ou idée hardware se cache derrière ? Réponse : CS délimite souvent une transaction et remet à zéro le bit counter côté slave. Certaines devices imposent des temps minimum avant/après SCLK.	EN - Explain chip select and transaction. What hardware structure or idea is behind it? Answer: CS often bounds a transaction and resets the bit counter on the slave side. Some devices require minimum timing before/after SCLK.
378. FR - Comment utiliseriez-vous chip select et transaction dans un design SystemVerilog réel ? Réponse : Je respecte les délais de datasheet et décide ce qui se passe si CS remonte au milieu d'un mot.	EN - How would you use chip select and transaction in a real SystemVerilog design? Answer: I meet the datasheet timing around CS and define what happens if CS deasserts in the middle of a word.
379. FR - Comment diagnostiqueriez-vous un problème lié à chip select et transaction ? Réponse : Je vérifie mode CPOL/CPHA, rôle master/slave, ordre des bits, fréquence SCLK et moment exact d'échantillonnage/changement. Beaucoup de pannes SPI sont un demi-cycle de déphasage. Pour chip select et transaction, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving chip select and transaction? Answer: I verify CPOL/CPHA mode, master/slave role, bit order, SCLK frequency, and exact sampling/change edges. Many SPI failures are a half-cycle phase mismatch. For chip select and transaction, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
380. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant chip select et transaction ? Réponse : Je définis le mode SPI comme paramètre d'interface et dessine un timing diagram avant de coder la FSM. Je traite CS comme faisant partie du protocole, pas comme un simple GPIO décoratif. Pour chip select et transaction, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding chip select and transaction? Answer: I define SPI mode as part of the interface and draw a timing diagram before coding the FSM. I treat CS as part of the protocol, not as a decorative GPIO. For chip select and transaction, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

20. I2C RTL et open-drain / I2C RTL and Open-Drain

381. FR - Expliquez open-drain SDA/SCL. Quelle structure ou idée hardware se cache derrière ? Réponse : Une ligne I2C est tirée au haut par pull-up et les participants ne forcent que le bas. En RTL I/O, on modélise donc drive-low ou high-Z.	EN - Explain open-drain SDA/SCL. What hardware structure or idea is behind it? Answer: An I2C line is pulled high by a resistor and participants only force it low. In RTL I/O, it is therefore modeled as drive-low or high-Z.
382. FR - Comment utiliseriez-vous open-drain SDA/SCL dans un design SystemVerilog réel ? Réponse : Je code <code>assign sda = sda_drive_low ? 1'b0 : 1'bz;</code> et lis la ligne séparément. <code>assign sda = sda_drive_low ? 1'b0 : 1'bz;</code> <code>wire sda_in = sda;</code>	EN - How would you use open-drain SDA/SCL in a real SystemVerilog design? Answer: I code <code>assign sda = sda_drive_low ? 1'b0 : 1'bz;</code> and sense the line separately. <code>assign sda = sda_drive_low ? 1'b0 : 1'bz;</code> <code>wire sda_in = sda;</code>
383. FR - Comment diagnostiqueriez-vous un problème lié à open-drain SDA/SCL ? Réponse : Je vérifie la nature open-drain, pull-ups, start/stop, ACK/NACK, direction SDA et clock stretching. Une ligne qui reste basse est souvent un problème d'ownership ou de state machine. Pour open-drain SDA/SCL, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving open-drain SDA/SCL? Answer: I verify open-drain behavior, pull-ups, start/stop, ACK/NACK, SDA direction, and clock stretching. A line stuck low is often an ownership or state-machine issue. For open-drain SDA/SCL, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
384. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant open-drain SDA/SCL ? Réponse : Je ne drive jamais un '1' actif sur SDA/SCL open-drain: je drive 0 ou je relâche en Z. Je sépare le contrôle de sortie de la lecture réelle de la ligne. Pour open-drain SDA/SCL, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding open-drain SDA/SCL? Answer: I never actively drive a '1' on open-drain SDA/SCL: I drive 0 or release to Z. I separate output drive control from actual line sensing. For open-drain SDA/SCL, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

385. FR - Expliquez START et STOP. Quelle structure ou idée hardware se cache derrière ? Réponse : START est une transition SDA 1→0 pendant SCL haut; STOP est SDA 0→1 pendant SCL haut. Ce sont des événements de protocole distincts des bits de données.	EN - Explain START and STOP. What hardware structure or idea is behind it? Answer: START is an SDA 1→0 transition while SCL is high; STOP is SDA 0→1 while SCL is high. They are protocol events distinct from data bits.
386. FR - Comment utiliseriez-vous START et STOP dans un design SystemVerilog réel ? Réponse : Une FSM master contrôle SDA et SCL dans le bon ordre tout en respectant les temps minimum.	EN - How would you use START and STOP in a real SystemVerilog design? Answer: A master FSM controls SDA and SCL in the required order while respecting minimum timing requirements.
387. FR - Comment diagnostiqueriez-vous un problème lié à START et STOP ? Réponse : Je vérifie la nature open-drain, pull-ups, start/stop, ACK/NACK, direction SDA et clock stretching. Une ligne qui reste basse est souvent un problème d'ownership ou de state machine. Pour START et STOP, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving START and STOP? Answer: I verify open-drain behavior, pull-ups, start/stop, ACK/NACK, SDA direction, and clock stretching. A line stuck low is often an ownership or state-machine issue. For START and STOP, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
388. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant START et STOP ? Réponse : Je ne drive jamais un '1' actif sur SDA/SCL open-drain: je drive 0 ou je relâche en Z. Je sépare le contrôle de sortie de la lecture réelle de la ligne. Pour START et STOP, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding START and STOP? Answer: I never actively drive a '1' on open-drain SDA/SCL: I drive 0 or release to Z. I separate output drive control from actual line sensing. For START and STOP, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
389. FR - Expliquez ACK/NACK. Quelle structure ou idée hardware se cache derrière ? Réponse : Après huit bits, le transmetteur relâche SDA et le récepteur pilote le neuvième bit: bas pour ACK, haut/released pour NACK.	EN - Explain ACK/NACK. What hardware structure or idea is behind it? Answer: After eight bits, the transmitter releases SDA and the receiver drives the ninth bit: low for ACK, high/released for NACK.
390. FR - Comment utiliseriez-vous ACK/NACK dans un design SystemVerilog réel ? Réponse : Je change explicitement l'ownership de SDA et échantillonne l'ACK au bon niveau de SCL.	EN - How would you use ACK/NACK in a real SystemVerilog design? Answer: I explicitly hand over SDA ownership and sample ACK/NACK at the correct SCL phase.
391. FR - Comment diagnostiqueriez-vous un problème lié à ACK/NACK ? Réponse : Je vérifie la nature open-drain, pull-ups, start/stop, ACK/NACK, direction SDA et clock stretching. Une ligne qui reste basse est souvent un problème d'ownership ou de state machine. Pour ACK/NACK, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving ACK/NACK? Answer: I verify open-drain behavior, pull-ups, start/stop, ACK/NACK, SDA direction, and clock stretching. A line stuck low is often an ownership or state-machine issue. For ACK/NACK, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
392. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant ACK/NACK ? Réponse : Je ne drive jamais un '1' actif sur SDA/SCL open-drain: je drive 0 ou je relâche en Z. Je sépare le contrôle de sortie de la lecture réelle de la ligne. Pour ACK/NACK, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding ACK/NACK? Answer: I never actively drive a '1' on open-drain SDA/SCL: I drive 0 or release to Z. I separate output drive control from actual line sensing. For ACK/NACK, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
393. FR - Expliquez clock stretching. Quelle structure ou idée hardware se cache derrière ? Réponse : Un slave peut maintenir SCL bas pour retarder le master. Un master compatible doit lire la ligne physique et attendre qu'elle remonte réellement.	EN - Explain clock stretching. What hardware structure or idea is behind it? Answer: A slave may hold SCL low to delay the master. A compatible master must sense the physical line and wait until it actually rises.
394. FR - Comment utiliseriez-vous clock stretching dans un design SystemVerilog réel ? Réponse : Je distingue `scl_drive_low` de `scl_in`; relâcher SCL ne garantit pas qu'elle soit déjà haute.	EN - How would you use clock stretching in a real SystemVerilog design? Answer: I distinguish `scl_drive_low` from `scl_in`; releasing SCL does not guarantee that the physical line is already high.

395. FR - Comment diagnostiqueriez-vous un problème lié à clock stretching ? Réponse : Je vérifie la nature open-drain, pull-ups, start/stop, ACK/NACK, direction SDA et clock stretching. Une ligne qui reste basse est souvent un problème d'ownership ou de state machine. Pour clock stretching, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock stretching? Answer: I verify open-drain behavior, pull-ups, start/stop, ACK/NACK, SDA direction, and clock stretching. A line stuck low is often an ownership or state-machine issue. For clock stretching, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
396. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock stretching ? Réponse : Je ne drive jamais un '1' actif sur SDA/SCL open-drain: je drive 0 ou je relâche en Z. Je sépare le contrôle de sortie de la lecture réelle de la ligne. Pour clock stretching, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock stretching? Answer: I never actively drive a '1' on open-drain SDA/SCL: I drive 0 or release to Z. I separate output drive control from actual line sensing. For clock stretching, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
397. FR - Expliquez arbitration I2C. Quelle structure ou idée hardware se cache derrière ? Réponse : En multi-master, un participant qui relâche SDA mais lit 0 perd l'arbitrage. Cela repose précisément sur la nature wired-AND/open-drain du bus.	EN - Explain I2C arbitration. What hardware structure or idea is behind it? Answer: In multi-master operation, a participant that releases SDA but reads 0 loses arbitration. This relies on the wired-AND/open-drain nature of the bus.
398. FR - Comment utiliseriez-vous arbitration I2C dans un design SystemVerilog réel ? Réponse : Je n'implémente arbitration que si la spécification système l'exige, mais je comprends le mécanisme en entretien.	EN - How would you use I2C arbitration in a real SystemVerilog design? Answer: I implement arbitration only when the system specification requires multi-master support, but I understand and can explain the mechanism.
399. FR - Comment diagnostiqueriez-vous un problème lié à arbitration I2C ? Réponse : Je vérifie la nature open-drain, pull-ups, start/stop, ACK/NACK, direction SDA et clock stretching. Une ligne qui reste basse est souvent un problème d'ownership ou de state machine. Pour arbitration I2C, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving I2C arbitration? Answer: I verify open-drain behavior, pull-ups, start/stop, ACK/NACK, SDA direction, and clock stretching. A line stuck low is often an ownership or state-machine issue. For I2C arbitration, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
400. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant arbitration I2C ? Réponse : Je ne drive jamais un '1' actif sur SDA/SCL open-drain: je drive 0 ou je relâche en Z. Je sépare le contrôle de sortie de la lecture réelle de la ligne. Pour arbitration I2C, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding I2C arbitration? Answer: I never actively drive a '1' on open-drain SDA/SCL: I drive 0 or release to Z. I separate output drive control from actual line sensing. For I2C arbitration, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

21. Testbench SystemVerilog: stimulus, interfaces, monitor et scoreboard / SystemVerilog Testbench: Stimulus, Interfaces, Monitor, and Scoreboard

401. FR - Expliquez un testbench SystemVerilog self-checking. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un testbench self-checking génère stimulus, observe le DUT, calcule l'attendu et décide automatiquement pass/fail. La waveform devient un outil de debug et non le seul moyen de validation.	EN - Explain a self-checking SystemVerilog testbench. What important difference, structure, or idea should be understood? Answer: A self-checking testbench generates stimulus, observes the DUT, computes expected behavior, and automatically decides pass/fail. The waveform becomes a debugging tool rather than the sole validation method.
402. FR - Comment utiliseriez-vous un testbench SystemVerilog self-checking dans un projet SystemVerilog réel ? Réponse : Je sépare stimulus, driver, monitor et checker/scoreboard même dans un environnement simple.	EN - How would you use a self-checking SystemVerilog testbench in a real SystemVerilog project? Answer: I separate stimulus, driver, monitor, and checker/scoreboard even in a simple environment.
403. FR - Comment diagnostiqueriez-vous un problème lié à un testbench SystemVerilog self-checking ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour un testbench SystemVerilog self-checking, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to a self-checking SystemVerilog testbench? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For a self-checking SystemVerilog testbench, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

404. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant un testbench SystemVerilog self-checking ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour un testbench SystemVerilog self-checking, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding a self-checking SystemVerilog testbench? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For a self-checking SystemVerilog testbench, I also explain the expected hardware or simulation-only role, together with important corner cases.
405. FR - Expliquez transaction versus activité signal-level. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une transaction décrit une opération de haut niveau; le driver la traduit en signaux et le monitor reconstruit les transactions observées. Cette abstraction rend les tests réutilisables.	EN - Explain transaction versus signal-level activity. What important difference, structure, or idea should be understood? Answer: A transaction describes a high-level operation; the driver translates it into signals and the monitor reconstructs observed transactions. This abstraction makes tests reusable.
406. FR - Comment utiliseriez-vous transaction versus activité signal-level dans un projet SystemVerilog réel ? Réponse : Je garde le protocole bit-level dans le driver/monitor et le scénario fonctionnel au niveau transaction.	EN - How would you use transaction versus signal-level activity in a real SystemVerilog project? Answer: I keep bit-level protocol details in the driver/monitor and functional scenarios at transaction level.
407. FR - Comment diagnostiqueriez-vous un problème lié à transaction versus activité signal-level ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour transaction versus activité signal-level, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to transaction versus signal-level activity? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For transaction versus signal-level activity, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
408. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant transaction versus activité signal-level ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour transaction versus activité signal-level, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding transaction versus signal-level activity? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For transaction versus signal-level activity, I also explain the expected hardware or simulation-only role, together with important corner cases.
409. FR - Expliquez clocking block dans un testbench. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un clocking block définit quand les signaux sont sampled et driven autour d'une horloge et aide à éviter les races entre DUT et testbench.	EN - Explain clocking block in a testbench. What important difference, structure, or idea should be understood? Answer: A clocking block defines when signals are sampled and driven around a clock and helps avoid races between DUT and testbench.
410. FR - Comment utiliseriez-vous clocking block dans un testbench dans un projet SystemVerilog réel ? Réponse : Je l'utilise sur les interfaces synchrones lorsque le testbench class-based doit piloter et observer précisément les cycles.	EN - How would you use clocking block in a testbench in a real SystemVerilog project? Answer: I use it on synchronous interfaces when a class-based testbench must drive and observe cycles precisely.
411. FR - Comment diagnostiqueriez-vous un problème lié à clocking block dans un testbench ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour clocking block dans un testbench, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to clocking block in a testbench? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For clocking block in a testbench, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
412. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clocking block dans un testbench ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour clocking block dans un testbench, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding clocking block in a testbench? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For clocking block in a testbench, I also explain the expected hardware or simulation-only role, together with important corner cases.
413. FR - Expliquez monitor et scoreboard. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Le monitor est passif et reconstruit ce qui s'est réellement passé; le scoreboard compare ces observations avec un modèle de référence ou des attentes.	EN - Explain monitor and scoreboard. What important difference, structure, or idea should be understood? Answer: The monitor is passive and reconstructs what actually happened; the scoreboard compares those observations with a reference model or expectations.

414. FR - Comment utiliseriez-vous monitor et scoreboard dans un projet SystemVerilog réel ? Réponse : Je garde le monitor indépendant du stimulus afin qu'un bug du driver ne soit pas automatiquement reproduit dans le checker.	EN - How would you use monitor and scoreboard in a real SystemVerilog project? Answer: I keep the monitor independent of stimulus so a driver bug is not automatically duplicated in the checker.
415. FR - Comment diagnostiqueriez-vous un problème lié à monitor et scoreboard ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour monitor et scoreboard, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to monitor and scoreboard? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For monitor and scoreboard, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
416. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant monitor et scoreboard ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour monitor et scoreboard, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding monitor and scoreboard? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For monitor and scoreboard, I also explain the expected hardware or simulation-only role, together with important corner cases.
417. FR - Expliquez termination déterministe et reporting de test. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un bon testbench termine avec un état pass/fail clair, une seed reproductible, un nombre d'erreurs et suffisamment de contexte pour reproduire le failure.	EN - Explain deterministic test termination and reporting. What important difference, structure, or idea should be understood? Answer: A good testbench ends with a clear pass/fail state, reproducible seed, error count, and enough context to reproduce the failure.
418. FR - Comment utiliseriez-vous termination déterministe et reporting de test dans un projet SystemVerilog réel ? Réponse : Je journalise test, seed, configuration et première erreur, puis je termine proprement après drain des transactions attendues.	EN - How would you use deterministic test termination and reporting in a real SystemVerilog project? Answer: I log test, seed, configuration, and first error, then terminate cleanly after expected transactions are drained.
419. FR - Comment diagnostiqueriez-vous un problème lié à termination déterministe et reporting de test ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour termination déterministe et reporting de test, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to deterministic test termination and reporting? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For deterministic test termination and reporting, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
420. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant termination déterministe et reporting de test ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour termination déterministe et reporting de test, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding deterministic test termination and reporting? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For deterministic test termination and reporting, I also explain the expected hardware or simulation-only role, together with important corner cases.

22. Classes, randomisation contrainte et collections dynamiques / Classes, Constrained Randomization, and Dynamic Collections

421. FR - Expliquez classes et object handles. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une class crée des objets dynamiques de verification. Les variables class sont des handles; deux handles peuvent référencer le même objet, ce qui est très différent d'une structure RTL statique.	EN - Explain classes and object handles. What important difference, structure, or idea should be understood? Answer: A class creates dynamic verification objects. Class variables are handles; two handles can reference the same object, which is very different from static RTL structure.
422. FR - Comment utiliseriez-vous classes et object handles dans un projet SystemVerilog réel ? Réponse : Je les utilise pour transactions et composants de testbench, avec `new`, copy/clone et ownership clairement définis.	EN - How would you use classes and object handles in a real SystemVerilog project? Answer: I use them for transactions and testbench components, with `new`, copy/clone, and clearly defined ownership.

423. FR - Comment diagnostiqueriez-vous un problème lié à classes et object handles ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour classes et object handles, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to classes and object handles? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For classes and object handles, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
424. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant classes et object handles ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour classes et object handles, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding classes and object handles? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For classes and object handles, I also explain the expected hardware or simulation-only role, together with important corner cases.
425. FR - Expliquez inheritance, virtual methods et polymorphism. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : L'héritage permet de spécialiser une classe de base; les méthodes virtuelles permettent un comportement polymorphe selon le type réel de l'objet.	EN - Explain inheritance, virtual methods, and polymorphism. What important difference, structure, or idea should be understood? Answer: Inheritance allows specialization of a base class; virtual methods enable polymorphic behavior according to the object's actual type.
426. FR - Comment utiliseriez-vous inheritance, virtual methods et polymorphism dans un projet SystemVerilog réel ? Réponse : Je les utilise pour réutiliser des transactions/drivers communs sans dupliquer l'infrastructure.	EN - How would you use inheritance, virtual methods, and polymorphism in a real SystemVerilog project? Answer: I use them to reuse common transactions/drivers without duplicating infrastructure.
427. FR - Comment diagnostiqueriez-vous un problème lié à inheritance, virtual methods et polymorphism ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour inheritance, virtual methods et polymorphism, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to inheritance, virtual methods, and polymorphism? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For inheritance, virtual methods, and polymorphism, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
428. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant inheritance, virtual methods et polymorphism ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour inheritance, virtual methods et polymorphism, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding inheritance, virtual methods, and polymorphism? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For inheritance, virtual methods, and polymorphism, I also explain the expected hardware or simulation-only role, together with important corner cases.
429. FR - Expliquez `rand`, constraints et `randomize()`. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : La randomisation contrainte explore automatiquement l'espace légal défini par des constraint blocks. `randomize()` peut échouer si les contraintes sont contradictoires.	EN - Explain `rand`, constraints, and `randomize()`. What important difference, structure, or idea should be understood? Answer: Constrained randomization automatically explores the legal space defined by constraint blocks. `randomize()` can fail when constraints conflict.
430. FR - Comment utiliseriez-vous `rand`, constraints et `randomize()` dans un projet SystemVerilog réel ? Réponse : Je vérifie toujours la valeur de retour, conserve la seed et utilise inline constraints pour les corner cases ciblés.	EN - How would you use `rand`, constraints, and `randomize()` in a real SystemVerilog project? Answer: I always check the return value, preserve the seed, and use inline constraints for targeted corner cases.
431. FR - Comment diagnostiqueriez-vous un problème lié à `rand`, constraints et `randomize()` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `rand`, constraints et `randomize()`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `rand`, constraints, and `randomize()`? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `rand`, constraints, and `randomize()`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

432. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `rand`, constraints et `randomize()` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour `rand`, constraints et `randomize()`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `rand`, constraints, and `randomize()`? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `rand`, constraints, and `randomize()`, I also explain the expected hardware or simulation-only role, together with important corner cases.
433. FR - Expliquez queues, dynamic arrays et associative arrays. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Ces collections dynamiques sont puissantes en testbench: queue pour ordre FIFO, dynamic array pour taille variable, associative array pour index sparse ou transaction ID.	EN - Explain queues, dynamic arrays, and associative arrays. What important difference, structure, or idea should be understood? Answer: These dynamic collections are powerful in testbenches: queues for FIFO ordering, dynamic arrays for variable size, and associative arrays for sparse indexes or transaction IDs.
434. FR - Comment utiliseriez-vous queues, dynamic arrays et associative arrays dans un projet SystemVerilog réel ? Réponse : Je choisis la collection selon la relation d'ordre et d'identité du protocole, pas uniquement par commodité syntaxique.	EN - How would you use queues, dynamic arrays, and associative arrays in a real SystemVerilog project? Answer: I choose the collection based on protocol ordering and identity, not merely syntax convenience.
435. FR - Comment diagnostiqueriez-vous un problème lié à queues, dynamic arrays et associative arrays ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour queues, dynamic arrays et associative arrays, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to queues, dynamic arrays, and associative arrays? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For queues, dynamic arrays, and associative arrays, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
436. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant queues, dynamic arrays et associative arrays ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour queues, dynamic arrays et associative arrays, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding queues, dynamic arrays, and associative arrays? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For queues, dynamic arrays, and associative arrays, I also explain the expected hardware or simulation-only role, together with important corner cases.
437. FR - Expliquez mailbox et semaphore. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une mailbox échange des objets entre threads de simulation; un semaphore protège une ressource partagée. Ce sont des primitives de testbench, pas du hardware synthétisé.	EN - Explain mailbox and semaphore. What important difference, structure, or idea should be understood? Answer: A mailbox exchanges objects between simulation threads; a semaphore protects a shared resource. These are testbench primitives, not synthesized hardware.
438. FR - Comment utiliseriez-vous mailbox et semaphore dans un projet SystemVerilog réel ? Réponse : Je les utilise dans un environnement class-based simple pour producer/consumer et mutual exclusion, avec attention aux deadlocks.	EN - How would you use mailbox and semaphore in a real SystemVerilog project? Answer: I use them in a simple class-based environment for producer/consumer and mutual exclusion, with attention to deadlocks.
439. FR - Comment diagnostiqueriez-vous un problème lié à mailbox et semaphore ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour mailbox et semaphore, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to mailbox and semaphore? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For mailbox and semaphore, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
440. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant mailbox et semaphore ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour mailbox et semaphore, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding mailbox and semaphore? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For mailbox and semaphore, I also explain the expected hardware or simulation-only role, together with important corner cases.

23. SVA, assertions, couverture fonctionnelle et formal concepts / SVA, Assertions, Functional Coverage, and Formal Concepts

441. FR - Expliquez immediate versus concurrent assertions. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Une immediate assertion teste une condition au point procédural courant; une concurrent assertion SVA vérifie une propriété temporelle échantillonnée sur un événement, souvent <code>`posedge clk`</code>.	EN - Explain immediate versus concurrent assertions. What important difference, structure, or idea should be understood? Answer: An immediate assertion checks a condition at the current procedural point; a concurrent SVA assertion checks a temporal property sampled on an event, often <code>`posedge clk`</code>.
442. FR - Comment utiliserez-vous immediate versus concurrent assertions dans un projet SystemVerilog réel ? Réponse : J'utilise immediate pour les invariants locaux et concurrent SVA pour handshakes, latences et protocoles multi-cycles.	EN - How would you use immediate versus concurrent assertions in a real SystemVerilog project? Answer: I use immediate assertions for local invariants and concurrent SVA for handshakes, latency, and multi-cycle protocols.
443. FR - Comment diagnostiqueriez-vous un problème lié à immediate versus concurrent assertions ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour immediate versus concurrent assertions, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to immediate versus concurrent assertions? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For immediate versus concurrent assertions, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
444. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant immediate versus concurrent assertions ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour immediate versus concurrent assertions, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding immediate versus concurrent assertions? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For immediate versus concurrent assertions, I also explain the expected hardware or simulation-only role, together with important corner cases.
445. FR - Expliquez <code>` -></code>, <code>` =></code>, <code>`##</code> et timing SVA. Quelle différence, structure ou idée importante faut-il comprendre ?</p> <p>Réponse : <code>` -></code> est une implication overlapped, <code>` =></code> commence le conséquent au cycle suivant, et <code>`##</code> décrit un délai en événements d'échantillonnage. Les erreurs d'un cycle sont fréquentes.	EN - Explain <code>` -></code>, <code>` =></code>, <code>`##</code>, and SVA timing. What important difference, structure, or idea should be understood?</p> <p>Answer: <code>` -></code> is overlapped implication, <code>` =></code> starts the consequent on the next cycle, and <code>`##</code> describes delay in sampling events. One-cycle errors are common.
446. FR - Comment utiliserez-vous <code>` -></code>, <code>` =></code>, <code>`##</code> et timing SVA dans un projet SystemVerilog réel ?</p> <p>Réponse : Je dessine la timeline attendue avant d'écrire une propriété complexe et j'utilise <code>`disable iff`</code> pour gérer reset.	EN - How would you use <code>` -></code>, <code>` =></code>, <code>`##</code>, and SVA timing in a real SystemVerilog project?</p> <p>Answer: I draw the expected timeline before writing a complex property and use <code>`disable iff`</code> to handle reset.
447. FR - Comment diagnostiqueriez-vous un problème lié à <code>` -></code>, <code>` =></code>, <code>`##</code> et timing SVA ?</p> <p>Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour <code>` -></code>, <code>` =></code>, <code>`##</code> et timing SVA, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to <code>` -></code>, <code>` =></code>, <code>`##</code>, and SVA timing?</p> <p>Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For <code>` -></code>, <code>` =></code>, <code>`##</code>, and SVA timing, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
448. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant <code>` -></code>, <code>` =></code>, <code>`##</code> et timing SVA ?</p> <p>Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour <code>` -></code>, <code>` =></code>, <code>`##</code> et timing SVA, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding <code>` -></code>, <code>` =></code>, <code>`##</code>, and SVA timing?</p> <p>Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For <code>` -></code>, <code>` =></code>, <code>`##</code>, and SVA timing, I also explain the expected hardware or simulation-only role, together with important corner cases.
449. FR - Expliquez <code>`assert`</code>, <code>`assume`</code> et <code>`cover`</code>. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : <code>`assert`</code> exprime une obligation, <code>`assume`</code> une hypothèse d'environnement surtout en formal, et <code>`cover`</code> cherche/mesure un scénario intéressant.	EN - Explain <code>`assert`</code>, <code>`assume`</code>, and <code>`cover`</code>. What important difference, structure, or idea should be understood? Answer: <code>`assert`</code> expresses an obligation, <code>`assume`</code> an environment assumption especially in formal, and <code>`cover`</code> seeks/measures an interesting scenario.
450. FR - Comment utiliserez-vous <code>`assert`</code>, <code>`assume`</code> et <code>`cover`</code> dans un projet SystemVerilog réel ? Réponse : Je limite les assumptions aux vraies contraintes externes afin de ne pas prouver le design dans un environnement irréaliste.	EN - How would you use <code>`assert`</code>, <code>`assume`</code>, and <code>`cover`</code> in a real SystemVerilog project? Answer: I limit assumptions to true external constraints so the design is not proven only in an unrealistic environment.

451. FR - Comment diagnostiqueriez-vous un problème lié à `assert`, `assume` et `cover` ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour `assert`, `assume` et `cover`, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to `assert`, `assume`, and `cover`? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For `assert`, `assume`, and `cover`, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
452. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant `assert`, `assume` et `cover` ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour `assert`, `assume` et `cover`, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding `assert`, `assume`, and `cover`? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For `assert`, `assume`, and `cover`, I also explain the expected hardware or simulation-only role, together with important corner cases.
453. FR - Expliquez covergroup, coverpoint et cross coverage. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : La functional coverage mesure quels scénarios définis par le plan de verification ont été exercés. Les crosses mesurent les combinaisons significatives entre paramètres.	EN - Explain covergroup, coverpoint, and cross coverage. What important difference, structure, or idea should be understood? Answer: Functional coverage measures which verification-plan scenarios were exercised. Crosses measure meaningful combinations among parameters.
454. FR - Comment utiliseriez-vous covergroup, coverpoint et cross coverage dans un projet SystemVerilog réel ? Réponse : Je définis bins et crosses à partir des exigences, puis j'analyse les holes au lieu de viser aveuglément 100 %.	EN - How would you use covergroup, coverpoint, and cross coverage in a real SystemVerilog project? Answer: I define bins and crosses from requirements, then analyze holes instead of blindly targeting 100%.
455. FR - Comment diagnostiqueriez-vous un problème lié à covergroup, coverpoint et cross coverage ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour covergroup, coverpoint et cross coverage, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to covergroup, coverpoint, and cross coverage? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For covergroup, coverpoint, and cross coverage, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
456. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant covergroup, coverpoint et cross coverage ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour covergroup, coverpoint et cross coverage, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding covergroup, coverpoint, and cross coverage? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For covergroup, coverpoint, and cross coverage, I also explain the expected hardware or simulation-only role, together with important corner cases.
457. FR - Expliquez code coverage versus functional coverage. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : La code coverage indique si lignes, branches, toggles ou états ont été exercés; la functional coverage indique si les scénarios de la spec ont été testés. Aucune des deux ne prouve seule la correction.	EN - Explain code coverage versus functional coverage. What important difference, structure, or idea should be understood? Answer: Code coverage indicates whether lines, branches, toggles, or states were exercised; functional coverage indicates whether specification scenarios were tested. Neither alone proves correctness.
458. FR - Comment utiliseriez-vous code coverage versus functional coverage dans un projet SystemVerilog réel ? Réponse : Je combine coverage avec assertions, scoreboards et plan de verification pour le signoff.	EN - How would you use code coverage versus functional coverage in a real SystemVerilog project? Answer: I combine coverage with assertions, scoreboards, and the verification plan for signoff.
459. FR - Comment diagnostiqueriez-vous un problème lié à code coverage versus functional coverage ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour code coverage versus functional coverage, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to code coverage versus functional coverage? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For code coverage versus functional coverage, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

460. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant code coverage versus functional coverage ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la vérification. Pour code coverage versus functional coverage, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding code coverage versus functional coverage? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For code coverage versus functional coverage, I also explain the expected hardware or simulation-only role, together with important corner cases.
---	--

24. Scheduling, synthèse, lint, timing et simulation-synthesis mismatch / Scheduling, Synthesis, Lint, Timing, and Simulation-Synthesis Mismatch

461. FR - Expliquez setup time. Quelle structure ou idée hardware se cache derrière ? Réponse : Le setup exige que la donnée soit stable suffisamment avant le front actif du flip-flop destination. Une violation signifie que le chemin de données est trop lent pour la période et les incertitudes imposées.	EN - Explain setup time. What hardware structure or idea is behind it? Answer: Setup requires data to be stable sufficiently before the destination flip-flop's active edge. A violation means the data path is too slow for the imposed period and uncertainties.
--	--

462. FR - Comment utiliseriez-vous setup time dans un design SystemVerilog réel ? Réponse : Je réduis la profondeur combinatoire, pipeline ou ajuste l'architecture plutôt que de simplement ignorer la violation.	EN - How would you use setup time in a real SystemVerilog design? Answer: I reduce combinational depth, pipeline, or change the architecture rather than simply ignoring a setup violation.
--	---

463. FR - Comment diagnostiqueriez-vous un problème lié à setup time ? Réponse : Je sépare problème fonctionnel et problème temporel. Pour le timing, j'identifie startpoint, endpoint, clock relationship, combinational delay et contraintes avant de modifier le RTL. Pour setup time, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving setup time? Answer: I separate functional problems from timing problems. For timing, I identify startpoint, endpoint, clock relationship, combinational delay, and constraints before changing RTL. For setup time, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
--	---

464. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant setup time ? Réponse : Le RTL seul ne contient pas toutes les exigences de timing. Je considère les clocks et I/O constraints comme partie de la spécification du design et je pipeline les chemins critiques plutôt que de cacher une violation. Pour setup time, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding setup time? Answer: RTL alone does not contain all timing requirements. I treat clocks and I/O constraints as part of the design specification and pipeline critical paths rather than hiding a violation. For setup time, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	--

465. FR - Expliquez hold time. Quelle structure ou idée hardware se cache derrière ? Réponse : Le hold exige que la donnée reste stable un court instant après le front. C'est un problème de chemin trop rapide/skew, différent du setup.	EN - Explain hold time. What hardware structure or idea is behind it? Answer: Hold requires data to remain stable for a short time after the edge. It is a too-fast-path/skew issue, different from setup.
--	--

466. FR - Comment utiliseriez-vous hold time dans un design SystemVerilog réel ? Réponse : Je laisse généralement le place-and-route insérer du délai lorsqu'approprié et je vérifie les clocks/contraintes.	EN - How would you use hold time in a real SystemVerilog design? Answer: I generally let place-and-route apply appropriate delay fixes while I verify clock definitions, constraints, and path classification.
--	--

467. FR - Comment diagnostiqueriez-vous un problème lié à hold time ? Réponse : Je sépare problème fonctionnel et problème temporel. Pour le timing, j'identifie startpoint, endpoint, clock relationship, combinational delay et contraintes avant de modifier le RTL. Pour hold time, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving hold time? Answer: I separate functional problems from timing problems. For timing, I identify startpoint, endpoint, clock relationship, combinational delay, and constraints before changing RTL. For hold time, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
--	---

468. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant hold time ? Réponse : Le RTL seul ne contient pas toutes les exigences de timing. Je considère les clocks et I/O constraints comme partie de la spécification du design et je pipeline les chemins critiques plutôt que de cacher une violation. Pour hold time, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding hold time? Answer: RTL alone does not contain all timing requirements. I treat clocks and I/O constraints as part of the design specification and pipeline critical paths rather than hiding a violation. For hold time, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	--

469. FR - Expliquez critical path. Quelle structure ou idée hardware se cache derrière ? Réponse : Le critical path est le chemin temporel qui limite la fréquence maximale ou la marge de timing. Il peut traverser mux, additionneurs, comparateurs et routing.	EN - Explain critical path. What hardware structure or idea is behind it? Answer: The critical path is the timing path that limits maximum frequency or timing margin. It may traverse muxes, adders, comparators, and routing.
470. FR - Comment utiliseriez-vous critical path dans un design SystemVerilog réel ? Réponse : Je lis le timing report pour identifier les cellules exactes avant d'ajouter un pipeline ciblé.	EN - How would you use critical path in a real SystemVerilog design? Answer: I read the timing report to identify the exact startpoint, endpoint, and cells before adding a targeted pipeline stage.
471. FR - Comment diagnostiqueriez-vous un problème lié à critical path ? Réponse : Je sépare problème fonctionnel et problème temporel. Pour le timing, j'identifie startpoint, endpoint, clock relationship, combinational delay et contraintes avant de modifier le RTL. Pour critical path, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving critical path? Answer: I separate functional problems from timing problems. For timing, I identify startpoint, endpoint, clock relationship, combinational delay, and constraints before changing RTL. For critical path, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
472. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant critical path ? Réponse : Le RTL seul ne contient pas toutes les exigences de timing. Je considère les clocks et I/O constraints comme partie de la spécification du design et je pipeline les chemins critiques plutôt que de cacher une violation. Pour critical path, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding critical path? Answer: RTL alone does not contain all timing requirements. I treat clocks and I/O constraints as part of the design specification and pipeline critical paths rather than hiding a violation. For critical path, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
473. FR - Expliquez clock domain constraints. Quelle structure ou idée hardware se cache derrière ? Réponse : Les outils doivent connaître la période et les relations entre clocks pour analyser correctement les chemins. Les CDC asynchrones ne doivent pas être "réparés" par une fausse contrainte fonctionnelle.	EN - Explain clock domain constraints. What hardware structure or idea is behind it? Answer: Tools must know clock periods and relationships to analyze paths correctly. Asynchronous CDCs must not be "fixed" with a false functional constraint.
474. FR - Comment utiliseriez-vous clock domain constraints dans un design SystemVerilog réel ? Réponse : Je déclare les clocks et relations puis utilise une architecture CDC correcte pour les crossings.	EN - How would you use clock domain constraints in a real SystemVerilog design? Answer: I declare the clocks and their relationships, then use a correct CDC architecture for asynchronous crossings.
475. FR - Comment diagnostiqueriez-vous un problème lié à clock domain constraints ? Réponse : Je sépare problème fonctionnel et problème temporel. Pour le timing, j'identifie startpoint, endpoint, clock relationship, combinational delay et contraintes avant de modifier le RTL. Pour clock domain constraints, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock domain constraints? Answer: I separate functional problems from timing problems. For timing, I identify startpoint, endpoint, clock relationship, combinational delay, and constraints before changing RTL. For clock domain constraints, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
476. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock domain constraints ? Réponse : Le RTL seul ne contient pas toutes les exigences de timing. Je considère les clocks et I/O constraints comme partie de la spécification du design et je pipeline les chemins critiques plutôt que de cacher une violation. Pour clock domain constraints, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock domain constraints? Answer: RTL alone does not contain all timing requirements. I treat clocks and I/O constraints as part of the design specification and pipeline critical paths rather than hiding a violation. For clock domain constraints, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
477. FR - Expliquez reset recovery/removal. Quelle structure ou idée hardware se cache derrière ? Réponse : La désassertion d'un reset asynchrone trop proche d'un front de clock peut violer recovery/removal et libérer des registres de façon incohérente.	EN - Explain reset recovery/removal. What hardware structure or idea is behind it? Answer: Deasserting an asynchronous reset too close to a clock edge can violate recovery/removal and release registers inconsistently.
478. FR - Comment utiliseriez-vous reset recovery/removal dans un design SystemVerilog réel ? Réponse : Je synchronise généralement la désassertion dans chaque domaine de clock selon la stratégie reset du système.	EN - How would you use reset recovery/removal in a real SystemVerilog design? Answer: I generally synchronize asynchronous-reset deassertion within each clock domain according to the system reset strategy.

479. FR - Comment diagnostiqueriez-vous un problème lié à reset recovery/removal ? Réponse : Je sépare problème fonctionnel et problème temporel. Pour le timing, j'identifie startpoint, endpoint, clock relationship, combinational delay et contraintes avant de modifier le RTL. Pour reset recovery/removal, je reviens ensuite au signal ou à la construction précise et je vérifie largeur, drivers, scheduling, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving reset recovery/removal? Answer: I separate functional problems from timing problems. For timing, I identify startpoint, endpoint, clock relationship, combinational delay, and constraints before changing RTL. For reset recovery/removal, I then return to the exact signal or construct and verify width, drivers, scheduling, waveform, and inferred hardware.
480. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant reset recovery/removal ? Réponse : Le RTL seul ne contient pas toutes les exigences de timing. Je considère les clocks et I/O constraints comme partie de la spécification du design et je pipeline les chemins critiques plutôt que de cacher une violation. Pour reset recovery/removal, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding reset recovery/removal? Answer: RTL alone does not contain all timing requirements. I treat clocks and I/O constraints as part of the design specification and pipeline critical paths rather than hiding a violation. For reset recovery/removal, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

25. UVM, troubleshooting et scénarios d'entretien / UVM, Troubleshooting, and Interview Scenarios

481. FR - Expliquez UVM par rapport à SystemVerilog. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : SystemVerilog est le langage; UVM est une méthodologie/bibliothèque standard construite dessus pour organiser des environnements de verification réutilisables.	EN - Explain UVM relative to SystemVerilog. What important difference, structure, or idea should be understood? Answer: SystemVerilog is the language; UVM is a standard methodology/library built on it to organize reusable verification environments.
482. FR - Comment utiliseriez-vous UVM par rapport à SystemVerilog dans un projet SystemVerilog réel ? Réponse : Pour un hardware engineer, je connais le rôle des agents, sequences, driver, monitor et scoreboard; pour un poste verification, j'approfondis phases, factory, config DB et TLM.	EN - How would you use UVM relative to SystemVerilog in a real SystemVerilog project? Answer: As a hardware engineer, I know the role of agents, sequences, driver, monitor, and scoreboard; for a verification role, I go deeper into phases, factory, config DB, and TLM.
483. FR - Comment diagnostiqueriez-vous un problème lié à UVM par rapport à SystemVerilog ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour UVM par rapport à SystemVerilog, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to UVM relative to SystemVerilog? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For UVM relative to SystemVerilog, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
484. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant UVM par rapport à SystemVerilog ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour UVM par rapport à SystemVerilog, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding UVM relative to SystemVerilog? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For UVM relative to SystemVerilog, I also explain the expected hardware or simulation-only role, together with important corner cases.
485. FR - Expliquez agent, sequencer, driver et monitor. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Un agent encapsule généralement le sequencer/driver qui produit l'activité et un monitor passif qui reconstruit les transactions observées.	EN - Explain agent, sequencer, driver, and monitor. What important difference, structure, or idea should be understood? Answer: An agent generally encapsulates the sequencer/driver producing activity and a passive monitor reconstructing observed transactions.
486. FR - Comment utiliseriez-vous agent, sequencer, driver et monitor dans un projet SystemVerilog réel ? Réponse : Je sépare génération, driving et observation afin qu'un bug d'un composant ne soit pas automatiquement reproduit dans un autre.	EN - How would you use agent, sequencer, driver, and monitor in a real SystemVerilog project? Answer: I separate generation, driving, and observation so a bug in one component is not automatically reproduced in another.
487. FR - Comment diagnostiqueriez-vous un problème lié à agent, sequencer, driver et monitor ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour agent, sequencer, driver et monitor, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to agent, sequencer, driver, and monitor? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For agent, sequencer, driver, and monitor, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.

488. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant agent, sequencer, driver et monitor ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour agent, sequencer, driver et monitor, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding agent, sequencer, driver, and monitor? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For agent, sequencer, driver, and monitor, I also explain the expected hardware or simulation-only role, together with important corner cases.
489. FR - Expliquez méthode de debug d'un failure randomisé. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Je conserve la seed, reproduis le failure, identifie la première divergence puis sépare stimulus, driver, DUT, monitor, scoreboard et assertions.	EN - Explain debug method for a randomized failure. What important difference, structure, or idea should be understood? Answer: I preserve the seed, reproduce the failure, identify the first divergence, then separate stimulus, driver, DUT, monitor, scoreboard, and assertions.
490. FR - Comment utiliseriez-vous méthode de debug d'un failure randomisé dans un projet SystemVerilog réel ? Réponse : Je combine transaction logs et waveform ciblée plutôt que d'observer immédiatement tous les signaux du design.	EN - How would you use debug method for a randomized failure in a real SystemVerilog project? Answer: I combine transaction logs with a targeted waveform rather than immediately viewing every signal in the design.
491. FR - Comment diagnostiqueriez-vous un problème lié à méthode de debug d'un failure randomisé ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour méthode de debug d'un failure randomisé, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to debug method for a randomized failure? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For debug method for a randomized failure, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
492. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant méthode de debug d'un failure randomisé ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour méthode de debug d'un failure randomisé, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding debug method for a randomized failure? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For debug method for a randomized failure, I also explain the expected hardware or simulation-only role, together with important corner cases.
493. FR - Expliquez choix SystemVerilog pour un nouveau FPGA. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : SystemVerilog est souvent préférable à SystemVerilog pour un nouveau RTL grâce aux types, `always_comb/ff`, enum, interfaces et assertions, si les outils et IP du projet le supportent.	EN - Explain choosing SystemVerilog for a new FPGA. What important difference, structure, or idea should be understood? Answer: SystemVerilog is often preferable to SystemVerilog for new RTL because of types, `always_comb/ff`, enum, interfaces, and assertions, if project tools and IP support it.
494. FR - Comment utiliseriez-vous choix SystemVerilog pour un nouveau FPGA dans un projet SystemVerilog réel ? Réponse : J'explique en entretien le compromis entre support du toolchain, compatibilité legacy, maintenabilité et besoins de verification.	EN - How would you use choosing SystemVerilog for a new FPGA in a real SystemVerilog project? Answer: In an interview, I explain the tradeoff among toolchain support, legacy compatibility, maintainability, and verification needs.
495. FR - Comment diagnostiqueriez-vous un problème lié à choix SystemVerilog pour un nouveau FPGA ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour choix SystemVerilog pour un nouveau FPGA, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to choosing SystemVerilog for a new FPGA? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For choosing SystemVerilog for a new FPGA, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
496. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant choix SystemVerilog pour un nouveau FPGA ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour choix SystemVerilog pour un nouveau FPGA, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding choosing SystemVerilog for a new FPGA? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For choosing SystemVerilog for a new FPGA, I also explain the expected hardware or simulation-only role, together with important corner cases.

497. FR - Expliquez méthode de revue SystemVerilog en entretien. Quelle différence, structure ou idée importante faut-il comprendre ? Réponse : Je classe d'abord le code en RTL combinatoire, séquentiel, mémoire, CDC ou verification; puis je vérifie types/largeurs, ownership, clocks/resets, scheduling, synthétisabilité, assertions et hardware attendu.	EN - Explain SystemVerilog review method in an interview. What important difference, structure, or idea should be understood? Answer: I first classify code as combinational RTL, sequential RTL, memory, CDC, or verification; then I check types/widths, ownership, clocks/resets, scheduling, synthesizability, assertions, and expected hardware.
498. FR - Comment utiliseriez-vous méthode de revue SystemVerilog en entretien dans un projet SystemVerilog réel ? Réponse : Pour un snippet, je verbalise le hardware ou le rôle simulation-only, puis le risque principal et la méthode de debug.	EN - How would you use SystemVerilog review method in an interview in a real SystemVerilog project? Answer: For a snippet, I verbalize the hardware or simulation-only role, then the main risk and debugging method.
499. FR - Comment diagnostiqueriez-vous un problème lié à méthode de revue SystemVerilog en entretien ? Réponse : Je commence par identifier si le problème vient du type/construct SystemVerilog, du scheduling de simulation, du RTL synthétisé ou du testbench. Pour méthode de revue SystemVerilog en entretien, je regarde ensuite les drivers, les waveforms, les warnings de compilation/lint et la première divergence observable.	EN - How would you diagnose a problem related to SystemVerilog review method in an interview? Answer: I first identify whether the issue comes from the SystemVerilog type/construct, simulation scheduling, synthesized RTL, or the testbench. For SystemVerilog review method in an interview, I then inspect drivers, waveforms, compile/lint warnings, and the first observable divergence.
500. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant méthode de revue SystemVerilog en entretien ? Réponse : Je rends l'intention explicite et je distingue toujours ce qui est synthétisable de ce qui sert uniquement à la verification. Pour méthode de revue SystemVerilog en entretien, j'explique aussi le hardware attendu ou le rôle simulation-only, ainsi que les cas limites importants.	EN - What best practice or interview pitfall should you know regarding SystemVerilog review method in an interview? Answer: I make intent explicit and always distinguish what is synthesizable from what is verification-only. For SystemVerilog review method in an interview, I also explain the expected hardware or simulation-only role, together with important corner cases.

Rappels essentiels / Essential Reminders

FR: Verilog décrit du matériel concurrent; pensez en registres, logique combinatoire, mémoire, clocks et interfaces. EN: Verilog describes concurrent hardware; think in registers, combinational logic, memory, clocks, and interfaces.

FR: `reg` signifie variable procédurale en Verilog classique; il ne garantit pas qu'un registre physique soit inféré. EN: `reg` means a procedural variable in classic Verilog; it does not guarantee that a physical register is inferred.

FR: Règle de style: blocking `=` pour combinatoire procédurale, non-blocking `<=` pour logique séquentielle clockée. EN: Style rule: blocking `=` for procedural combinational logic, non-blocking `<=` for clocked sequential logic.

FR: Un port logique n'est pas un pin physique, et un signal externe n'est pas automatiquement une interruption. EN: A logical port is not a physical pin, and an external signal is not automatically an interrupt.

FR: Simulation fonctionnelle, synthèse, CDC review et static timing analysis répondent à des questions différentes et sont toutes nécessaires. EN: Functional simulation, synthesis, CDC review, and static timing analysis answer different questions and are all necessary.